

JENKINS-TASK - 03

1. Create one Jenkins job using the below code and create three stages.

- **stage1:** Git clone to download the source code.
- **stage2:** Sonarqube Integration to check the quality of code.
- **stage3:** Slack Integration to send the alerts to slack.

Source Code: <https://github.com/betawins/VProfile-1.git>

stage1: Git clone to download the source code.

Step 1: Create an EC2 instance

Go to:

AWS Console → EC2 → Instances → Launch instance

Use:

Setting	Value
Name	Jenkins-Server
AMI	Amazon Linux 2023
Instance type	t3.micro
Key pair	Create/select your key pair
Network	Your VPC
Subnet	Public subnet
Auto-assign Public IP Enable	
Security Group	Create new

Step 2: Configure Security Group

Allow these inbound rules:

Type	Port	Source
SSH	22	My IP
Custom TCP	8080	My IP
HTTP	80	My IP

Port 8080 is required because Jenkins normally runs on port 8080.

For learning, you can temporarily use 0.0.0.0/0 for port 8080, but **My IP** is safer.

Step 3: Connect to the EC2 instance

From your local machine:

```
ssh -i your-key.pem ec2-user@<PUBLIC-IP>
```

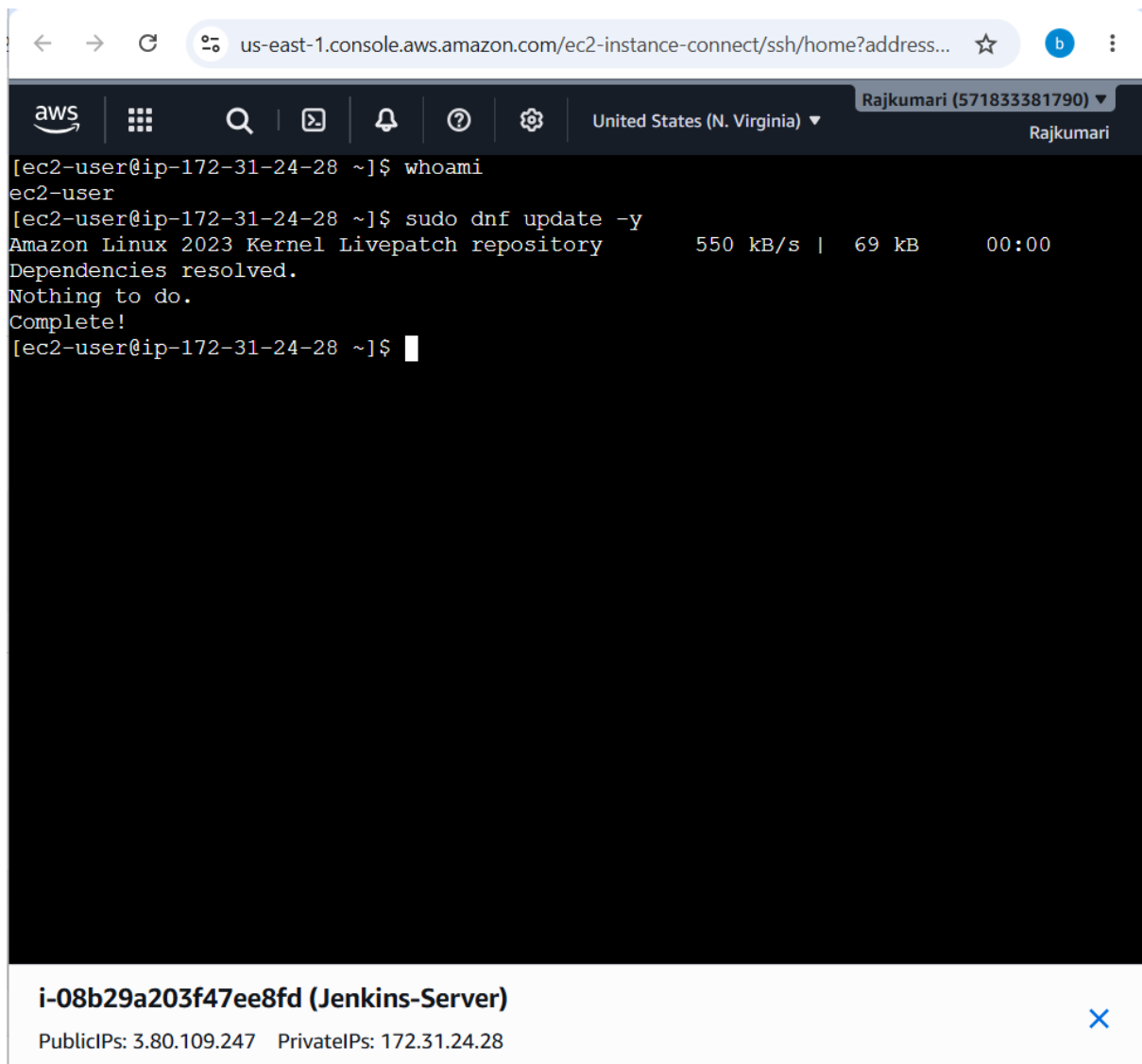
Verify:

```
whoami
```

Expected: ec2-user

Step 4: Update the server

```
Run: sudo dnf update -y
```



```
[ec2-user@ip-172-31-24-28 ~]$ whoami
ec2-user
[ec2-user@ip-172-31-24-28 ~]$ sudo dnf update -y
Amazon Linux 2023 Kernel Livepatch repository    550 kB/s | 69 kB    00:00
Dependencies resolved.
Nothing to do.
Complete!
[ec2-user@ip-172-31-24-28 ~]$
```

i-08b29a203f47ee8fd (Jenkins-Server)

PublicIPs: 3.80.109.247 PrivateIPs: 172.31.24.28

Step 5: Install Java

Jenkins requires Java.

Install Java:

```
sudo dnf install java-21-amazon-corretto -y
```

Check:

```
java -version
```

You should see Java 21 information.

```
aws | [grid icon] | [search icon] | [terminal icon] | [notification icon] | [help icon] | [settings icon] | United States (N. Virginia) ▾ | Rajkumari (571833381790) ▾ | Rajkumari

harfbuzz-7.0.0-2.amzn2023.0.2.x86_64
java-21-amazon-corretto-1:21.0.12+8-1.amzn2023.1.x86_64
java-21-amazon-corretto-headless-1:21.0.12+8-1.amzn2023.1.x86_64
javapackages-filesystem-6.0.0-7.amzn2023.0.6.noarch
langpacks-core-font-en-3.0-21.amzn2023.0.4.noarch
libICE-1.1.1-3.amzn2023.0.1.x86_64
libSM-1.2.4-3.amzn2023.0.1.x86_64
libX11-1.8.10-2.amzn2023.0.1.x86_64
libX11-common-1.8.10-2.amzn2023.0.1.noarch
libXau-1.0.11-6.amzn2023.0.1.x86_64
libXext-1.3.6-1.amzn2023.0.1.x86_64
libXi-1.8.2-1.amzn2023.0.1.x86_64
libXinerama-1.1.5-6.amzn2023.0.1.x86_64
libXrandr-1.5.4-3.amzn2023.0.1.x86_64
libXrender-0.9.11-6.amzn2023.0.1.x86_64
libXt-1.3.0-3.amzn2023.0.1.x86_64
libXtst-1.2.5-1.amzn2023.0.1.x86_64
libbrotli-1.0.9-4.amzn2023.0.2.x86_64
libjpeg-turbo-2.1.4-2.amzn2023.0.5.x86_64
libpng-2:1.6.37-10.amzn2023.0.13.x86_64
libxcb-1.17.0-1.amzn2023.0.1.x86_64
pixman-0.43.4-1.amzn2023.0.4.x86_64
xml-common-0.6.3-56.amzn2023.0.2.noarch

Complete!
[root@ip-172-31-24-28 ~]# java -version
openjdk version "21.0.12" 2026-07-21 LTS
OpenJDK Runtime Environment Corretto-21.0.12.8.1 (build 21.0.12+8-LTS)
OpenJDK 64-Bit Server VM Corretto-21.0.12.8.1 (build 21.0.12+8-LTS, mixed mode, sharing)
[root@ip-172-31-24-28 ~]#
```

Step 6: Install Git

Git is required for **Stage 1**, because Jenkins needs Git to clone the GitHub repository.

```
sudo dnf install git -y
```

Verify:

```
git --version
```

Example:

```
git version 2.x.x
```

```
aws | [grid icon] | [search icon] | [terminal icon] | [bell icon] | [help icon] | [settings icon] | United States (N. Virginia) ▼ | Rajkumari (571833381790) ▼ | Rajkumari

Installing      : git-core-doc-2.50.1-1.amzn2023.0.1.noarch      2/8
Installing      : perl-lib-0.65-477.amzn2023.0.9.x86_64        3/8
Installing      : perl-TermReadKey-2.38-9.amzn2023.0.3.x86_64   4/8
Installing      : perl-File-Find-1.37-477.amzn2023.0.9.noarch   5/8
Installing      : perl-Error-1:0.17030-2.amzn2023.0.1.noarch    6/8
Installing      : perl-Git-2.50.1-1.amzn2023.0.1.noarch         7/8
Installing      : git-2.50.1-1.amzn2023.0.1.x86_64             8/8
Running scriptlet: git-2.50.1-1.amzn2023.0.1.x86_64            8/8
Verifying       : git-2.50.1-1.amzn2023.0.1.x86_64             1/8
Verifying       : git-core-2.50.1-1.amzn2023.0.1.x86_64        2/8
Verifying       : git-core-doc-2.50.1-1.amzn2023.0.1.noarch    3/8
Verifying       : perl-Error-1:0.17030-2.amzn2023.0.1.noarch   4/8
Verifying       : perl-File-Find-1.37-477.amzn2023.0.9.noarch  5/8
Verifying       : perl-Git-2.50.1-1.amzn2023.0.1.noarch        6/8
Verifying       : perl-TermReadKey-2.38-9.amzn2023.0.3.x86_64  7/8
Verifying       : perl-lib-0.65-477.amzn2023.0.9.x86_64       8/8

Installed:
git-2.50.1-1.amzn2023.0.1.x86_64
git-core-2.50.1-1.amzn2023.0.1.x86_64
git-core-doc-2.50.1-1.amzn2023.0.1.noarch
perl-Error-1:0.17030-2.amzn2023.0.1.noarch
perl-File-Find-1.37-477.amzn2023.0.9.noarch
perl-Git-2.50.1-1.amzn2023.0.1.noarch
perl-TermReadKey-2.38-9.amzn2023.0.3.x86_64
perl-lib-0.65-477.amzn2023.0.9.x86_64

Complete!
[root@ip-172-31-24-28 ~]# git --version
git version 2.50.1
[root@ip-172-31-24-28 ~]#
```

Step 7: Install Jenkins

Add the Jenkins repository:

```
sudo wget -O /etc/yum.repos.d/jenkins.repo \
https://pkg.jenkins.io/redhat-stable/jenkins.repo
```

Import the Jenkins key:

```
sudo rpm --import https://pkg.jenkins.io/redhat-stable/jenkins.io-2023.key
```

Install Jenkins:

```
sudo dnf install jenkins -y
```

```
aws | [grid icon] | [search icon] | [terminal icon] | [bell icon] | [help icon] | [gear icon] | United States (N. Virginia) ▼ | Rajkumari (571833381790) ▼ | Rajkumari

[root@ip-172-31-24-28 ~]# sudo wget -O /etc/yum.repos.d/jenkins.repo \
https://pkg.jenkins.io/redhat-stable/jenkins.repo
--2026-08-13 12:46:00-- https://pkg.jenkins.io/redhat-stable/jenkins.repo
Resolving pkg.jenkins.io (pkg.jenkins.io)... 146.75.38.133, 2a04:4e42:79::645
Connecting to pkg.jenkins.io (pkg.jenkins.io)|146.75.38.133|:443... connected.
HTTP request sent, awaiting response... 301 Moved Permanently
Location: https://pkg.jenkins.io/rpm-stable/jenkins.repo [following]
--2026-08-13 12:46:00-- https://pkg.jenkins.io/rpm-stable/jenkins.repo
Reusing existing connection to pkg.jenkins.io:443.
HTTP request sent, awaiting response... 200 OK
Length: 267 [application/octet-stream]
Saving to: '/etc/yum.repos.d/jenkins.repo'

/etc/yum.repos.d/jen 100%[=====>]      267  --.-KB/s    in 0s

2026-08-13 12:46:00 (15.2 MB/s) - '/etc/yum.repos.d/jenkins.repo' saved [267/267]

[root@ip-172-31-24-28 ~]# sudo rpm --import https://pkg.jenkins.io/redhat-stable/jen
kins.io-2023.key
[root@ip-172-31-24-28 ~]# sudo dnf install jenkins -y
Jenkins-stable                11 kB/s | 833 B      00:00
Jenkins-stable                108 kB/s | 1.6 kB    00:00
Importing GPG key 0x14ABFC68:
  Userid      : "Jenkins Project <jenkinsci-board@googlegroups.com>"
  Fingerprint: 5E38 6EAD B55F 0150 4CAE 8BCF 7198 F4B7 14AB FC68
  From        : https://pkg.jenkins.io/rpm-stable/repodata/repomd.xml.key
Jenkins-stable                22 kB/s | 2.6 kB    00:00
Dependencies resolved.

=====
Package                Architecture      Version           Repository        Size
=====
```

```
aws | [grid icon] | [search icon] | [terminal icon] | [bell icon] | [help icon] | [settings icon] | United States (N. Virginia) ▼ | Rajkumari (571833381790) ▼ | Rajkumari

=====
Install 1 Package

Total download size: 96 M
Installed size: 96 M
Downloading Packages:
jenkins-2.568.2-1.noarch.rpm                                40 MB/s | 96 MB    00:02
=====
Total                                                    40 MB/s | 96 MB    00:02
Jenkins-stable                                           109 kB/s | 1.6 kB   00:00
Importing GPG key 0x14ABFC68:
  Userid      : "Jenkins Project <jenkinsci-board@googlegroups.com>"
  Fingerprint: 5E38 6EAD B55F 0150 4CAE 8BCF 7198 F4B7 14AB FC68
  From        : https://pkg.jenkins.io/rpm-stable/repodata/repomd.xml.key
Key imported successfully
Running transaction check
Transaction check succeeded.
Running transaction test
Transaction test succeeded.
Running transaction
  Preparing      :                                1/1
  Running scriptlet: jenkins-2.568.2-1.noarch    1/1
  Installing     : jenkins-2.568.2-1.noarch    1/1
  Running scriptlet: jenkins-2.568.2-1.noarch    1/1
  Verifying      : jenkins-2.568.2-1.noarch    1/1

Installed:
jenkins-2.568.2-1.noarch

Complete!
[root@ip-172-31-24-28 ~]#
```

Step 8: Start Jenkins

`sudo systemctl start jenkins`

Enable Jenkins at boot:

`sudo systemctl enable jenkins`

Check status:

`sudo systemctl status jenkins`

You should see:

Active: active (running)

```
aws | [grid icon] | [search icon] | [terminal icon] | [bell icon] | [help icon] | [gear icon] | United States (N. Virginia) ▼ | Rajkumari (571833381790) ▼ | Rajkumari

[root@ip-172-31-24-28 ~]# sudo systemctl start jenkins
[root@ip-172-31-24-28 ~]# sudo systemctl enable jenkins
Created symlink /etc/systemd/system/multi-user.target.wants/jenkins.service → /usr/lib/systemd/system/jenkins.service.
[root@ip-172-31-24-28 ~]# exit
logout
[ec2-user@ip-172-31-24-28 ~]$ sudo systemctl status jenkins
● jenkins.service - Jenkins Continuous Integration Server
   Loaded: loaded (/usr/lib/systemd/system/jenkins.service; enabled; preset: disabled)
   Active: active (running) since Thu 2026-08-13 12:49:13 UTC; 25s ago
     Main PID: 28064 (java)
        Tasks: 43 (limit: 1059)
      Memory: 396.6M
         CPU: 20.190s
       CGroup: /system.slice/jenkins.service
               └─28064 /usr/bin/java -Djava.awt.headless=true -jar /usr/share/java/jenkins.war

Aug 13 12:49:09 ip-172-31-24-28.ec2.internal jenkins[28064]: [LF]>
Aug 13 12:49:09 ip-172-31-24-28.ec2.internal jenkins[28064]: [LF]> *****>
Aug 13 12:49:09 ip-172-31-24-28.ec2.internal jenkins[28064]: [LF]> *****>
Aug 13 12:49:09 ip-172-31-24-28.ec2.internal jenkins[28064]: [LF]> *****>
Aug 13 12:49:13 ip-172-31-24-28.ec2.internal jenkins[28064]: 2026-08-13 12:49:13.92>
Aug 13 12:49:13 ip-172-31-24-28.ec2.internal jenkins[28064]: 2026-08-13 12:49:13.94>
Aug 13 12:49:13 ip-172-31-24-28.ec2.internal systemd[1]: Started jenkins.service - >
Aug 13 12:49:14 ip-172-31-24-28.ec2.internal jenkins[28064]: 2026-08-13 12:49:14.28>
Aug 13 12:49:14 ip-172-31-24-28.ec2.internal jenkins[28064]: 2026-08-13 12:49:14.29>
Aug 13 12:49:19 ip-172-31-24-28.ec2.internal jenkins[28064]: 2026-08-13 12:49:19.02>

[ec2-user@ip-172-31-24-28 ~]$
```

Step 9: Open Jenkins in the browser

Get the EC2 public IP:

`curl ifconfig.me`

Suppose the IP is:

13.xx.xx.xx

Open:

`http://13.xx.xx.xx:8080`

You should see the Jenkins setup page

Step 10: Get the initial Jenkins password

On the EC2 server:

`sudo cat /var/lib/jenkins/secrets/initialAdminPassword`

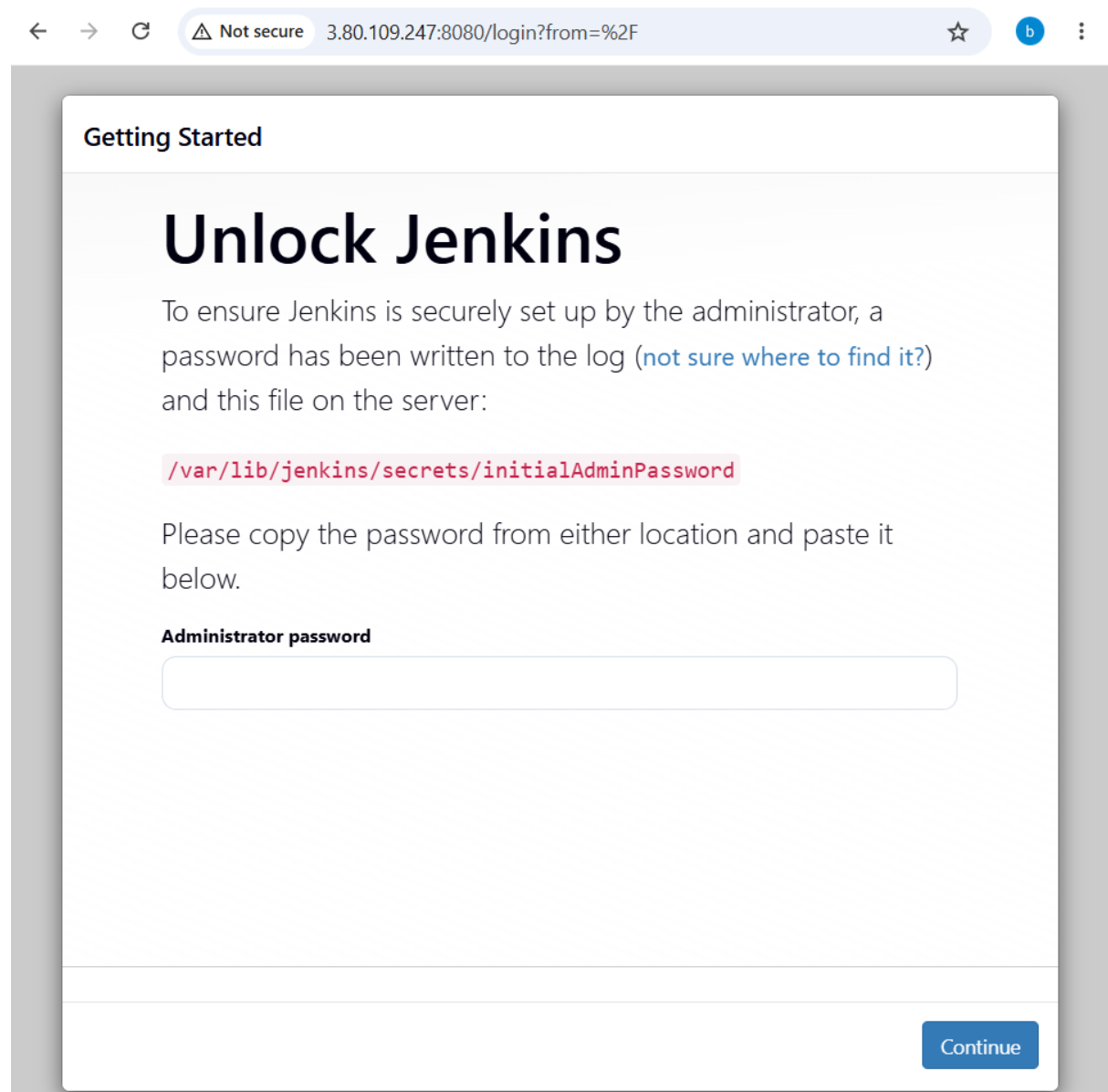
Copy the password.

Paste it into the Jenkins browser page.

Then select:

Install suggested plugins

Create your Jenkins administrator account.



The screenshot shows a web browser window with the address bar displaying "3.80.109.247:8080/login?from=%2F". The page title is "Getting Started". The main heading is "Unlock Jenkins". Below the heading, the text reads: "To ensure Jenkins is securely set up by the administrator, a password has been written to the log (not sure where to find it?) and this file on the server: /var/lib/jenkins/secrets/initialAdminPassword". The path is highlighted in red. Below this, the text says: "Please copy the password from either location and paste it below." There is a label "Administrator password" above a text input field. At the bottom right, there is a blue "Continue" button.

Getting Started

Unlock Jenkins

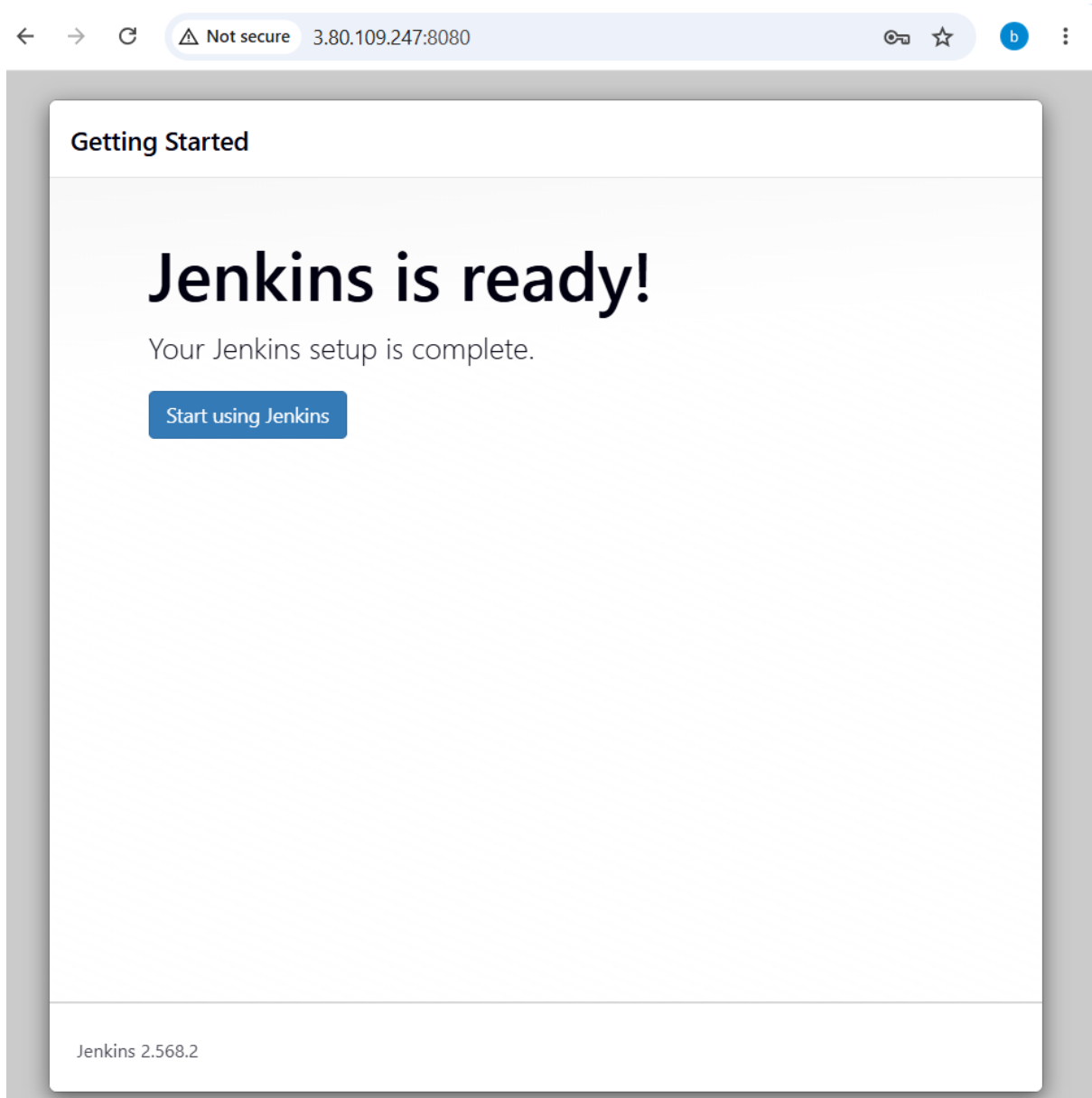
To ensure Jenkins is securely set up by the administrator, a password has been written to the log (not sure where to find it?) and this file on the server:

```
/var/lib/jenkins/secrets/initialAdminPassword
```

Please copy the password from either location and paste it below.

Administrator password

Continue



Step 11: Create the Jenkins Pipeline job

In Jenkins:

Dashboard → New Item

Enter: VProfile

Select: Freestyle Job

Click: **OK**

Step 12: Configure Source Code Management

Scroll to:

Source Code Management

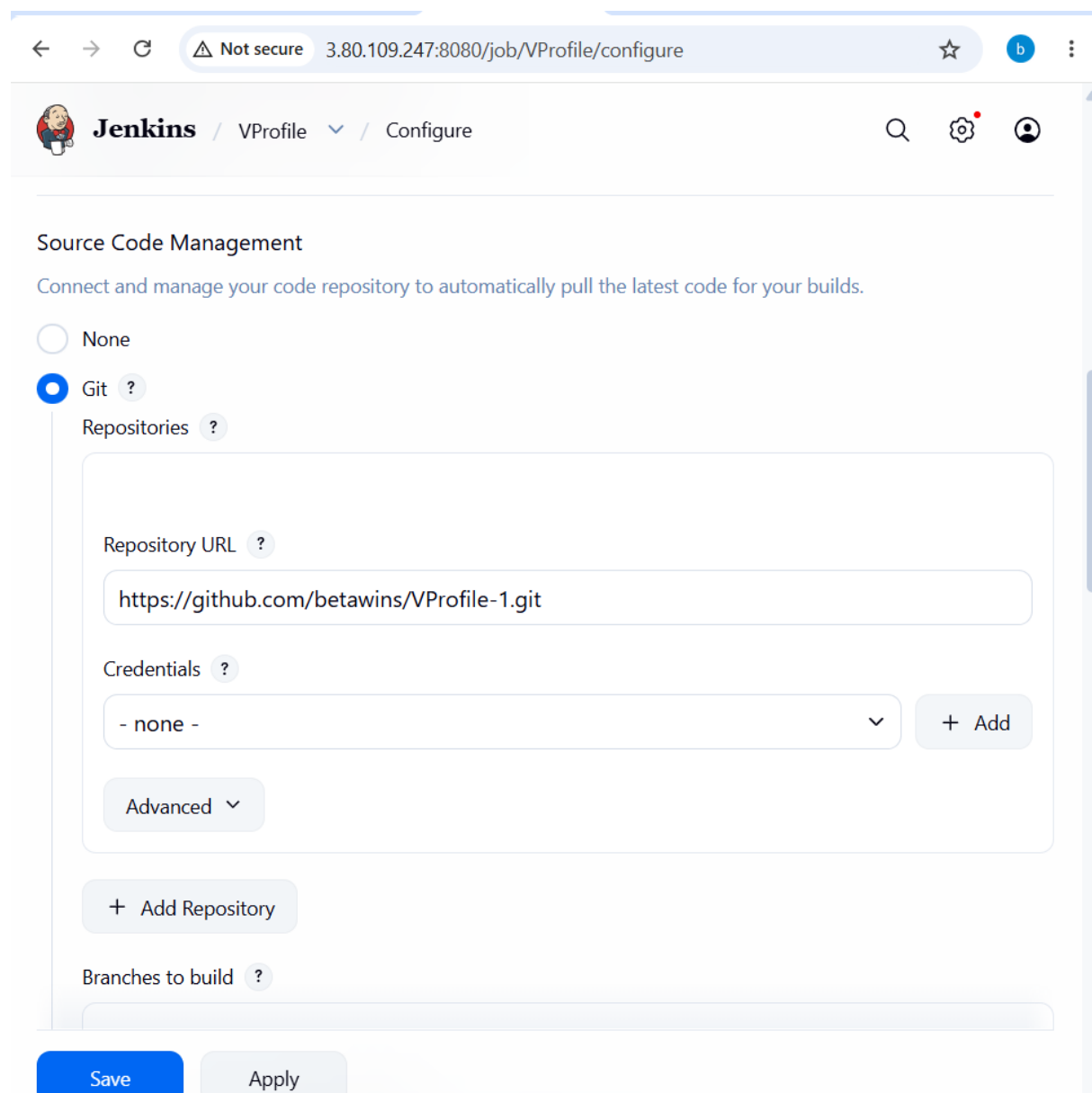
Select:

Git

For **Repository URL**, enter:

`https://github.com/betawins/VProfile-1.git`

This repository is public, so you don't need GitHub credentials for cloning it.



The screenshot shows the Jenkins web interface for configuring a job named 'VProfile'. The browser address bar indicates the URL '3.80.109.247:8080/job/VProfile/configure'. The Jenkins logo and job name 'VProfile' are in the top left, and search, settings, and user icons are in the top right. The 'Source Code Management' section is active, with a description: 'Connect and manage your code repository to automatically pull the latest code for your builds.' Two radio buttons are present: 'None' (unselected) and 'Git' (selected). Below the 'Git' button is a 'Repositories' section with a help icon. It contains a 'Repository URL' field with the value 'https://github.com/betawins/VProfile-1.git', a 'Credentials' dropdown menu currently showing '- none -' with an 'Add' button, and an 'Advanced' expandable section. At the bottom of the repository list is a '+ Add Repository' button. Below the repository list is a 'Branches to build' field. At the very bottom of the configuration page are 'Save' and 'Apply' buttons.

Step 13: Build the job

Click:

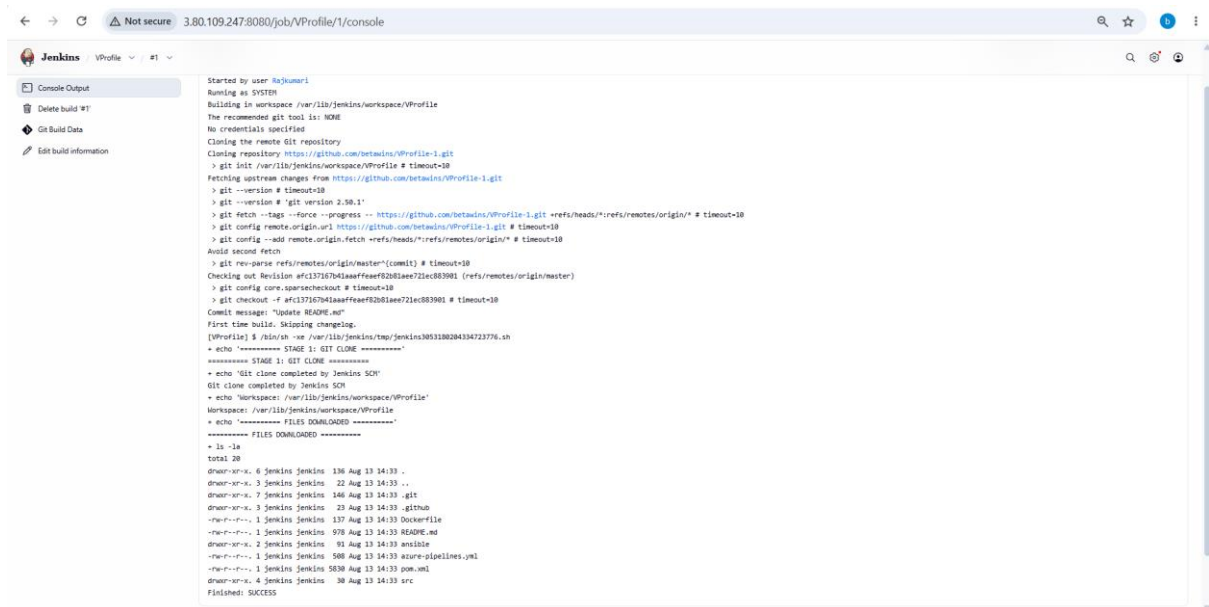
Build Now

You should get:

Build #1

Click:

Build #1 → Console Output

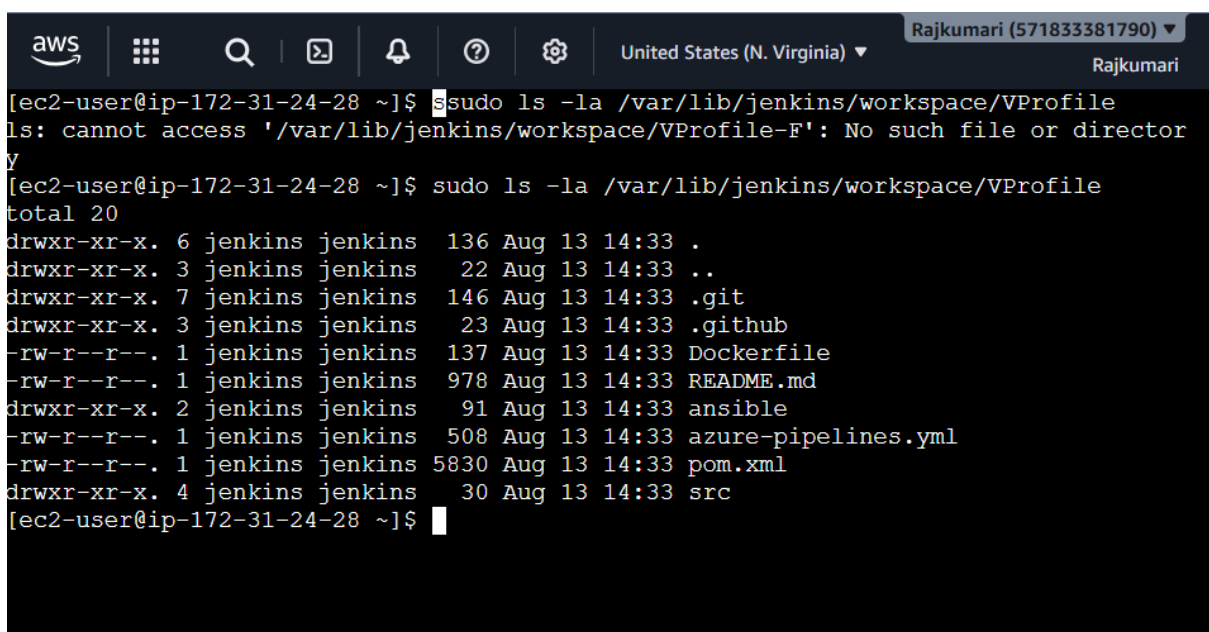


```
Started by user Rajkumari
Running as SYSTEM
Building in workspace /var/lib/jenkins/workspace/VProfile
The recommended git tool is: NONE
No credentials specified
Cloning the remote Git repository
Cloning repository https://github.com/betadins/VProfile-1.git
> git init /var/lib/jenkins/workspace/VProfile # timeout=10
Fetching upstream changes from https://github.com/betadins/VProfile-1.git
> git --version # timeout=10
> git --version # 'git version 2.58.1'
> git fetch --tags --force --progress -- https://github.com/betadins/VProfile-1.git --refs/heads/*:refs/remotes/origin/* # timeout=10
> git config remote.origin.url https://github.com/betadins/VProfile-1.git # timeout=10
> git config --add remote.origin.fetch +refs/heads/*:refs/remotes/origin/* # timeout=10
Avoid second fetch
> git rev-parse refs/remotes/origin/master^{commit} # timeout=10
Checking out Revision af13716704aaaf9201ee721ec83901 (refs/remotes/origin/master)
> git config core.sparsecheckout # timeout=10
> git checkout -f af13716704aaaf9201ee721ec83901 # timeout=10
Commit message: "Update README.md"
First time build. Skipping changelog.
[VPProfile] $ /bin/sh -oe /var/lib/jenkins/tmp/jenkins3853180284334723776.sh
+ echo '***** STAGE 1: GET CLONE *****'
***** STAGE 1: GET CLONE *****
+ echo 'git clone completed by Jenkins SH'
git clone completed by Jenkins SH
+ echo 'workspace: /var/lib/jenkins/workspace/VProfile'
workspace: /var/lib/jenkins/workspace/VProfile
+ echo '***** FILES DOWNLOADED *****'
***** FILES DOWNLOADED *****
+ ls -la
total 20
drwxr-xr-x. 6 jenkins jenkins 136 Aug 13 14:33 .
drwxr-xr-x. 3 jenkins jenkins 22 Aug 13 14:33 ..
drwxr-xr-x. 7 jenkins jenkins 146 Aug 13 14:33 .git
drwxr-xr-x. 3 jenkins jenkins 23 Aug 13 14:33 .github
-rw-r--r--. 1 jenkins jenkins 137 Aug 13 14:33 Dockerfile
-rw-r--r--. 1 jenkins jenkins 978 Aug 13 14:33 README.md
drwxr-xr-x. 2 jenkins jenkins 91 Aug 13 14:33 ansible
-rw-r--r--. 1 jenkins jenkins 508 Aug 13 14:33 azure-pipelines.yml
-rw-r--r--. 1 jenkins jenkins 5830 Aug 13 14:33 pom.xml
drwxr-xr-x. 4 jenkins jenkins 30 Aug 13 14:33 src
Finished: SUCCESS
```

1. Verify the workspace

On your EC2 instance:

```
sudo ls -la /var/lib/jenkins/workspace/VProfile
```



```
aws | United States (N. Virginia) | Rajkumari (571833381790) | Rajkumari
[ec2-user@ip-172-31-24-28 ~]$ sudo ls -la /var/lib/jenkins/workspace/VProfile
ls: cannot access '/var/lib/jenkins/workspace/VProfile-F': No such file or director
[ec2-user@ip-172-31-24-28 ~]$ sudo ls -la /var/lib/jenkins/workspace/VProfile
total 20
drwxr-xr-x. 6 jenkins jenkins 136 Aug 13 14:33 .
drwxr-xr-x. 3 jenkins jenkins 22 Aug 13 14:33 ..
drwxr-xr-x. 7 jenkins jenkins 146 Aug 13 14:33 .git
drwxr-xr-x. 3 jenkins jenkins 23 Aug 13 14:33 .github
-rw-r--r--. 1 jenkins jenkins 137 Aug 13 14:33 Dockerfile
-rw-r--r--. 1 jenkins jenkins 978 Aug 13 14:33 README.md
drwxr-xr-x. 2 jenkins jenkins 91 Aug 13 14:33 ansible
-rw-r--r--. 1 jenkins jenkins 508 Aug 13 14:33 azure-pipelines.yml
-rw-r--r--. 1 jenkins jenkins 5830 Aug 13 14:33 pom.xml
drwxr-xr-x. 4 jenkins jenkins 30 Aug 13 14:33 src
[ec2-user@ip-172-31-24-28 ~]$
```

Stage2: Sonarqube Integration to check the quality of code.

Source Code:<https://github.com/betawins/VProfile-1.git>

Step 1: Create a SonarQube EC2 instance

Go to:

AWS Console → EC2 → Instances → Launch instance

Use:

Setting	Value
Name	SonarQube-Server
AMI	Amazon Linux 2023
Instance type	t3.medium
Key pair	Create/select your key pair
Network	Your VPC
Subnet	Public subnet
Auto-assign Public IP Enable	
Security Group	Create new

SonarQube needs more resources than Jenkins. For a lab, t3.medium is a reasonable starting point.

Step 2: Configure the SonarQube Security Group

Add these inbound rules:

Type **Port Source**

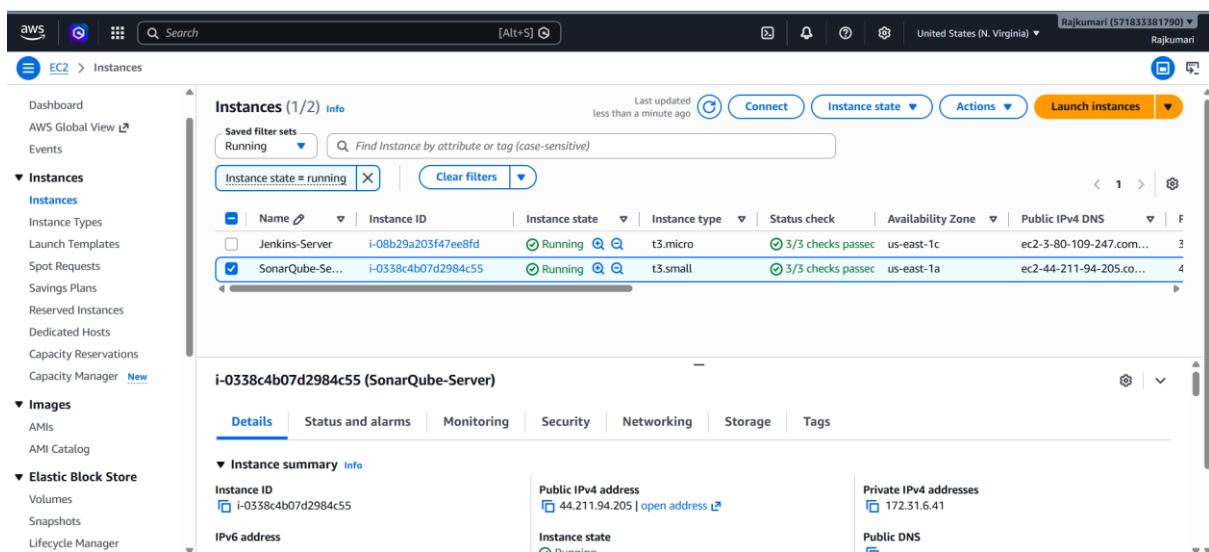
SSH 22 My IP

Custom TCP 9000 My IP

Custom TCP 9000 Jenkins Security Group

Port **9000** is the default SonarQube web interface port.

For Jenkins → SonarQube communication, allowing port 9000 from the **Jenkins Security Group** is preferable.



Step 3: Connect to the SonarQube server

Connect using SSH:

```
ssh -i your-key.pem ec2-user@<SONARQUBE-PUBLIC-IP>
```

Verify the operating system:

```
cat /etc/os-release
```

Step 4: Install Java

Update packages:

```
sudo dnf update -y
```

Install Java:

```
sudo dnf install java-21-amazon-corretto -y
```

Verify:

java -version

You should see Java 21.

```
aws [Search] [Alt+S] United States (N. Virginia)
Verifying : libjpeg-turbo-2.1.4-2.amzn2023.0.5.x86_64
Verifying : libpng-2:1.6.37-10.amzn2023.0.13.x86_64
Verifying : libxcb-1.17.0-1.amzn2023.0.1.x86_64
Verifying : pixman-0.43.4-1.amzn2023.0.4.x86_64
Verifying : xml-common-0.6.3-56.amzn2023.0.2.noarch

Installed:
alsa-lib-1.2.7.2-1.amzn2023.0.3.x86_64
dejavu-sans-fonts-2.37-16.amzn2023.0.2.noarch
dejavu-serif-fonts-2.37-16.amzn2023.0.2.noarch
fonts-filesystem-1:2.0.5-12.amzn2023.0.2.noarch
giflib-5.2.1-9.amzn2023.0.4.x86_64
google-noto-sans-vf-fonts-20240401-1.amzn2023.0.2.noarch
harfbuzz-7.0.0-2.amzn2023.0.2.x86_64
java-21-amazon-corretto-headless-1:21.0.12+8-1.amzn2023.1.x86_64
langpacks-core-font-en-3.0-21.amzn2023.0.4.noarch
libSM-1.2.4-3.amzn2023.0.1.x86_64
libX11-common-1.8.10-2.amzn2023.0.1.noarch
libXext-1.3.6-1.amzn2023.0.1.x86_64
libXinerama-1.1.5-6.amzn2023.0.1.x86_64
libXrender-0.9.11-6.amzn2023.0.1.x86_64
libXtst-1.2.5-1.amzn2023.0.1.x86_64
libjpeg-turbo-2.1.4-2.amzn2023.0.5.x86_64
libxcb-1.17.0-1.amzn2023.0.1.x86_64
xml-common-0.6.3-56.amzn2023.0.2.noarch
cairo-1.18.0-4.amzn2023.0.3.x86_64
dejavu-sans-mono-fonts-2.37-16.amzn2023.0.2.noarch
fontconfig-2.13.94-2.amzn2023.0.2.x86_64
freetype-2.13.2-5.amzn2023.0.2.x86_64
google-noto-fonts-common-20240401-1.amzn2023.0.2.noarch
graphite2-1.3.14-7.amzn2023.0.3.x86_64
java-21-amazon-corretto-1:21.0.12+8-1.amzn2023.1.x86_64
javapackages-filesystem-6.0.0-7.amzn2023.0.6.noarch
libICE-1.1.1-3.amzn2023.0.1.x86_64
libX11-1.8.10-2.amzn2023.0.1.x86_64
libXau-1.0.11-6.amzn2023.0.1.x86_64
libXi-1.8.2-1.amzn2023.0.1.x86_64
libXrandr-1.5.4-3.amzn2023.0.1.x86_64
libXt-1.3.0-3.amzn2023.0.1.x86_64
libbrotli-1.0.9-4.amzn2023.0.2.x86_64
libpng-2:1.6.37-10.amzn2023.0.13.x86_64
pixman-0.43.4-1.amzn2023.0.4.x86_64

Complete!
[ec2-user@ip-172-31-6-41 ~]$ java --version
openjdk 21.0.12 2026-07-21 LTS
OpenJDK Runtime Environment Corretto-21.0.12.8.1 (build 21.0.12+8-LTS)
OpenJDK 64-Bit Server VM Corretto-21.0.12.8.1 (build 21.0.12+8-LTS, mixed mode, sharing)
[ec2-user@ip-172-31-6-41 ~]$
```

Step 5: Download and install SonarQube

Download a compatible SonarQube Community Build release from the official SonarQube distribution page.

Download directly on your EC2

```
cd /opt
```

```
sudo wget https://binaries.sonarsource.com/Distribution/sonarqube/sonarqube-26.1.0.118079.zip
```

```
sudo unzip sonarqube-26.1.0.118079.zip -d
```

Rename the directory:

```
sudo mv sonarqube-26.1.0.118079 sonarqube
```

Create a dedicated SonarQube user:

```
sudo useradd sonar
```

Give ownership:

```
sudo chown -R sonar:sonar /opt/sonarqube
```

```
aws | United States (N. Virginia) | Rajkumari (571833381790) | Rajkumari

creating: sonarqube-26.1.0.118079/web/WEB-INF/classes/com/
creating: sonarqube-26.1.0.118079/web/WEB-INF/classes/com/sonarsource/
inflating: sonarqube-26.1.0.118079/web/WEB-INF/classes/com/sonarsource/branding
creating: sonarqube-26.1.0.118079/lib/jdbc/
creating: sonarqube-26.1.0.118079/lib/jdbc/mssql/
inflating: sonarqube-26.1.0.118079/lib/jdbc/mssql/mssql-jdbc-12.10.2.jre11.jar
inflating: sonarqube-26.1.0.118079/lib/jdbc/mssql/msal4j-1.23.1.jar
inflating: sonarqube-26.1.0.118079/lib/jdbc/mssql/slf4j-api-2.0.17.jar
inflating: sonarqube-26.1.0.118079/lib/jdbc/mssql/azure-json-1.4.0.jar
creating: sonarqube-26.1.0.118079/lib/jdbc/postgresql/
inflating: sonarqube-26.1.0.118079/lib/jdbc/postgresql/postgresql-42.7.8.jar
creating: sonarqube-26.1.0.118079/lib/jdbc/h2/
inflating: sonarqube-26.1.0.118079/lib/jdbc/h2/h2-2.3.232.jar
inflating: sonarqube-26.1.0.118079/lib/sonar-shutdowner-26.1.0.118079.jar
creating: sonarqube-26.1.0.118079/elasticsearch/plugins/
[ec2-user@ip-172-31-6-41 opt]$ ll
total 888028
drwxr-xr-x.  4 root root      33 Aug  1 04:14 aws
drwxr-xr-x. 13 root root    16384 Jan  6 2026 sonarqube-26.1.0.118079
-rw-r--r--.  1 root root 909320623 Jan  6 2026 sonarqube-26.1.0.118079.zip
[ec2-user@ip-172-31-6-41 opt]$ mv ^C
[ec2-user@ip-172-31-6-41 opt]$ mv sonarqube-26.1.0.118079.zip sonarqube
mv: cannot move 'sonarqube-26.1.0.118079.zip' to 'sonarqube': Permission denied
[ec2-user@ip-172-31-6-41 opt]$ sudo mv sonarqube-26.1.0.118079.zip sonarqube
[ec2-user@ip-172-31-6-41 opt]$ ll
total 888028
drwxr-xr-x.  4 root root      33 Aug  1 04:14 aws
-rw-r--r--.  1 root root 909320623 Jan  6 2026 sonarqube
drwxr-xr-x. 13 root root    16384 Jan  6 2026 sonarqube-26.1.0.118079
[ec2-user@ip-172-31-6-41 opt]$
```

i-0338c4b07d2984c55 (SonarQube-Server)

PublicIPs: 44.211.94.205 PrivateIPs: 172.31.6.41

Step 6: Start SonarQube

Switch to the SonarQube user:

```
sudo su - sonar
```

Start SonarQube:

```
/opt/sonarqube/bin/linux-x86-64/sonar.sh start
```

Check the status:

```
/opt/sonarqube/bin/linux-x86-64/sonar.sh status
```

You should see:

SonarQube is running


```
aws | Search [Alt+S] | [Icons] | Unit
[ec2-user@ip-172-31-6-41 opt]$ ll
total 16
drwxr-xr-x. 4 root root 33 Aug 1 04:14 aws
drwxr-xr-x. 13 sonar sonar 16384 Jan 6 2026 sonarqube
[ec2-user@ip-172-31-6-41 opt]$ sudo su - sonar
Last login: Thu Aug 13 15:39:54 UTC 2026 on pts/1
[sonar@ip-172-31-6-41 ~]$ ll
total 0
[sonar@ip-172-31-6-41 ~]$ cd opt
-bash: cd: opt: No such file or directory
[sonar@ip-172-31-6-41 ~]$ cd /opt
[sonar@ip-172-31-6-41 opt]$ ll
total 16
drwxr-xr-x. 4 root root 33 Aug 1 04:14 aws
drwxr-xr-x. 13 sonar sonar 16384 Jan 6 2026 sonarqube
[sonar@ip-172-31-6-41 opt]$ cd sonarqube
[sonar@ip-172-31-6-41 sonarqube]$ ll
total 128
-rw-r--r--. 1 root root 7796 Jan 6 2026 COPYING
drwxr-xr-x. 6 root root 117 Jan 6 2026 bin
drwxr-xr-x. 2 root root 30 Jan 6 2026 conf
drwxr-xr-x. 2 root root 24 Jan 6 2026 data
-rw-r--r--. 1 root root 87874 Jan 6 2026 dependency-license.json
drwxr-xr-x. 7 root root 132 Jan 6 2026 elasticsearch
drwxr-xr-x. 4 root root 40 Jan 6 2026 extensions
drwxr-xr-x. 2 root root 16384 Jan 6 2026 jres
drwxr-xr-x. 5 root root 136 Jan 6 2026 lib
drwxr-xr-x. 2 root root 24 Jan 6 2026 logs
drwxr-xr-x. 2 root root 68 Jan 6 2026 security
drwxr-xr-x. 2 root root 24 Jan 6 2026 temp
drwxr-xr-x. 7 root root 16384 Jan 6 2026 web
[sonar@ip-172-31-6-41 sonarqube]$
```

i-0338c4b07d2984c55 (SonarQube-Server)

PublicIPs: 44.211.94.205 PrivateIPs: 172.31.6.41

```
aws | [Icons] | [Icons] | [Icons] | [Icons] | [Icons] | United States (N. Virginia) | Rajkumari (571833381790) | Rajkumari
total 16
drwxr-xr-x. 4 root root 33 Aug 1 04:14 aws
drwxr-xr-x. 13 sonar sonar 16384 Jan 6 2026 sonarqube
[sonar@ip-172-31-6-41 opt]$ cd sonarqube
[sonar@ip-172-31-6-41 sonarqube]$ ll
total 128
-rw-r--r--. 1 root root 7796 Jan 6 2026 COPYING
drwxr-xr-x. 6 root root 117 Jan 6 2026 bin
drwxr-xr-x. 2 root root 30 Jan 6 2026 conf
drwxr-xr-x. 2 root root 24 Jan 6 2026 data
-rw-r--r--. 1 root root 87874 Jan 6 2026 dependency-license.json
drwxr-xr-x. 7 root root 132 Jan 6 2026 elasticsearch
drwxr-xr-x. 4 root root 40 Jan 6 2026 extensions
drwxr-xr-x. 2 root root 16384 Jan 6 2026 jres
drwxr-xr-x. 5 root root 136 Jan 6 2026 lib
drwxr-xr-x. 2 root root 24 Jan 6 2026 logs
drwxr-xr-x. 2 root root 68 Jan 6 2026 security
drwxr-xr-x. 2 root root 24 Jan 6 2026 temp
drwxr-xr-x. 7 root root 16384 Jan 6 2026 web
[sonar@ip-172-31-6-41 sonarqube]$ cd bin
[sonar@ip-172-31-6-41 bin]$ ls
elasticsearch linux-x86-64 macosx-universal-64 windows-x86-64 winsw-license
[sonar@ip-172-31-6-41 bin]$ cd linux-x86-64
[sonar@ip-172-31-6-41 linux-x86-64]$ ls
sonar.sh
[sonar@ip-172-31-6-41 linux-x86-64]$ sonar.sh start
-bash: sonar.sh: command not found
[sonar@ip-172-31-6-41 linux-x86-64]$ ./sonar.sh
/usr/bin/java
Usage: ./sonar.sh { console | start | stop | force-stop | restart | status | dump }
```

i-0338c4b07d2984c55 (SonarQube-Server)

PublicIPs: 44.211.94.205 PrivateIPs: 172.31.6.41



```
aws | [grid icon] | [search icon] | [terminal icon] | [bell icon] | [help icon] | [settings icon] | United States (N. Virginia) ▼ | Rajkumari (571833381790) ▼ | Rajkumari

[sonar@ip-172-31-6-41 opt]$ exit
logout
[ec2-user@ip-172-31-6-41 opt]$ cd
[ec2-user@ip-172-31-6-41 ~]$ sudo chown -R sonar:sonar /opt/sonarqube
[ec2-user@ip-172-31-6-41 ~]$ ls -ld /opt/sonarqube
ls -ld /opt/sonarqube/logs
ls -ld /opt/sonarqube/data
ls -ld /opt/sonarqube/bin
drwxr-xr-x. 13 sonar sonar 16384 Jan  6  2026 /opt/sonarqube
drwxr-xr-x.  2 sonar sonar 24 Jan  6  2026 /opt/sonarqube/logs
drwxr-xr-x.  2 sonar sonar 24 Jan  6  2026 /opt/sonarqube/data
drwxr-xr-x.  6 sonar sonar 117 Jan  6  2026 /opt/sonarqube/bin
[ec2-user@ip-172-31-6-41 ~]$ sudo su - sonar
Last login: Thu Aug 13 16:00:13 UTC 2026 on pts/1
[sonar@ip-172-31-6-41 ~]$ cd /opt/sonarqube/bin/linux-x86-64
[sonar@ip-172-31-6-41 linux-x86-64]$ ls
sonar.sh
[sonar@ip-172-31-6-41 linux-x86-64]$ ./sonar.sh status
/usr/bin/java
SonarQube is not running.
[sonar@ip-172-31-6-41 linux-x86-64]$ ./sonar.sh start
/usr/bin/java
Starting SonarQube...
Started SonarQube.
[sonar@ip-172-31-6-41 linux-x86-64]$
```

i-0338c4b07d2984c55 (SonarQube-Server)

PublicIPs: 44.211.94.205 PrivateIPs: 172.31.6.41



Step 7: Access SonarQube

Open your browser:

<http://<SONARQUBE-PUBLIC-IP>:9000>

Example:

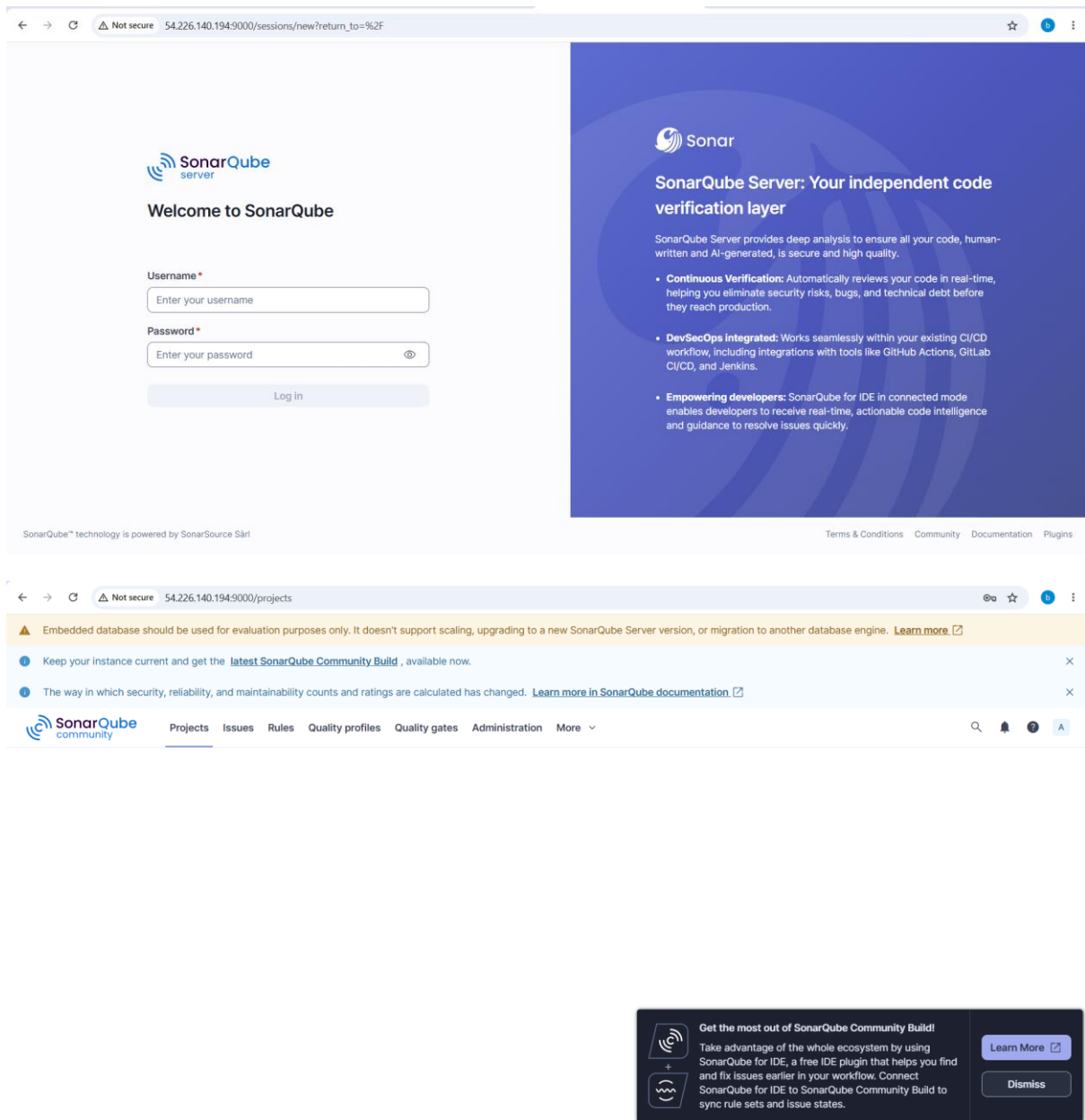
<http://13.234.XX.XX:9000>

Default login:

Username: admin

Password: admin

SonarQube will ask you to change the default password.



Step 8: Create a SonarQube project

In SonarQube, go to:

Projects → Create Project

Enter:

Project display name: VProfile

Project key: vprofile

Select the option to set up the project manually if prompted.

Your project will be used by Jenkins to send the source-code analysis results.

1 of 2

Create a local project

Project display name * ⓘ

Project key * ⓘ

Main branch name *

The name of your project's default branch [Learn More](#) ⓘ

Cancel

Next

← → ↻ ⚠ Not secure 54.226.140.194:9000/projects/create?mode=manual&setncd... 🔑 ☆ b ⋮

⚠ Embedded database should be used for evaluation purposes only. It doesn't support scaling, upgrading to a r

 **SonarQube**
community

Projects Issues Rules Quality profiles Quality gates Administration

2 of 2

Set up new code for project

The new code definition sets the criteria for what's considered new code, allowing you to focus on the most recent changes in your project.

This setting allows project administrators to customize the new code definition for their project. [Learn more in documentation](#) 

Follows the instance's default

Current default: Previous version

Is custom

Choose whether it should be the **previous version**, a **number of days** or a **reference branch**

[Back](#) [Create project](#)

SonarQube™ technology is powered by SonarSource Sàrl Community Build • v26.7.0.124771 • MQR MODE

Step 9: Generate a SonarQube token

Go to:

SonarQube → Administration/User → My Account → Security

Under **Tokens**, create:

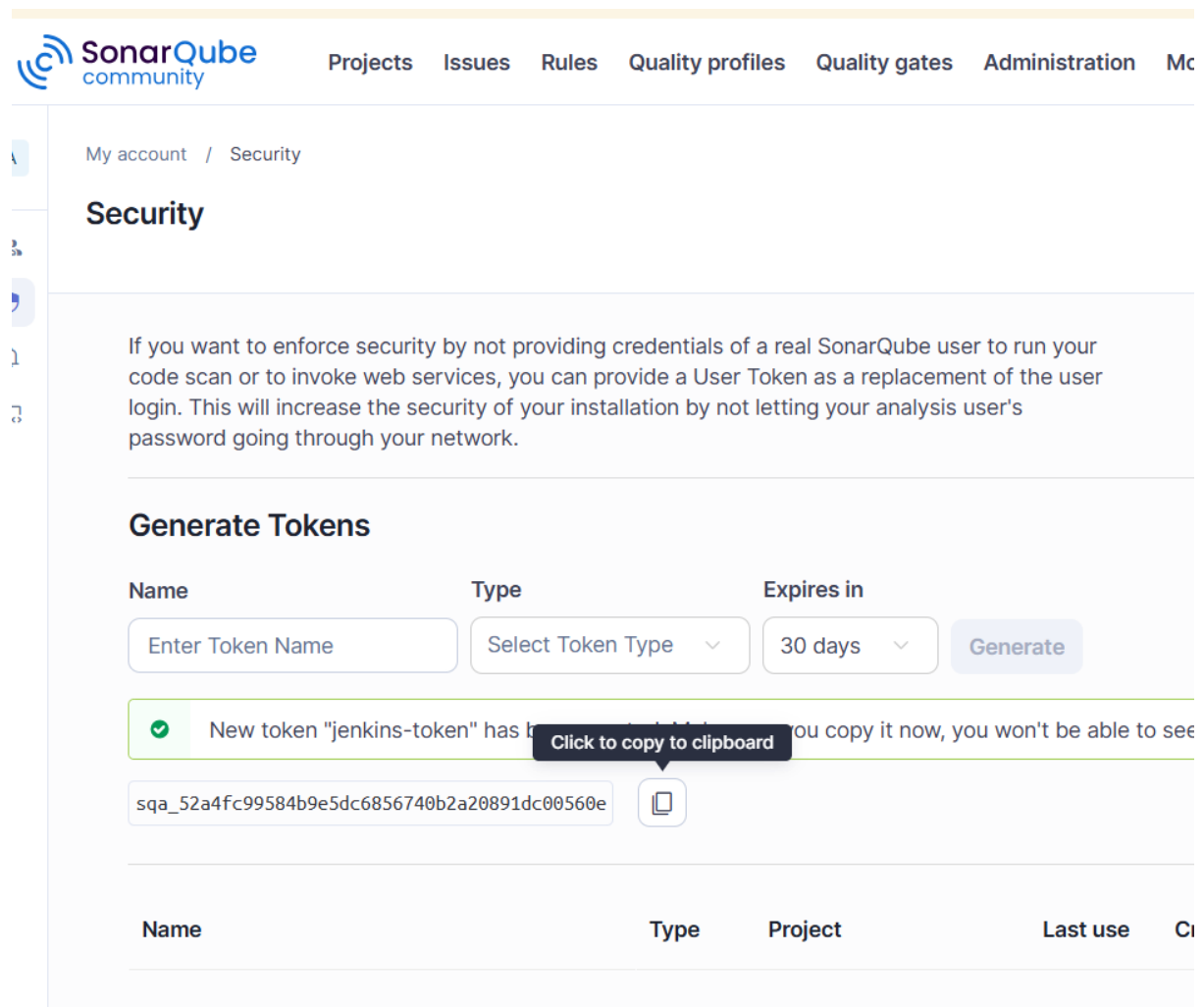
Name: jenkins-token

Click:

Generate

Copy the generated token.

Do not put this token directly into your Jenkins job configuration or source code. Store it securely in Jenkins Credentials.



The screenshot shows the SonarQube community interface. The top navigation bar includes links for Projects, Issues, Rules, Quality profiles, Quality gates, Administration, and My account. The left sidebar shows a navigation menu with 'My account' and 'Security' selected. The main content area is titled 'Security' and contains a message about enforcing security by using User Tokens. Below this is a 'Generate Tokens' section with a form. The form has three input fields: 'Name' (containing 'Enter Token Name'), 'Type' (a dropdown menu showing 'Select Token Type'), and 'Expires in' (a dropdown menu showing '30 days'). A 'Generate' button is to the right of these fields. Below the form, a green success message states: 'New token "jenkins-token" has been generated. If you copy it now, you won't be able to see it again.' A tooltip 'Click to copy to clipboard' points to a copy icon next to the token value 'sqa_52a4fc99584b9e5dc6856740b2a20891dc00560e'. Below the message is a table with columns: Name, Type, Project, Last use, and Created at.

My account / Security

Security

If you want to enforce security by not providing credentials of a real SonarQube user to run your code scan or to invoke web services, you can provide a User Token as a replacement of the user login. This will increase the security of your installation by not letting your analysis user's password going through your network.

Generate Tokens

Name	Type	Expires in	
Enter Token Name	Select Token Type	30 days	<button>Generate</button>

✓ New token "jenkins-token" has been generated. If you copy it now, you won't be able to see it again.

Name	Type	Project	Last use	Created at
------	------	---------	----------	------------

Step 10: Add SonarQube token to Jenkins Credentials

Open Jenkins:

Manage Jenkins → Credentials

Go to:

Global → Add Credentials

Configure:

Field	Value
Kind	Secret text
Secret	<SONARQUBE-TOKEN>

Field	Value
ID	sonar-token
Description	SonarQube Authentication Token

Click **Create**.

Jenkins / Manage Jenkins / Credentials / System / Global

Global

Credentials that

< Add Secret text X

Scope ?
Global (Jenkins, nodes, items, all child items, etc) v

Secret
.....

ID ?
sonar-token

Description ?
SonarQube Authentication Token

Create

REST API Jenkins 2.568.2

Step 11: Install SonarQube Scanner in Jenkins

Go to:

Jenkins → Manage Jenkins → Plugins

Search for:

SonarQube Scanner for Jenkins

Install it.

Then go to:

Manage Jenkins → Tools

Find:

SonarQube Scanner installations

Click:

Add SonarQube Scanner

Configure:

Name: SonarScanner

Install automatically: Yes

Click **Save**.



Plugins



Updates



Available plugins



Installed plugins



Advanced settings



Download progress

Download progress

Preparation

- Checking internet connectivity
- Checking update center connectivity
- Success

SonarQube Scanner



Success

Loading plugin extensions



Success

→ [Go back to the top page](#)

(you can start using the installed plugins right away)

☐

Restart Jenkins when installation is complete and no jobs are running

 **Jenkins** / Manage Jenkins ▾ / Tools 🔍 ⚙️ 👤

[+ Add Gradle](#)

SonarScanner for MSBuild installations

[+ Add SonarScanner for MSBuild](#)

⋮ SonarScanner for MSBuild ✕

Name

☒ Install automatically ?

⋮ Install from GitHub ✕

Version

▾

[+ Add Installer](#)

[Save](#) [Apply](#)

Step 12: Configure SonarQube Server in Jenkins

Go to:

Manage Jenkins → System

Find:

SonarQube servers

Click:

Add SonarQube

Configure:

Name: SonarQube

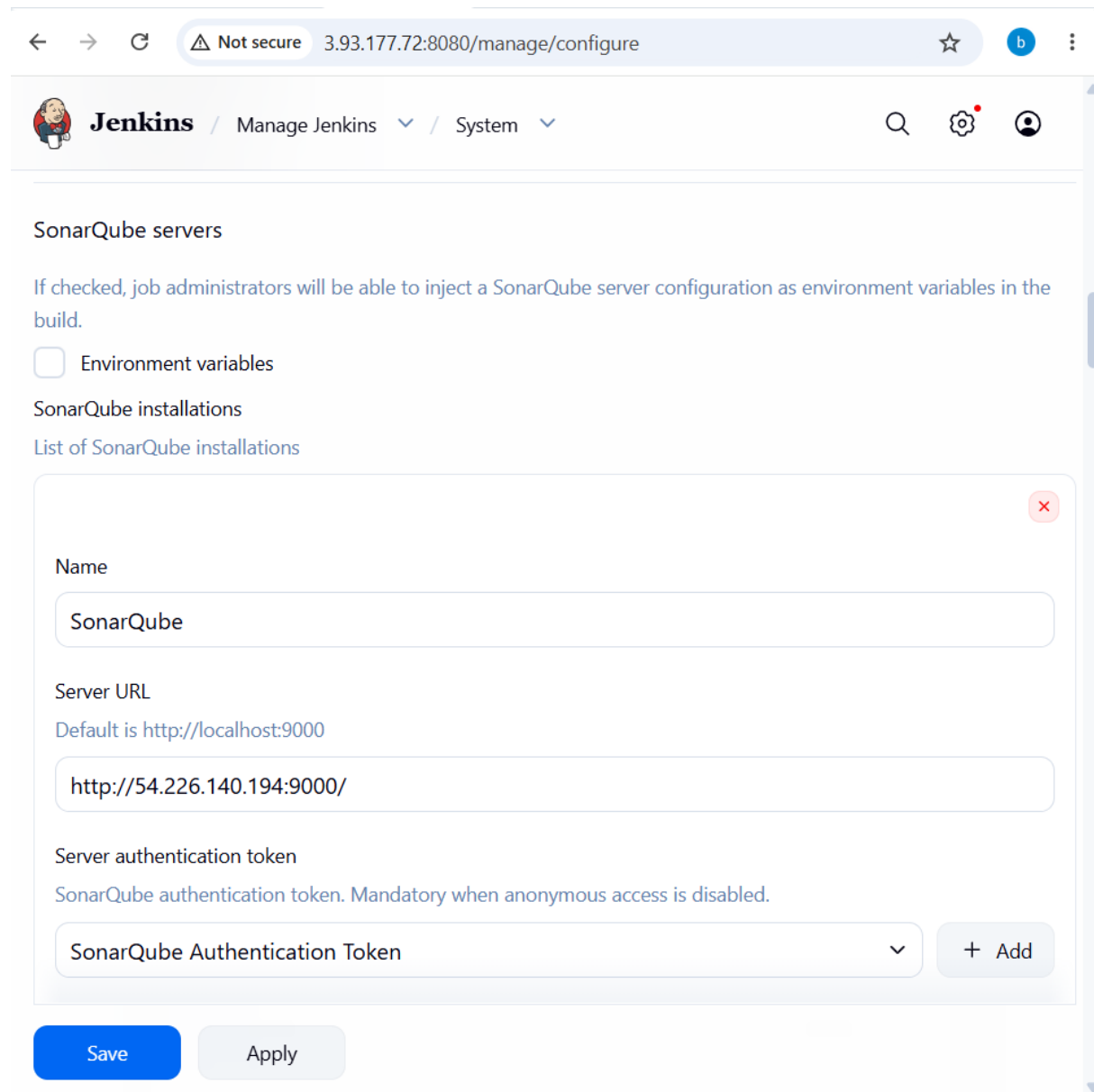
Server URL: `http://<SONARQUBE-PRIVATE-IP>:9000`

Under **Server authentication token**, select:

sonar-token

Click **Save**.

Now Jenkins knows where the SonarQube server is and how to authenticate.



The screenshot shows the Jenkins web interface at the URL `3.93.177.72:8080/manage/configure`. The page title is "Jenkins / Manage Jenkins / System". The main section is "SonarQube servers". It contains a checkbox for "Environment variables" which is unchecked. Below this is the "SonarQube installations" section with a link "List of SonarQube installations". A modal form for adding a new installation is open, showing fields for "Name" (filled with "SonarQube"), "Server URL" (filled with `http://54.226.140.194:9000/`), and "Server authentication token" (filled with "SonarQube Authentication Token"). The "Server authentication token" field has a dropdown arrow and an "Add" button. At the bottom of the modal are "Save" and "Apply" buttons.

SonarQube servers

If checked, job administrators will be able to inject a SonarQube server configuration as environment variables in the build.

☐ Environment variables

SonarQube installations

[List of SonarQube installations](#)

Name

SonarQube

Server URL

Default is `http://localhost:9000`

`http://54.226.140.194:9000/`

Server authentication token

SonarQube authentication token. Mandatory when anonymous access is disabled.

SonarQube Authentication Token

+ Add

Save Apply

Step 13: Create the Stage 2 Freestyle Job

Go to:

Jenkins → New Item

Enter:

VProfile-SonarQube

Select:

Freestyle project

Click **OK**.

Under:

Source Code Management → Git

Enter:

Repository URL:

<https://github.com/betawins/VProfile-1.git>

Branch:

*/main

This allows Stage 2 to obtain the same source code for analysis.

Step 14: Configure SonarQube analysis

In the Freestyle job, go to:

Build Environment

Select:

Prepare SonarQube Scanner environment

Then go to:

Build Steps → Add build step → Execute SonarQube Scanner

Configure the analysis properties.

For example:

sonar.projectKey=vprofile

sonar.projectName=VProfile

sonar.sources=.

If the repository contains a specific application source directory, use that directory instead of ..

For example:

```
sonar.sources=src
```

Save the job.

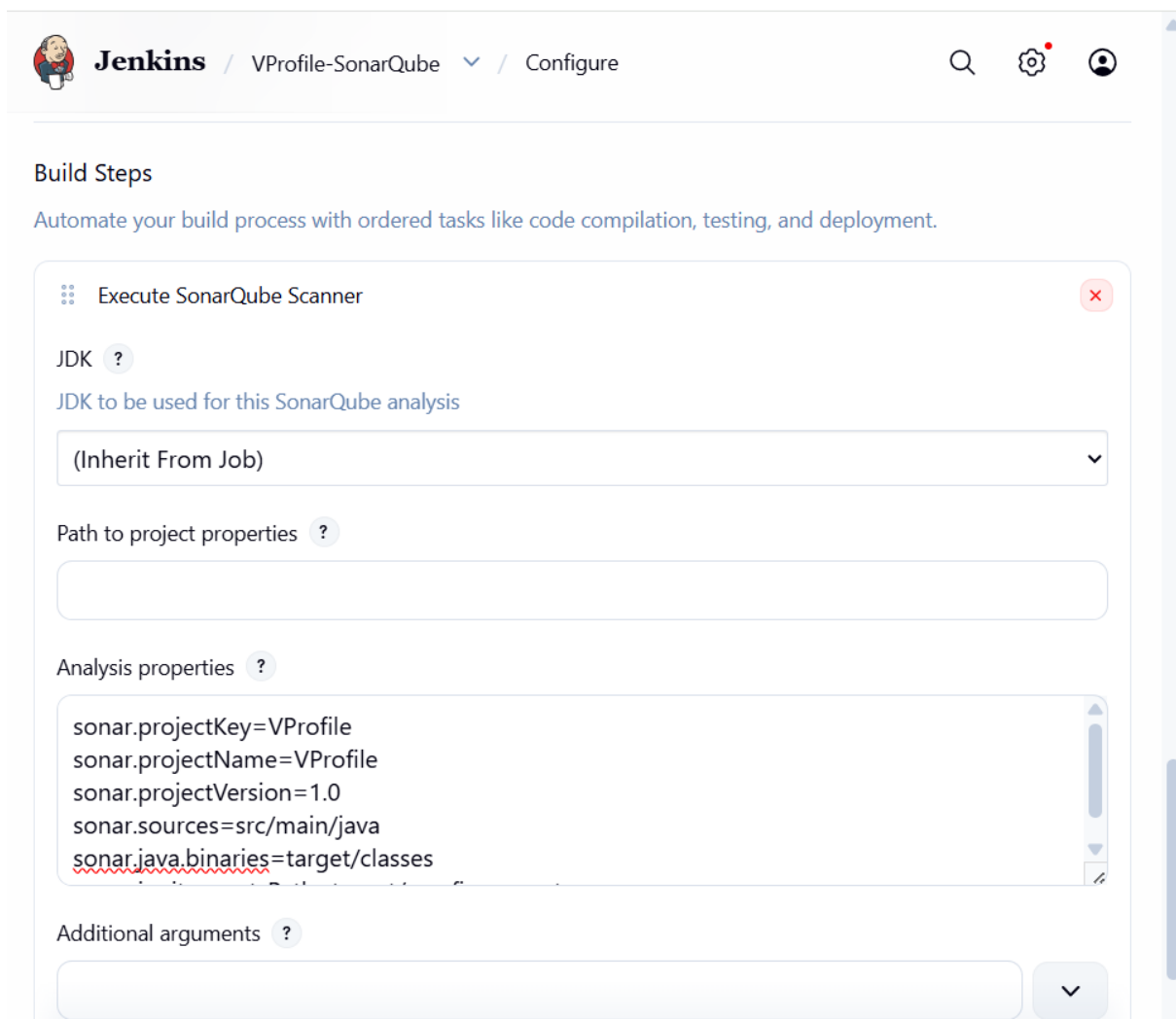
Click:

Build Now

Then open:

Build #1 → Console Output

You should see the SonarQube scanner analyzing the project.



The screenshot shows the Jenkins configuration page for a build job named "VProfile-SonarQube". The page has a header with the Jenkins logo, the job name, and a "Configure" link. Below the header, there is a section titled "Build Steps" with a description: "Automate your build process with ordered tasks like code compilation, testing, and deployment." The main configuration area is a light blue box with a red close button in the top right corner. It contains several sections: "JDK" with a dropdown menu set to "(Inherit From Job)", "Path to project properties" with an empty text input field, "Analysis properties" with a text area containing the following properties:

```
sonar.projectKey=VProfile
sonar.projectName=VProfile
sonar.projectVersion=1.0
sonar.sources=src/main/java
sonar.java.binaries=target/classes
```

 (Note: "sonar.java.binaries" is underlined with a red squiggly line), and "Additional arguments" with an empty text input field and a dropdown arrow.

 Status

 Changes

 Workspace

 Build Now

 Delete Project

 Configure

 Rename

 SonarQube

Builds >

⋮

Today

 #1 3:09 AM



VProfile-SonarQube

 Add description



Permalinks

Step 14: Configure SonarQube analysis

In the Pipeline, go to:

Jenkins / VProfile-Pipeline / Configure

Configure

- General
- Triggers
- Pipeline**
- Advanced

Pipeline

Define your Pipeline using Groovy directly or pull it from source control.

Definition

Pipeline script

```
Script ?
1~ pipeline {
2~   agent any
3~
4~   stages {
5~
6~     stage('Git Clone') {
7~       steps {
8~         git branch: 'master',
9~         url: 'https://github.com/rajkumari-hub/VProfile-1.git'
10~      }
11~    }
12~
13~    stage('Build') {
14~      steps {
15~        sh 'mvn clean verify'

```

Save Apply

Jenkins / VProfile-Pipeline / #13

Downloaded from central: <https://repo.maven.apache.org/maven2/org/codehaus/mojo/maven-metadata.xml> (20 kB at 49 kB/s)

Downloaded from central: <https://repo.maven.apache.org/maven2/org/apache/maven/plugins/maven-metadata.xml> (14 kB at 33 kB/s)

[INFO] -----

[INFO] BUILD FAILURE

[INFO] -----

[INFO] Total time: 1.415 s

[INFO] Finished at: 2026-08-14T14:17:43Z

[INFO] -----

[ERROR] No plugin found for prefix 'sonar' in the current project and in the plugin groups [org.apache.maven.plugins, org.codehaus.mojo] available from the repositories [local (/var/lib/jenkins/.m2/repository), central (<https://repo.maven.apache.org/maven2>)] -> [Help 1]

[ERROR]

[ERROR] To see the full stack trace of the errors, re-run Maven with the -e switch.

[ERROR] Re-run Maven using the -X switch to enable full debug logging.

[ERROR]

[ERROR] For more information about the errors and possible solutions, please read the following articles:

[ERROR] [Help 1] <http://wiki.apache.org/confluence/display/MAVEN/NoPluginFoundForPrefixException>

[Pipeline] }

WARN: Unable to locate 'report-task.txt' in the workspace. Did the SonarScanner succeed?

[Pipeline] // withSonarQubeEnv

[Pipeline] }

[Pipeline] }

[Pipeline] // stage

[Pipeline] }

[Pipeline] // node

[Pipeline] End of Pipeline

ERROR: script returned exit code 1

Finished: FAILURE

**It covers the exact problem you faced:
old Maven WAR plugin 2.2 → 3.4.0, Git
changes, commit/push, and changing the
Jenkins Pipeline repository from
betawins to rajkumari-hub.**

Jenkins + Maven Version Change + GitHub Repository Migration

1. Problem we faced

Our Jenkins Pipeline initially failed during:

```
mvn clean verify
```

The important error was:

```
maven-war-plugin:2.2
```

Cannot access defaults field of Properties

and:

BUILD FAILURE

Failed to execute goal

```
org.apache.maven.plugins:maven-war-plugin:2.2:war
```

Root cause

The project was using an old Maven WAR plugin version:

```
maven-war-plugin:2.2
```

The Jenkins server was running a newer Java/Maven environment, so the old WAR plugin was incompatible.

2. Check Java and Maven versions

On the Jenkins/EC2 server:

`java -version`

Check Maven:

`mvn -version`

For example:

Apache Maven 3.x

Java version: 17/21...

The important point is that the old WAR plugin 2.2 was incompatible with the modern Java/Maven environment.

3. Find the Maven WAR plugin in pom.xml

Go to the Jenkins workspace:

`cd /var/lib/jenkins/workspace/VProfile-Pipeline`

Check the WAR plugin:

`grep -n -A5 -B2 "maven-war-plugin" pom.xml`

Initially it was:

`<plugin>`

`<groupId>org.apache.maven.plugins</groupId>`

```
<artifactId>maven-war-plugin</artifactId>
<version>2.2</version>
</plugin>
```

4. Change WAR plugin from 2.2 to 3.4.0

Change:

```
<version>2.2</version>
```

to:

```
<version>3.4.0</version>
```

Final configuration:

```
<plugin>
  <groupId>org.apache.maven.plugins</groupId>
  <artifactId>maven-war-plugin</artifactId>
  <version>3.4.0</version>
</plugin>
```

Then verify:

```
grep -n -A5 -B2 "maven-war-plugin" pom.xml
```

5. Test Maven locally on Jenkins server

Run:

```
mvn clean verify
```

Before the fix:

```
maven-war-plugin:2.2
```

BUILD FAILURE

After the fix:

```
maven-war-plugin:3.4.0
```

and:

BUILD SUCCESS

In our case, all tests passed:

Tests run: 9, Failures: 0, Errors: 0, Skipped: 0

The WAR was successfully generated:

```
target/vprofile-v2.war
```

6. Create .gitignore

We don't want Maven's generated target/ directory to be pushed to GitHub.

Create .gitignore:

```
sudo -u jenkins bash -c 'echo "target/" >> .gitignore'
```

Check:

```
cat .gitignore
```

Expected:

```
target/
```

Check Git status:

```
sudo -u jenkins git status
```

You should see:

Untracked files:

```
.gitignore
```

7. Add .gitignore

Run:

```
sudo -u jenkins git add .gitignore
```

Check:

```
sudo -u jenkins git status
```

8. Commit the change

Run:

```
sudo -u jenkins git commit -m "Add gitignore for Maven target"
```

We got a commit similar to:

```
6572882 Add gitignore for Maven target
```

9. Check Git branch

Run:

```
sudo -u jenkins git branch
```

Expected:

```
* master
```

So our working branch is:

```
master
```

10. Check the old GitHub remote

Initially:

```
sudo -u jenkins git remote -v
```

showed:

```
origin https://github.com/betawins/VProfile-1.git
```

But we wanted to use:

```
rajkumari-hub/VProfile-1
```

11. Change Git remote

Change the remote:

```
sudo -u jenkins git remote set-url origin https://github.com/rajkumari-hub/VProfile-1.git
```

Verify:

```
sudo -u jenkins git remote -v
```

Expected:

```
origin https://github.com/rajkumari-hub/VProfile-1.git (fetch)
```

```
origin https://github.com/rajkumari-hub/VProfile-1.git (push)
```

12. Why we got the GitHub 403 error

Initially we tried:

`sudo -u jenkins git push origin master`

GitHub asked:

Username for 'https://github.com':

We entered:

betawins

But GitHub responded:

remote: Permission to betawins/VProfile-1.git denied to rajkumari-hub.

fatal: ... 403

Why?

The repository belonged to:

betawins/VProfile-1

but the authentication was associated with:

rajkumari-hub

Therefore GitHub said:

Permission denied

The important lesson:

The Git remote repository and the GitHub account used for authentication must have the appropriate permissions.

13. Change Jenkins Pipeline Git repository

This was especially important because we are using a Pipeline job, not a Freestyle job.

Changing the remote manually inside the workspace isn't enough because Jenkins performs the checkout according to the Pipeline script.

Our old Pipeline had:

```
git branch: 'master',  
url: 'https://github.com/betawins/VProfile-1.git'
```

We changed it to:

```
git branch: 'master',  
url: 'https://github.com/rajkumari-hub/VProfile-1.git'
```

14. Final Jenkins Pipeline Git stage

```
stage('Git Clone') {  
    steps {  
        git branch: 'master',  
        url: 'https://github.com/rajkumari-hub/VProfile-1.git'  
    }  
}
```



```
}
```

Now Jenkins clones:

rajkumari-hub/VProfile-1

instead of:

betawins/VProfile-1

15. Verify Jenkins is cloning the correct repository

After running Jenkins, check the console.

Previously:

```
git config remote.origin.url https://github.com/betawins/VProfile-1.git
```

After the change:

```
git config remote.origin.url https://github.com/rajkumari-hub/VProfile-1.git
```

We successfully reached:

Checking out Revision 6572882

and:

Commit message: "Add gitignore for Maven target"

This confirmed Jenkins was using our rajkumari-hub repository.

16. Verify Maven after GitHub migration

Jenkins then ran:

mvn clean verify

The important output became:

--- maven-war-plugin:3.4.0:war ---

instead of:

--- maven-war-plugin:2.2:war ---

Then:

Tests run: 9

Failures: 0

Errors: 0

Skipped: 0

And:

Building war:

target/vprofile-v2.war

Finally:

BUILD SUCCESS

So the Maven problem was completely fixed. :white_check_mark:

17. Important Git commands for future use

Check repository

`git remote -v`

Check branch

`git branch`

Check status

`git status`

Add changes

`git add .`

or:

`git add pom.xml`

Commit

`git commit -m "Update Maven WAR plugin"`

Push

`git push origin master`

View recent commits

`git log --oneline -5`

18. Recommended Git workflow

For future Maven/plugin changes:

Modify pom.xml



`git status`



`git add pom.xml`



`git commit -m "Update Maven WAR plugin"`



`git push origin master`



GitHub



Jenkins Pipeline



`mvn clean verify`



Build Success

19. One important lesson about Jenkins workspaces

Don't rely on manually changing:

```
git remote set-url ...
```

inside:

```
/var/lib/jenkins/workspace/VProfile-Pipeline
```

if your Pipeline itself contains the Git URL.

Instead, change the Pipeline:

```
git branch: 'master',
```

```
url: 'https://github.com/rajkumari-hub/VProfile-1.git'
```

because Jenkins will perform the checkout again during the next build.

20. Final Pipeline we reached

Our current Pipeline is:

```
pipeline {
```

```
    agent any
```

```
    stages {
```

```
        stage('Git Clone') {
```

```

steps {
    git branch: 'master',
        url: 'https://github.com/rajkumari-hub/VProfile-1.git'
    }
}

stage('Build') {
    steps {
        sh 'mvn clean verify'
    }
}

stage('SonarQube Analysis') {
    steps {
        withSonarQubeEnv('SonarQube') {
            sh '''
                mvn org.sonarsource.scanner.maven:sonar-maven-plugin:sonar \
                -Dsonar.projectKey=VProfile \
                -Dsonar.projectName=VProfile
            '''
        }
    }
}
}

```

The Git + Maven portion is now working. The remaining SonarQube issue we encountered was:

No plugin found for prefix 'sonar'

which we addressed by explicitly calling:

```
org.sonarsource.scanner.maven:sonar-maven-plugin:sonar
```

:star: Short version to remember

When an old Maven project fails with:

Cannot access defaults field of Properties

and you see:

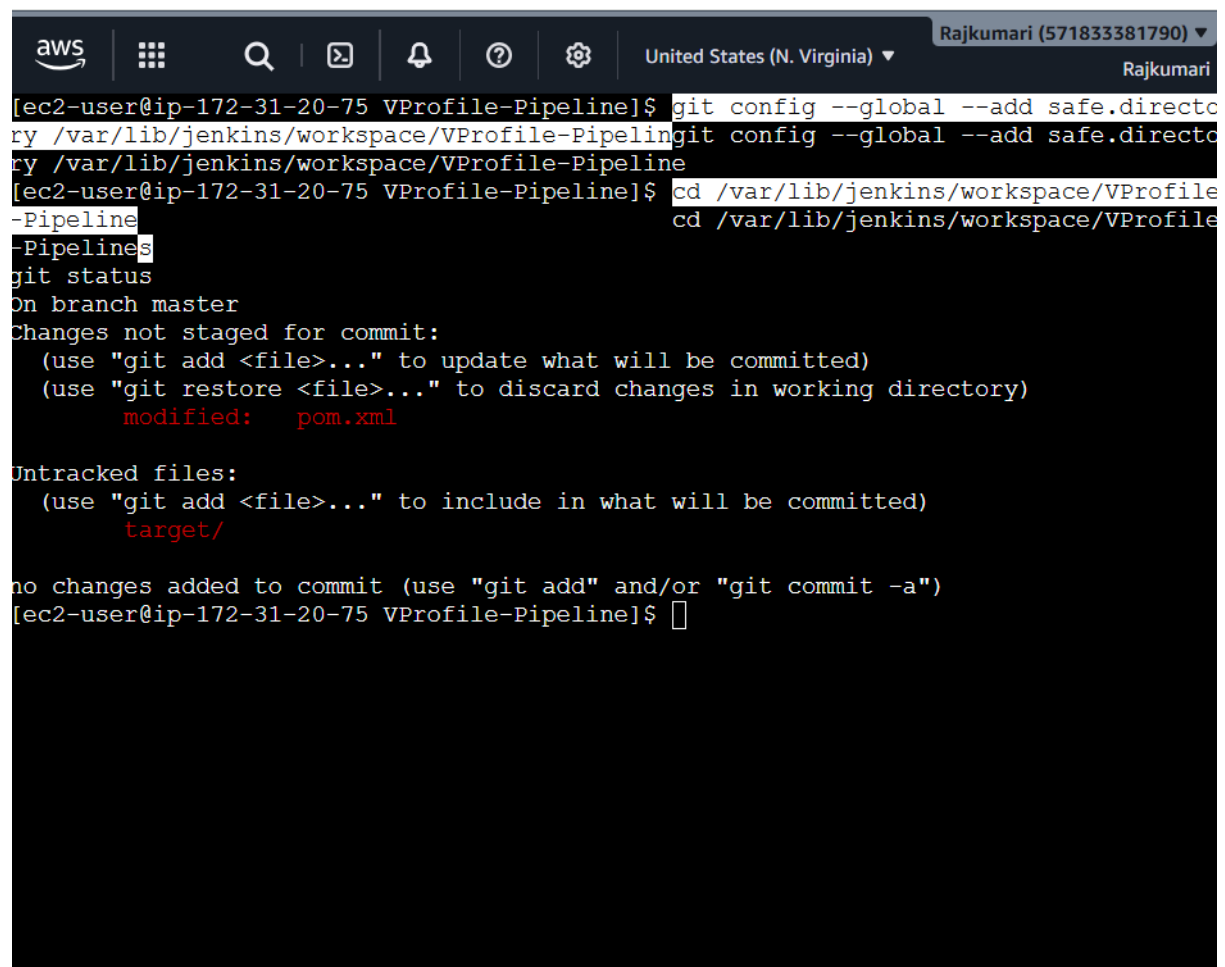
```
maven-war-plugin:2.2
```

Do this:

1. Open pom.xml
2. Find maven-war-plugin
3. Change 2.2 → 3.4.0
4. mvn clean verify
5. Confirm BUILD SUCCESS
6. Add .gitignore with target/
7. git add
8. git commit

9. Change Git remote if required
10. git push
11. Update Jenkins Pipeline Git URL
12. Run Jenkins
13. Verify Jenkins uses the new repository and WAR plugin

The key concept: target/ is Maven-generated output, while pom.xml contains the Maven build configuration. Commit the pom.xml change so Jenkins gets the fixed plugin version on every fresh checkout.



```
aws | [grid icon] | [search icon] | [terminal icon] | [bell icon] | [help icon] | [gear icon] | United States (N. Virginia) | Rajkumari (571833381790) | Rajkumari

[ec2-user@ip-172-31-20-75 VProfile-Pipeline]$ git config --global --add safe.directory /var/lib/jenkins/workspace/VProfile-Pipeline
[ec2-user@ip-172-31-20-75 VProfile-Pipeline]$ cd /var/lib/jenkins/workspace/VProfile-Pipeline
[ec2-user@ip-172-31-20-75 VProfile-Pipeline]$ git status
On branch master
Changes not staged for commit:
  (use "git add <file>..." to update what will be committed)
  (use "git restore <file>..." to discard changes in working directory)
        modified:   pom.xml

Untracked files:
  (use "git add <file>..." to include in what will be committed)
        target/

no changes added to commit (use "git add" and/or "git commit -a")
[ec2-user@ip-172-31-20-75 VProfile-Pipeline]$
```



```
aws | [grid icon] | [search icon] | [terminal icon] | [bell icon] | [help icon] | [settings icon] | United States (N. Virginia) ▼ | Rajkumari (571833381790) ▼ | Rajkumari

[ec2-user@ip-172-31-20-75 VProfile-Pipeline]$ git config --global --add safe.directory /var/lib/jenkins/workspace/VProfile-Pipeline
[ec2-user@ip-172-31-20-75 VProfile-Pipeline]$ git config --global --add safe.directory /var/lib/jenkins/workspace/VProfile-Pipeline
[ec2-user@ip-172-31-20-75 VProfile-Pipeline]$ cd /var/lib/jenkins/workspace/VProfile-Pipeline
[ec2-user@ip-172-31-20-75 VProfile-Pipeline]$ cd /var/lib/jenkins/workspace/VProfile-Pipeline
[ec2-user@ip-172-31-20-75 VProfile-Pipeline]$ git status
On branch master
Changes not staged for commit:
  (use "git add <file>..." to update what will be committed)
  (use "git restore <file>..." to discard changes in working directory)
        modified:   pom.xml

Untracked files:
  (use "git add <file>..." to include in what will be committed)
        target/

no changes added to commit (use "git add" and/or "git commit -a")
[ec2-user@ip-172-31-20-75 VProfile-Pipeline]$
```

```
aws | [grid icon] | [search icon] | [terminal icon] | [bell icon] | [help icon] | [settings icon] | United States (N. Virginia) | Rajkumari (571833381790) | Rajkumari

[ec2-user@ip-172-31-20-75 VProfile-Pipeline]$ git config --global --add safe.directory /var/lib/jenkins/workspace/VProfile-Pipeline
[ec2-user@ip-172-31-20-75 VProfile-Pipeline]$ git config --global --add safe.directory /var/lib/jenkins/workspace/VProfile-Pipeline
[ec2-user@ip-172-31-20-75 VProfile-Pipeline]$ cd /var/lib/jenkins/workspace/VProfile-Pipeline
[ec2-user@ip-172-31-20-75 VProfile-Pipeline]$ cd /var/lib/jenkins/workspace/VProfile-Pipeline
[ec2-user@ip-172-31-20-75 VProfile-Pipeline]$ git status
On branch master
Changes not staged for commit:
  (use "git add <file>..." to update what will be committed)
  (use "git restore <file>..." to discard changes in working directory)
        modified:   pom.xml

Untracked files:
  (use "git add <file>..." to include in what will be committed)
        target/

no changes added to commit (use "git add" and/or "git commit -a")
[ec2-user@ip-172-31-20-75 VProfile-Pipeline]$
```

← → ↻ ⚠ Not secure 100.30.255.90:8080/job/VProfile-Pipeline/ ☆ ⓘ

 **Jenkins** / VProfile-Pipeline 🔍 ⚙️ 👤

🔗 Changes

🏗 Build Now

🗑 Delete Pipeline

⚙ Configure

✎ Rename

🔗 SonarQube

📁 Stages

📖 Pipeline Syntax

SonarQube Quality Gate

VProfile Passed

server-side processing: Success

Permalinks

- Last build (#16), 50 sec ago
- Last stable build (#15), 20 min ago
- Last successful build (#15), 20 min ago
- Last failed build (#13), 27 min ago
- Last unsuccessful build (#13), 27 min ago
- Last completed build (#15), 20 min ago

Builds >

🔍 Filter

Today

🟢 #16 2:44 PM

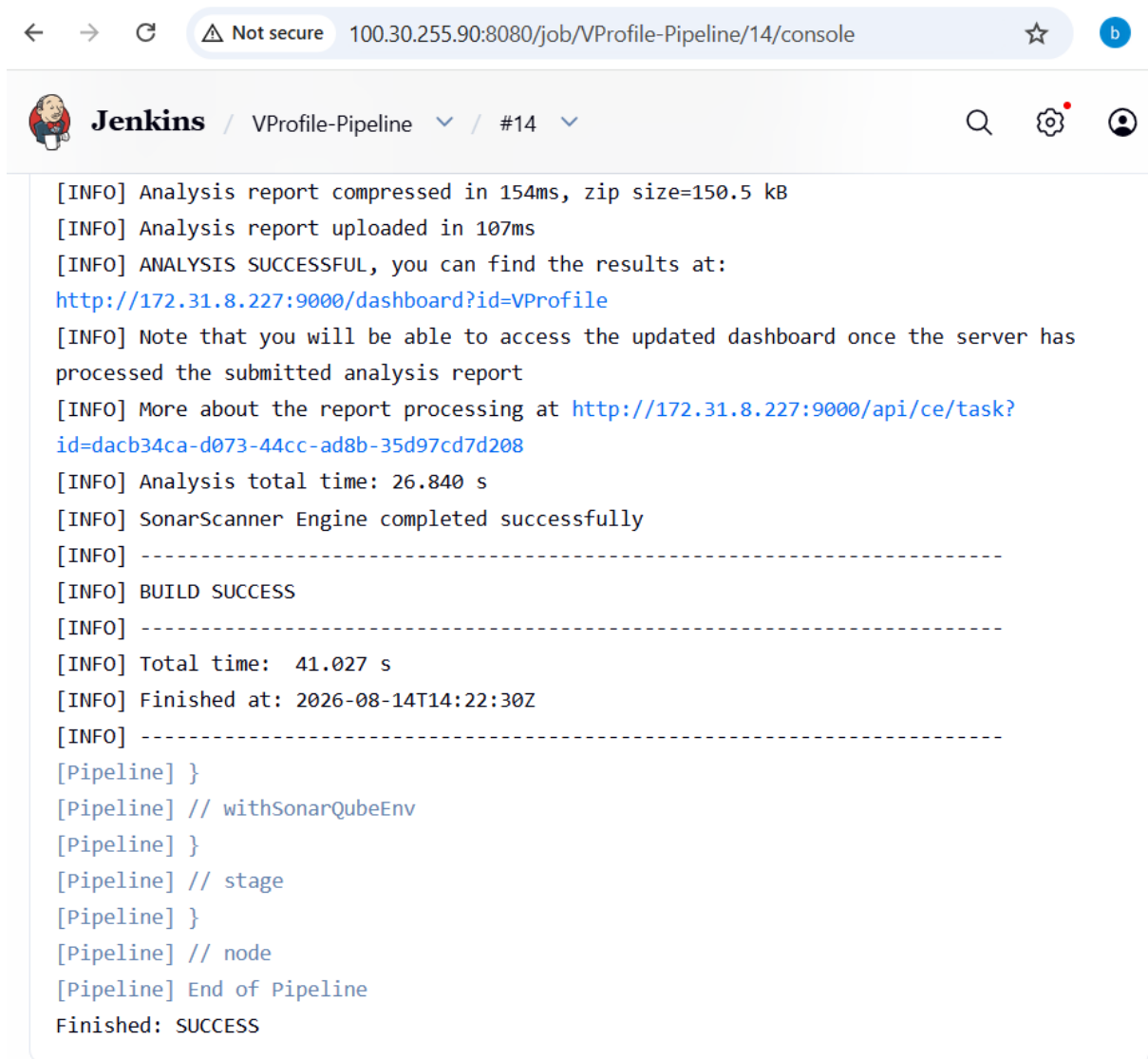
🟢 #15 2:24 PM

🟢 #14 2:21 PM

🔴 #13 2:17 PM

🔴 #12 2:12 PM

🔴 #11 11:42 AM



The screenshot shows a web browser window with the address bar displaying '100.30.255.90:8080/job/VProfile-Pipeline/14/console'. The Jenkins interface header includes the Jenkins logo, the job name 'VProfile-Pipeline', and the build number '#14'. The console output contains the following text:

```
[INFO] Analysis report compressed in 154ms, zip size=150.5 kB
[INFO] Analysis report uploaded in 107ms
[INFO] ANALYSIS SUCCESSFUL, you can find the results at:
http://172.31.8.227:9000/dashboard?id=VProfile
[INFO] Note that you will be able to access the updated dashboard once the server has
processed the submitted analysis report
[INFO] More about the report processing at http://172.31.8.227:9000/api/ce/task?id=dacb34ca-d073-44cc-ad8b-35d97cd7d208
[INFO] Analysis total time: 26.840 s
[INFO] SonarScanner Engine completed successfully
[INFO] -----
[INFO] BUILD SUCCESS
[INFO] -----
[INFO] Total time: 41.027 s
[INFO] Finished at: 2026-08-14T14:22:30Z
[INFO] -----
[Pipeline] }
[Pipeline] // withSonarQubeEnv
[Pipeline] }
[Pipeline] // stage
[Pipeline] }
[Pipeline] // node
[Pipeline] End of Pipeline
Finished: SUCCESS
```

Step 16: Verify the SonarQube result

After the Jenkins build completes, open:

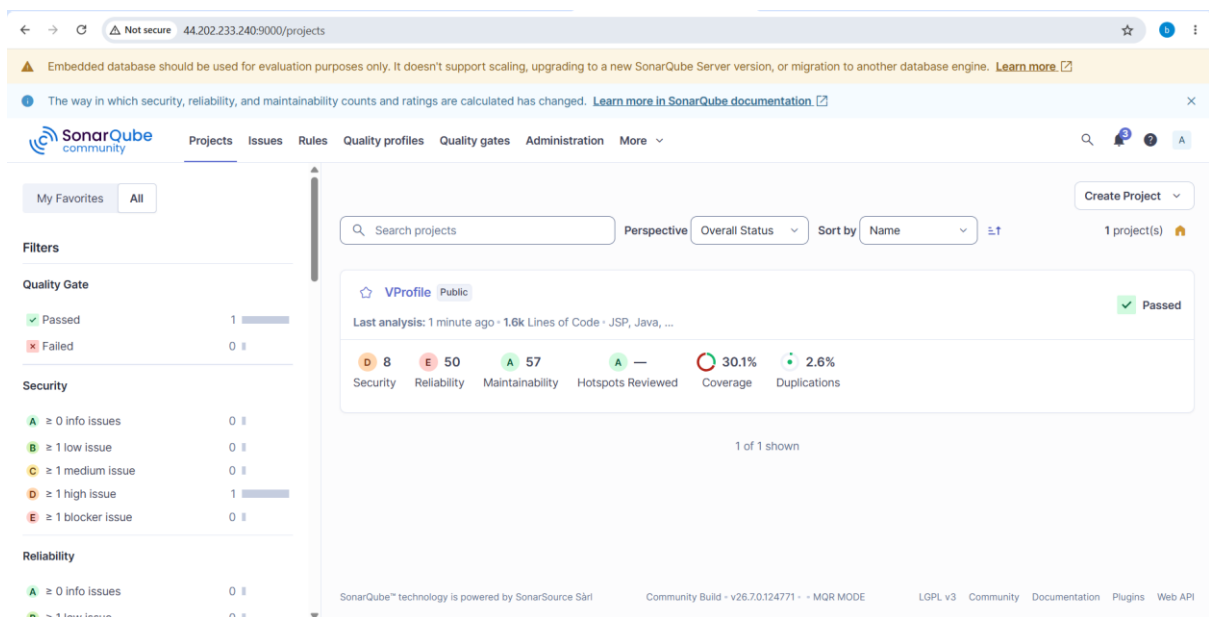
SonarQube → Projects → VProfile

You should see the analysis results.

Check:

- **Bugs**
- **Vulnerabilities**
- **Code Smells**

- Security Hotspots
- Duplications
- Quality Gate
- Reliability
- Maintainability
- Security



Your flow is now:

GitHub

|

v

VProfile-1 Repository

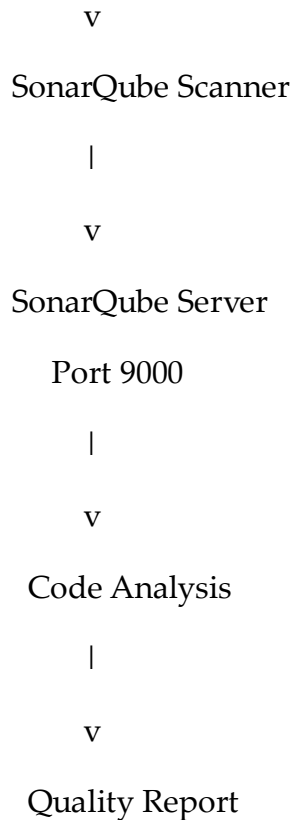
|

v

Jenkins Freestyle Job

VProfile-SonarQube

|



stage3: Slack Integration to send the alerts to slack.

SourceCode: <https://github.com/betawins/VProfile-1.git>

Stage 3: Slack Integration to Send Jenkins Alerts

1. Objective

The objective of this stage is to integrate **Jenkins with Slack** so that the development/DevOps team receives automatic notifications about Jenkins pipeline activities.

The integration will send alerts to a Slack channel whenever the Jenkins pipeline:

- Starts a build
- Completes successfully

- Fails
- Becomes unstable
- Encounters a pipeline error

This helps the team monitor CI/CD activities in real time without continuously checking the Jenkins dashboard.

2. Source Code

GitHub Repository: <https://github.com/rajkumari-hub/VProfile-1.git>

The project source code is cloned from the GitHub repository and used as the input for the Jenkins Pipeline.

3. Prerequisites

Before starting Slack integration, ensure the following are available:

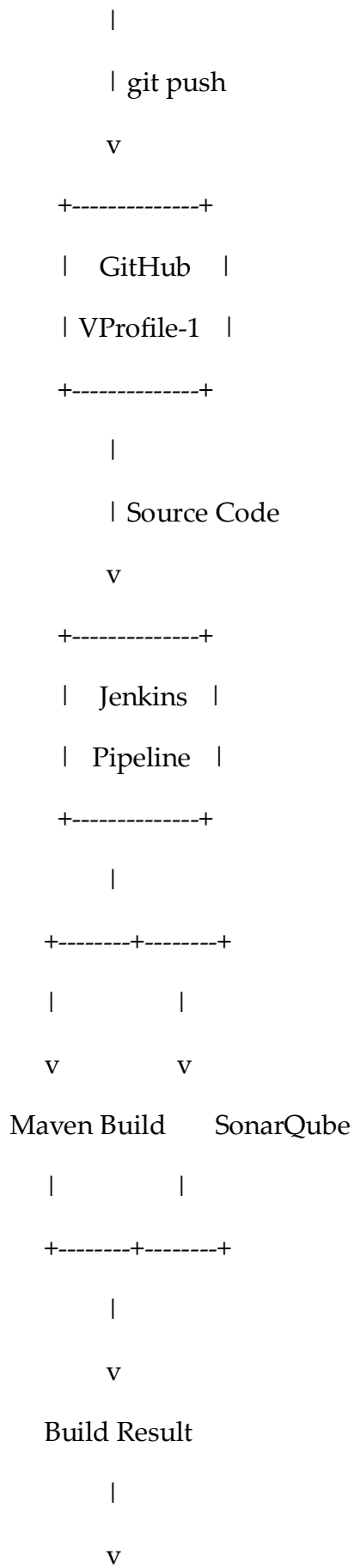
- Jenkins server
 - Jenkins Pipeline job
 - GitHub repository
 - Slack workspace
 - Slack channel for Jenkins notifications
 - Internet connectivity from Jenkins
 - Jenkins administrator access
-

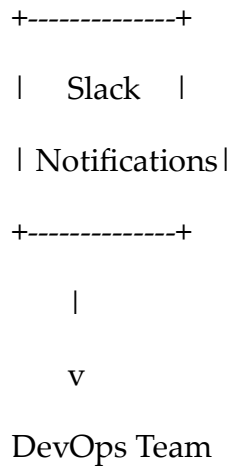
4. Architecture

The notification flow is:

Developer → GitHub → Jenkins → Build/Test → Slack

Developer





Step 1 – Create Slack Channel

1. Login to your Slack workspace.
2. Create a new channel.

Example:

#jenkins-alerts

3. This channel will be used to receive Jenkins notifications.

You can also use an existing channel.

Step 2 – Install Slack Notification Plugin in Jenkins

1. Login to Jenkins.
2. Go to:

Manage Jenkins

3. Select:

Plugins

4. Open the **Available plugins** section.
5. Search for:

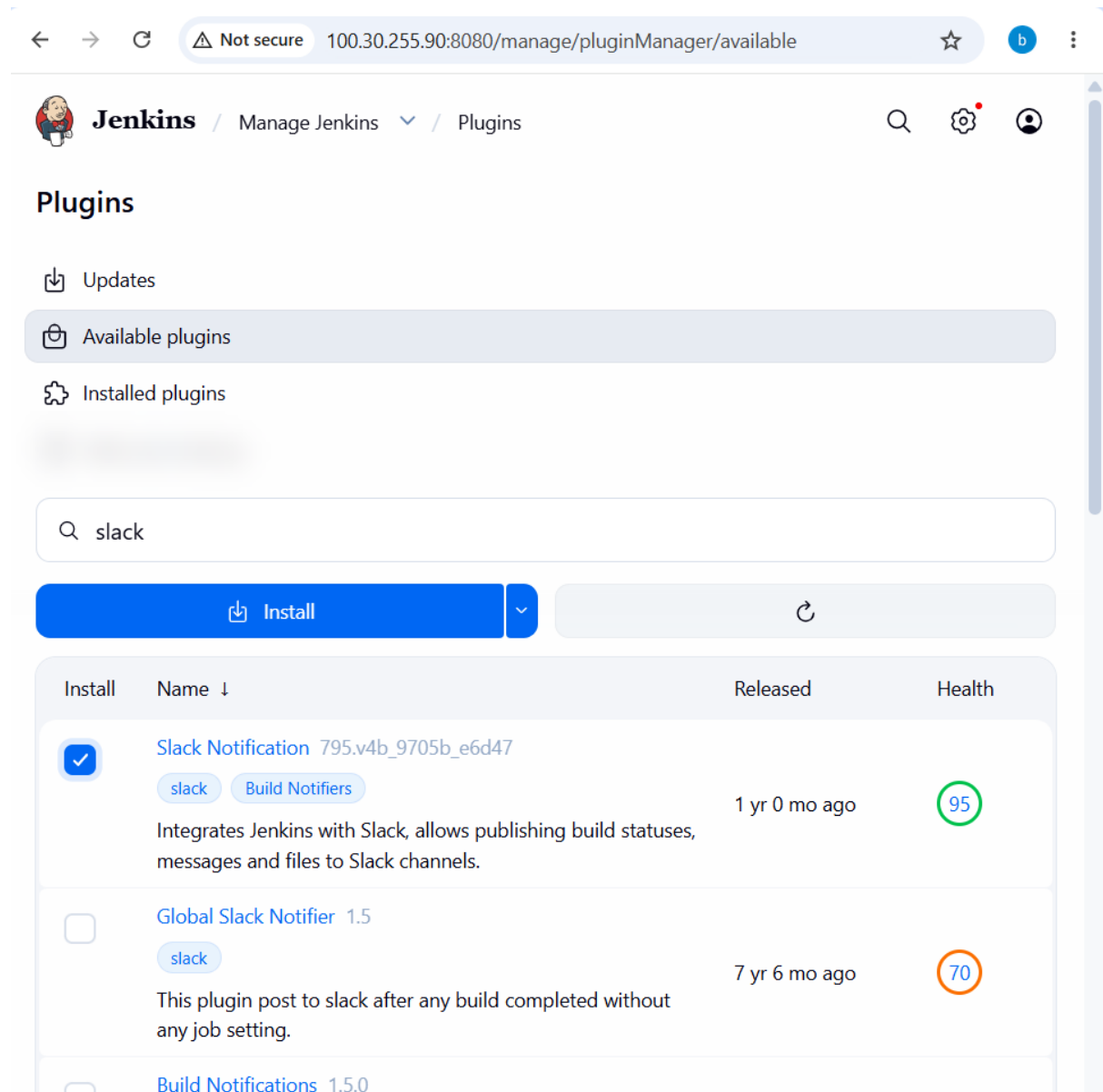
Slack Notification

6. Select the Slack plugin.
7. Click:

Install

8. Restart Jenkins if required.

The official Jenkins Slack plugin documentation provides the same installation path through **Manage Jenkins** → **Manage Plugins** → **Available**, followed by searching for Slack.



The screenshot shows the Jenkins web interface at the URL `100.30.255.90:8080/manage/pluginManager/available`. The page title is "Jenkins / Manage Jenkins / Plugins". The "Plugins" section is active, with a search bar containing "slack". Below the search bar, there is a blue "Install" button and a refresh button. A table lists the available plugins:

Install	Name ↓	Released	Health
☒	Slack Notification 795.v4b_9705b_e6d47 slack Build Notifiers Integrates Jenkins with Slack, allows publishing build statuses, messages and files to Slack channels.	1 yr 0 mo ago	95
☐	Global Slack Notifier 1.5 slack This plugin post to slack after any build completed without any job setting.	7 yr 6 mo ago	70
☐	Build Notifications 1.5.0		

The screenshot shows the Jenkins web interface in a browser. The address bar indicates the URL is `100.30.255.90:8080/manage/pluginManager/updates/`. The page title is "Jenkins / Manage Jenkins / Plugins". The left sidebar contains links for "Updates", "Available plugins", "Installed plugins", "Advanced settings", and "Download progress". The "Download progress" section is active and shows the following status:

- Preparation
 - Checking internet connectivity
 - Checking update center connectivity
 - Success
- Apache HttpComponents Client 5.x API ✓ Success
- Slack Notification ✓ Success
- Loading plugin extensions ✓ Success

Below the progress list, there are two actions:

- [Go back to the top page](#)
(you can start using the installed plugins right away)
- ☐ Restart Jenkins when installation is complete and no jobs are running

Step 3 – Create Slack App

To securely connect Jenkins with Slack, create a Slack application.

1. Open Slack API.
2. Go to:

<https://api.slack.com/apps>

3. Click:

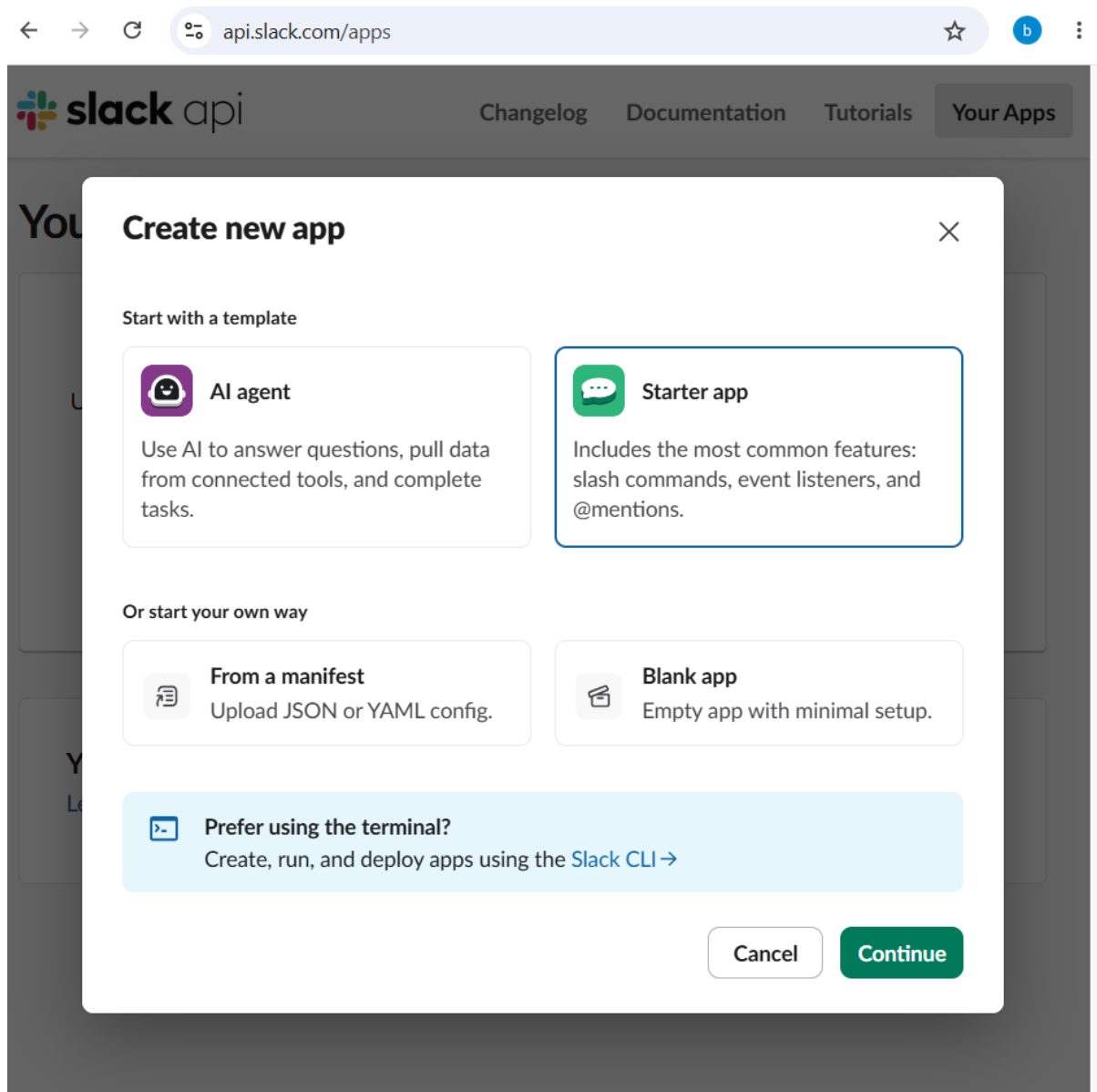
Create New App

4. Select:

From an app manifest

5. Select your Slack workspace.
6. Create the application.

The current Jenkins Slack plugin documentation recommends creating a Slack app, installing it into the workspace, and using the Bot User OAuth token for Jenkins integration.



Step 4 – Configure Slack App Permissions

The Jenkins Slack integration requires appropriate permissions for the bot.

Typical permissions include:

channels:read

chat:write

chat:write.customize

files:write

reactions:write

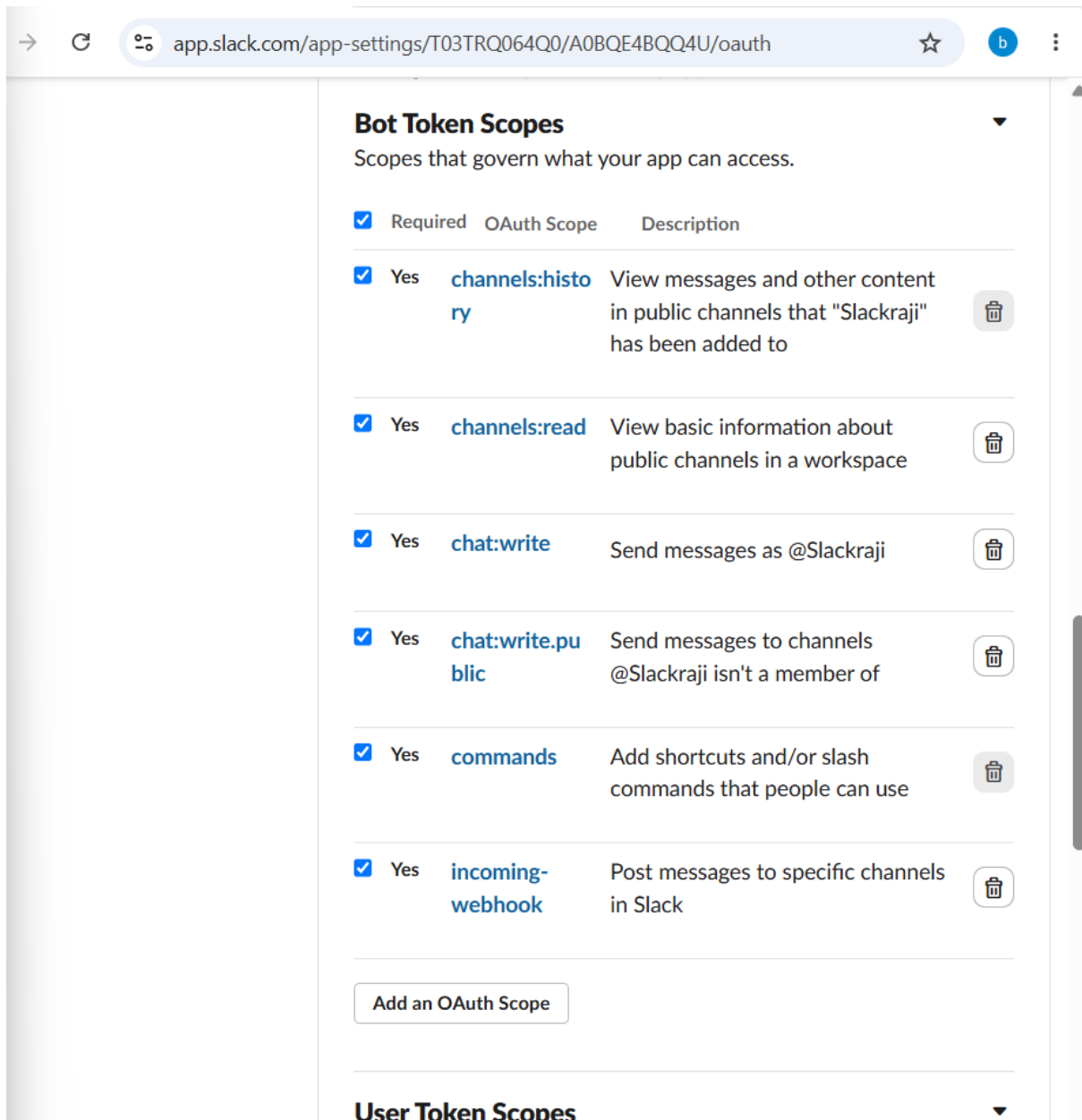
users:read

users:read.email

groups:read

The exact permissions required depend on the features you use.

For basic Jenkins build notifications, the important capability is permission to send messages to the target Slack channel.



1. Create your own Slack workspace

Go to the official Slack workspace creation page:

Create a Slack workspace

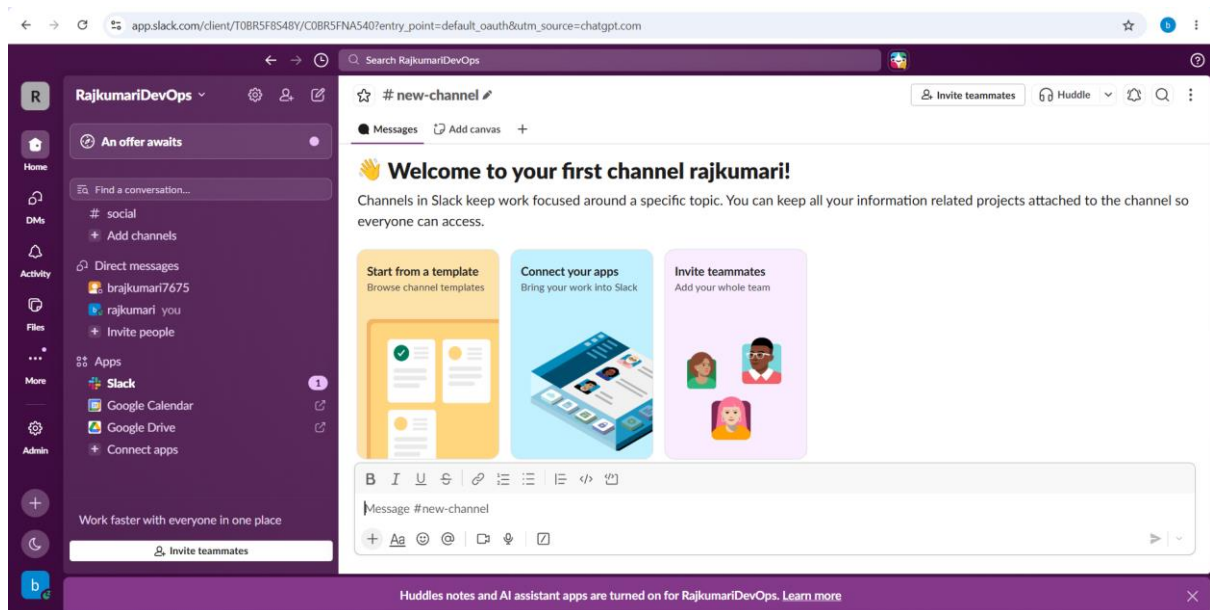
Slack's current process is:

1. Enter your email address, or continue with Google/Apple.
2. Confirm your email using the verification code.
3. Choose **Create a workspace**.
4. Give it a name, for example:

Rajkumari DevOps

5. Choose a workspace URL/name when Slack asks.
6. Finish the setup.

When you create the workspace, **you become its Primary Owner**, so you'll have the permissions needed to create channels and install your app.



2. Create #jenkins-alerts

Once you're inside your new workspace:

1. Look at the left sidebar.
2. Find **Channels**.
3. Click the + beside Channels.
4. Select **Create a channel**.
5. Enter:

jenkins-alerts

6. Select **Public**.
7. Click **Create**.

You should now have:

Your Workspace

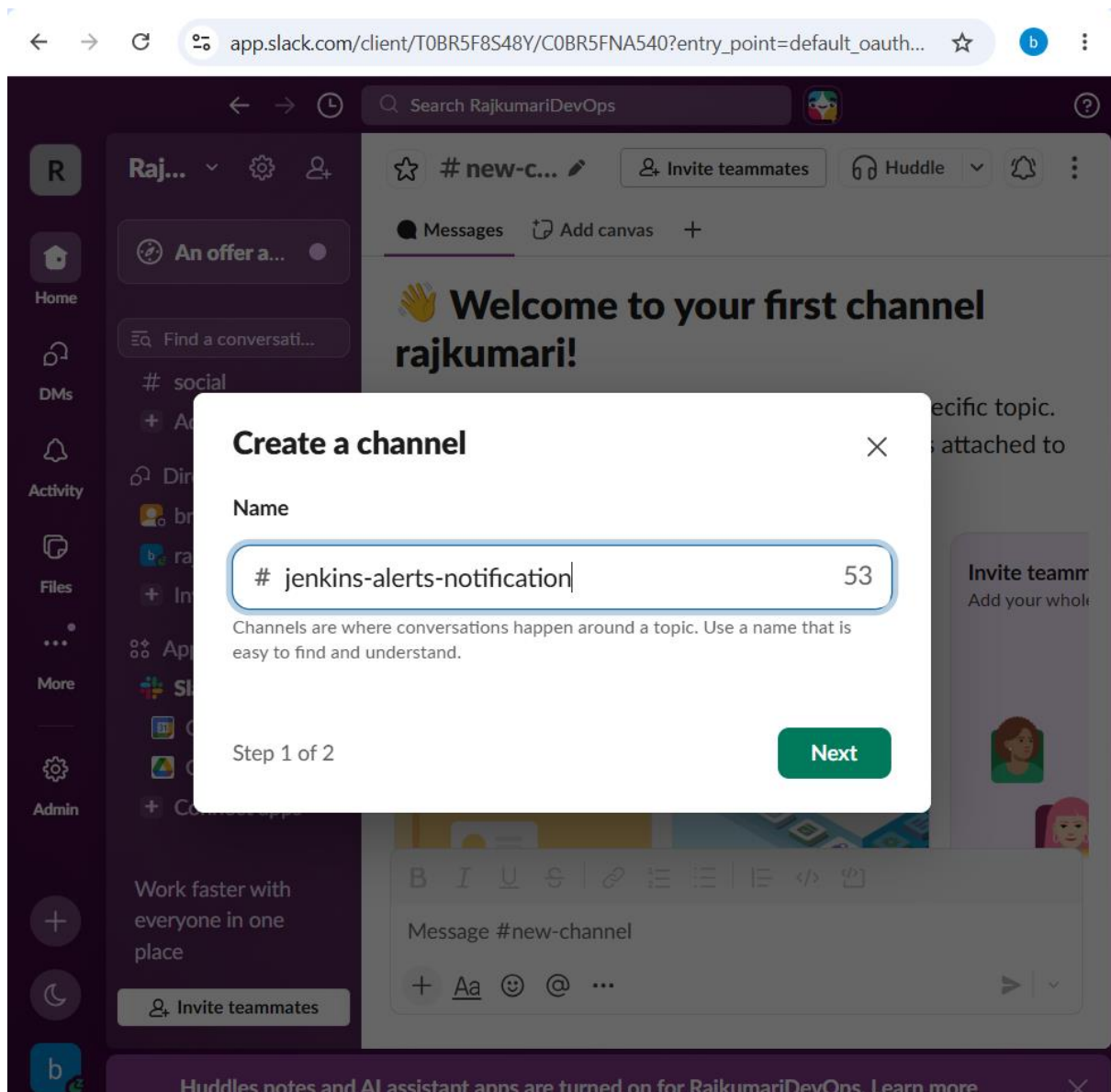
|

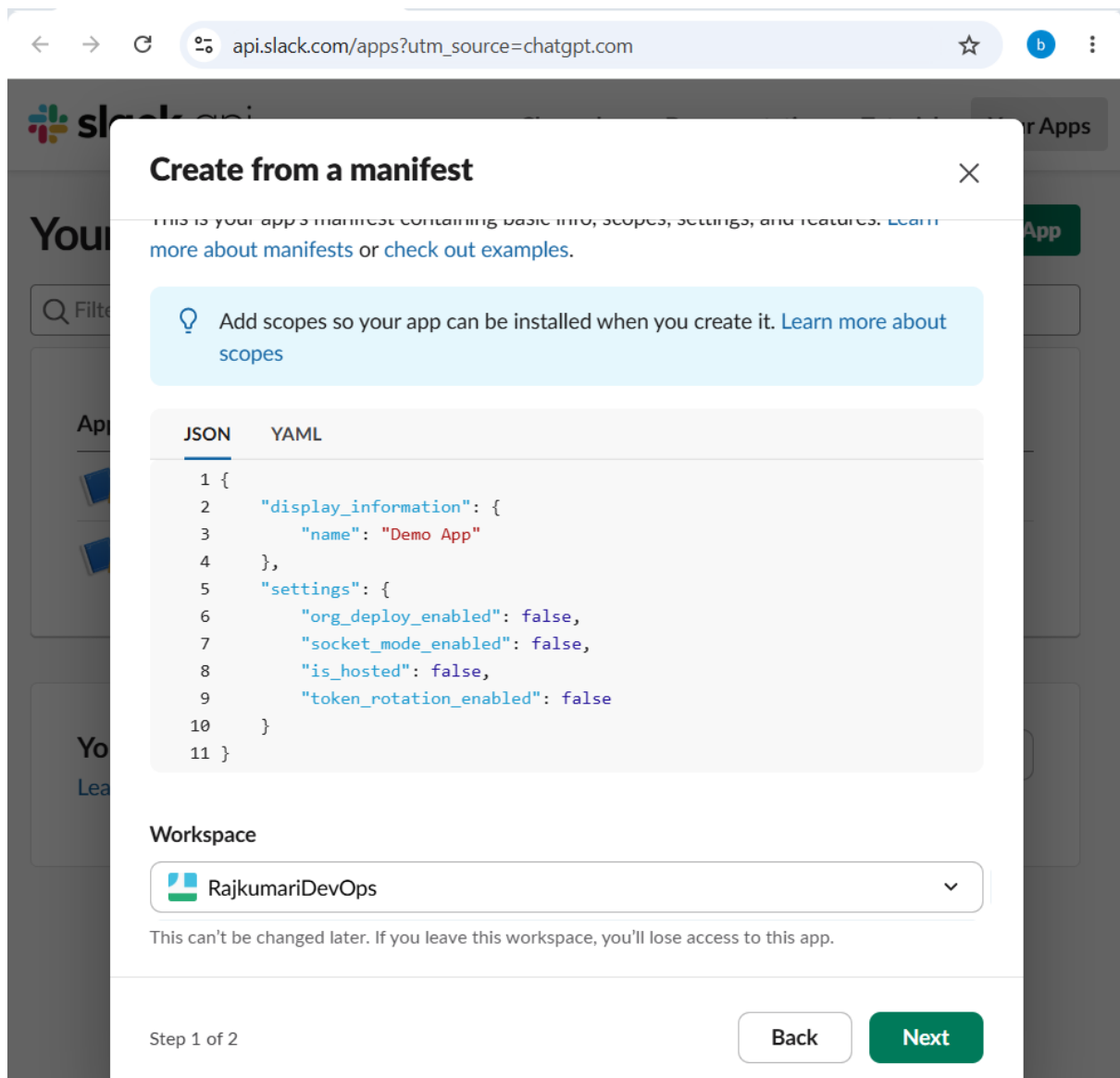
|— #general

|— #random

|— #jenkins-alerts

You don't need to ask anyone for permission because **this is your workspace**.





3. Create the Slackraji app

Now we'll create your Slack app inside your **new workspace**, not TechieHorizon.

Open:

[Slack API – Your Apps](#)

Click:

Create New App

Choose:

From scratch

Enter:

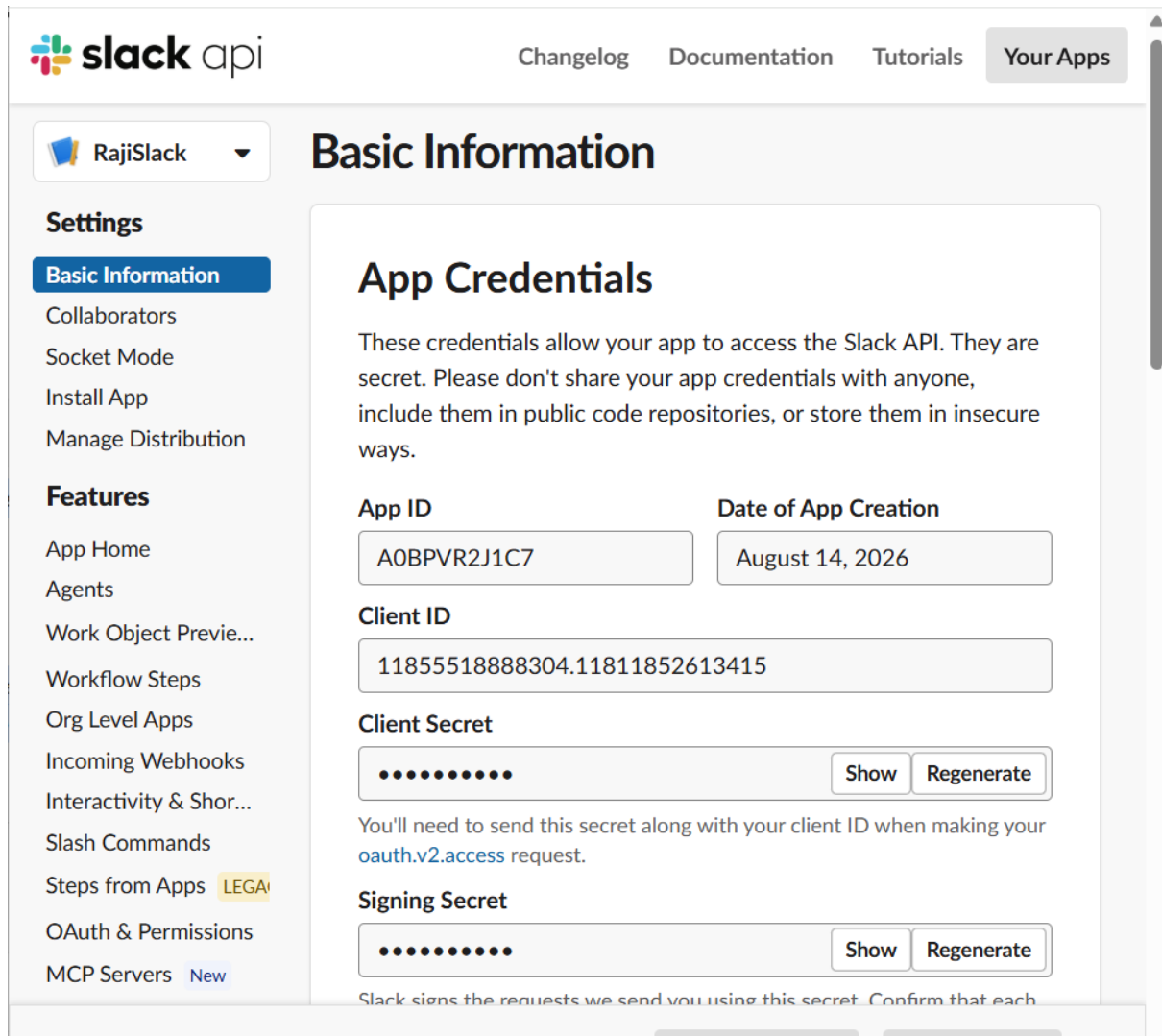
App Name: Slackraji

For the workspace, select your **new personal workspace**.

Then click:

Create App

Slack officially supports creating an app from the app settings and selecting the workspace where the app will be developed.



The screenshot shows the 'Basic Information' page for a Slack app named 'RajiSlack'. The page is divided into a left sidebar with 'Settings' and 'Features' sections, and a main content area titled 'App Credentials'. The 'App Credentials' section contains the following information:

- App ID:** A0BPVR2J1C7
- Date of App Creation:** August 14, 2026
- Client ID:** 11855518888304.11811852613415
- Client Secret:** A masked field with a 'Show' and 'Regenerate' button. Below it, a note states: 'You'll need to send this secret along with your client ID when making your [oauth.v2.access](#) request.'
- Signing Secret:** A masked field with a 'Show' and 'Regenerate' button. Below it, a note states: 'Slack signs the requests we send you using this secret. Confirm that each'

4. Configure Slackraji permissions

Inside Slackraji, go to:

OAuth & Permissions

Scroll down to:

Bot Token Scopes

For the Jenkins Slack plugin, the basic useful scopes are:

channels:read

chat:write

If you're following the Jenkins plugin's custom Slack app setup, the plugin documentation lists these scopes:

channels:read

chat:write

chat:write.customize

files:write

reactions:write

users:read

users:read.email

groups:read

The Jenkins plugin documentation specifically recommends these for its custom Slack app configuration.

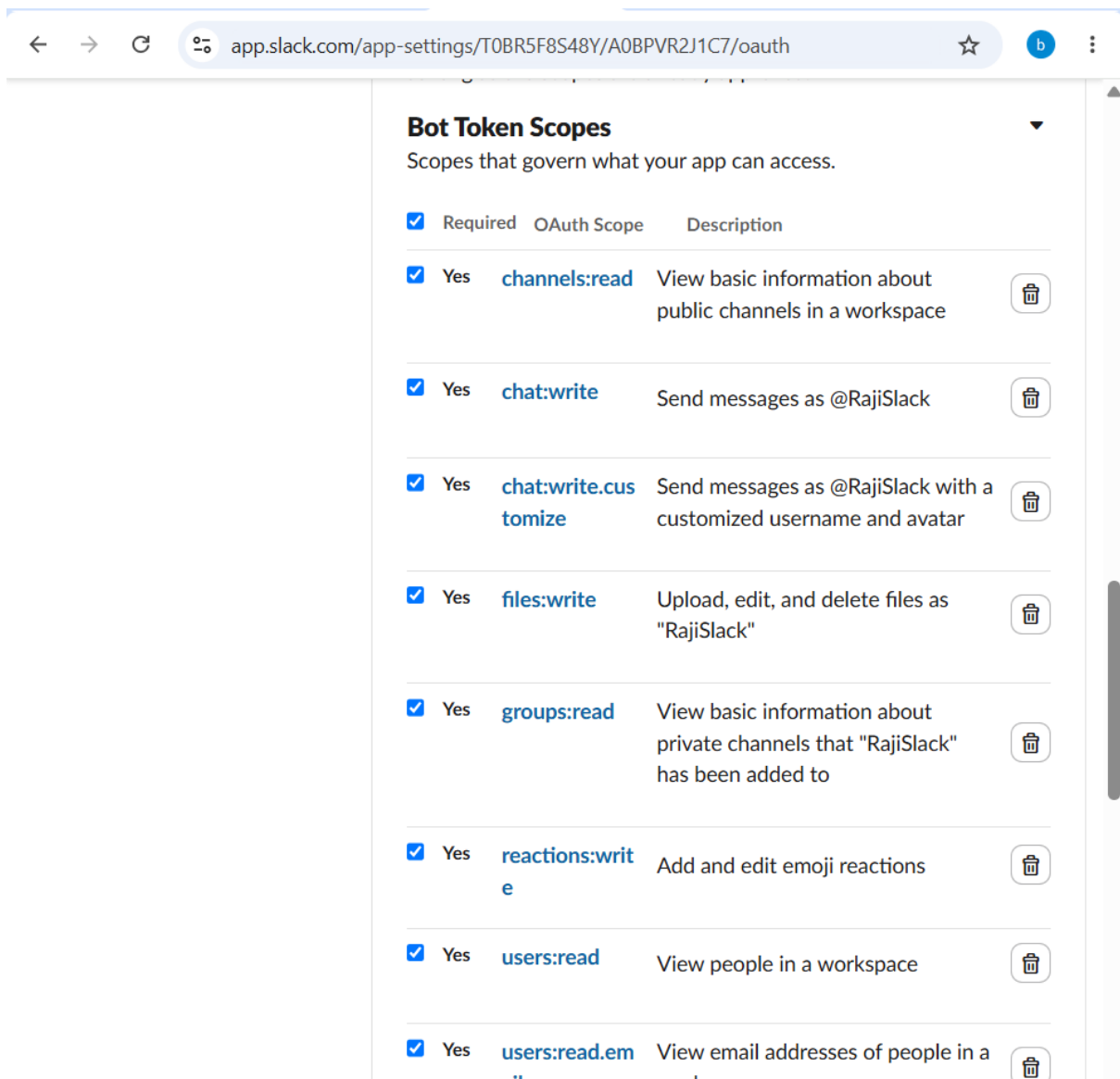
For your first test

I recommend keeping it simple:

channels:read

chat:write

You can add the others later if you need those features.



5. Install Slackraji into your workspace

Go to:

Settings → Install App

You should see:

Install to Workspace

Click it.

Slack will show the permissions being requested.

Click:

Allow

Now Slackraji is installed.

Go back to:

OAuth & Permissions

You should now see:

OAuth Tokens

Something similar to:

Bot User OAuth Token

xoxb-*****

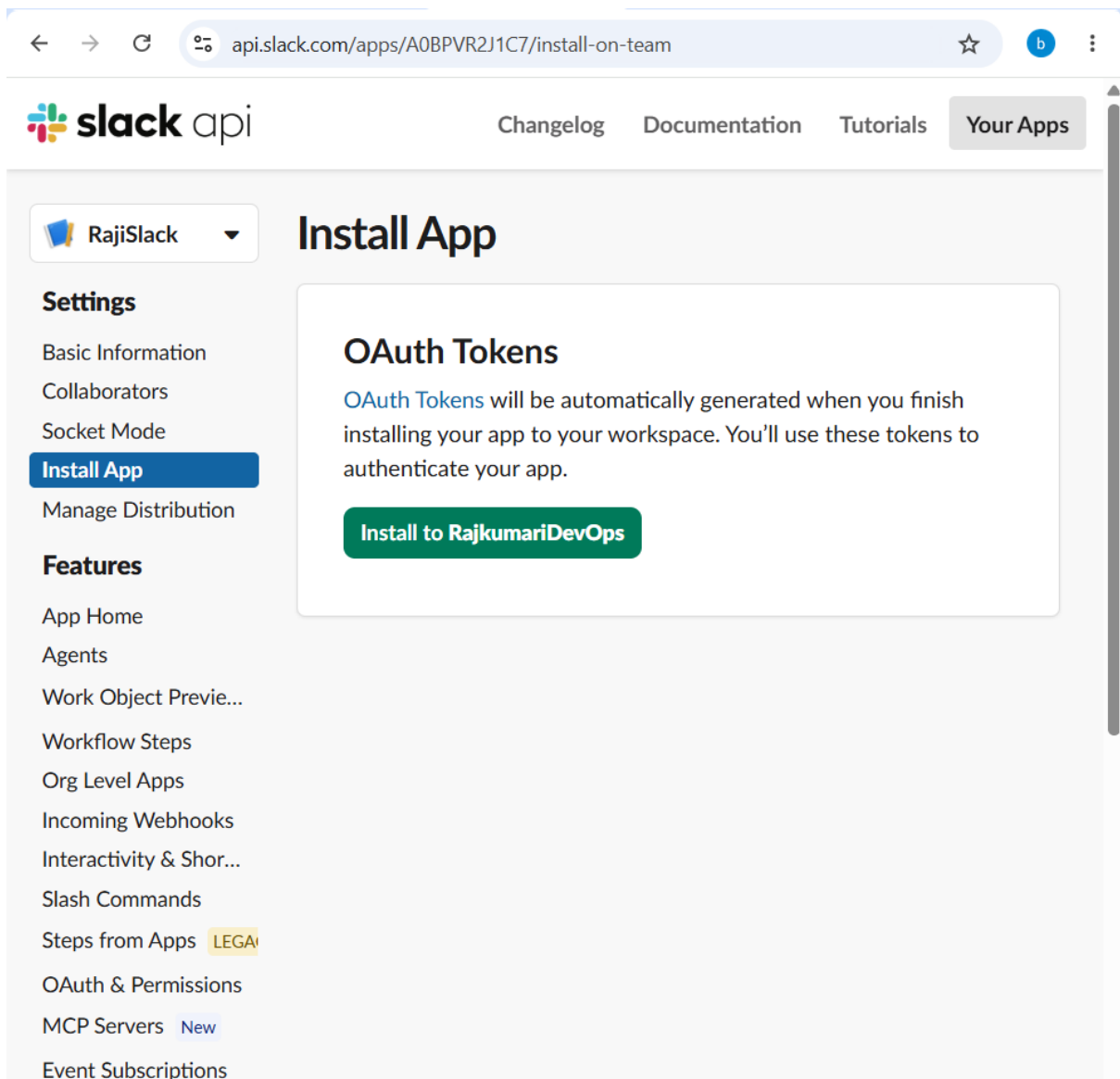
That is your **Bot User OAuth Token**.

⚠ Important

Do **not** send the xoxb-... token to me or put it in your Jenkinsfile.

We'll store it securely in **Jenkins Credentials**.

The Jenkins plugin documentation specifically recommends storing the Slack token as a Jenkins **Secret text** credential rather than exposing the token in configuration/code.



6. Add Slackraji to #jenkins-alerts

Now open Slack.

Go to:

#jenkins-alerts

Add/invite your app:

@Slackraji

Depending on your Slack UI, you can use:

Channel name → Members → Add people/apps

and select **Slackraji**.

This is important because the Jenkins Slack plugin needs the bot to be a member of the channel you want it to notify.

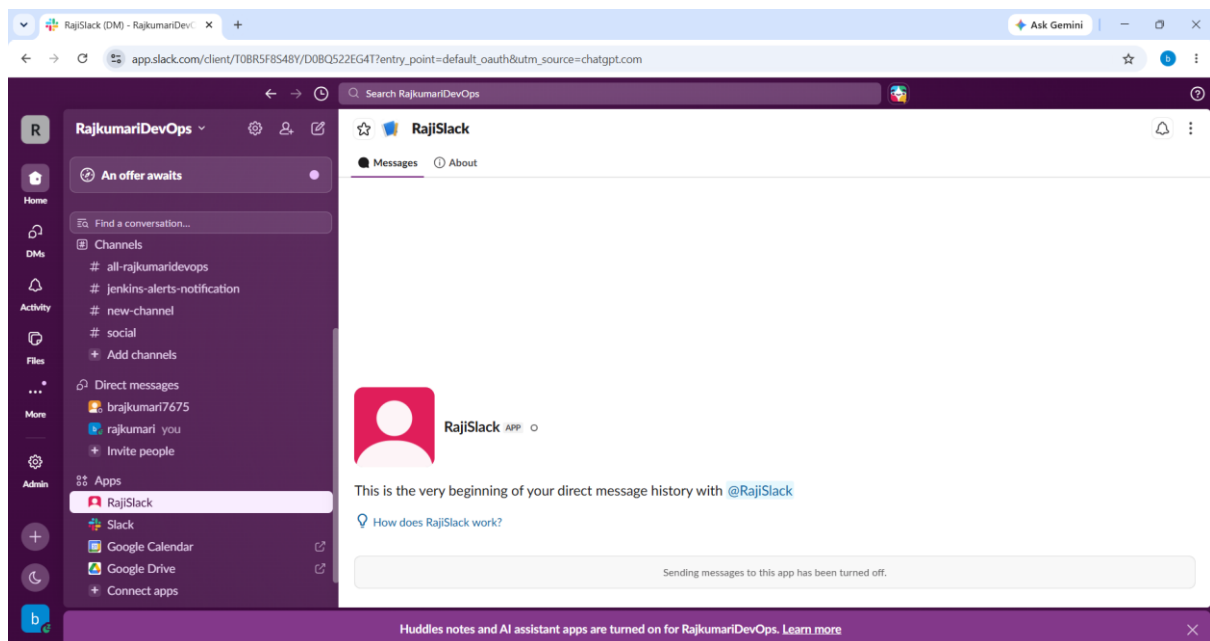
Your setup should now look like:

Rajkumari DevOps Workspace

Slackraji



#jenkins-alerts



7. Install the Jenkins Slack plugin

Now go to your Jenkins server.

Open:

Manage Jenkins

→ **Plugins**

→ **Available plugins**

Search:

Slack Notification

Install the Jenkins **Slack Notification** plugin.

The official Jenkins plugin instructions use:

Manage Jenkins → Manage Plugins → Available → search for Slack.

Restart Jenkins if Jenkins asks you to.

8. Add the Slack token to Jenkins Credentials

Now go to:

Manage Jenkins → Credentials

Select your Jenkins credential store, normally:

Jenkins → Global credentials

Click:

Add Credentials

Set:

Kind

Secret text

Secret

Paste your:

xoxb-...

Bot User OAuth Token.

ID

Use:

slack-token

Description

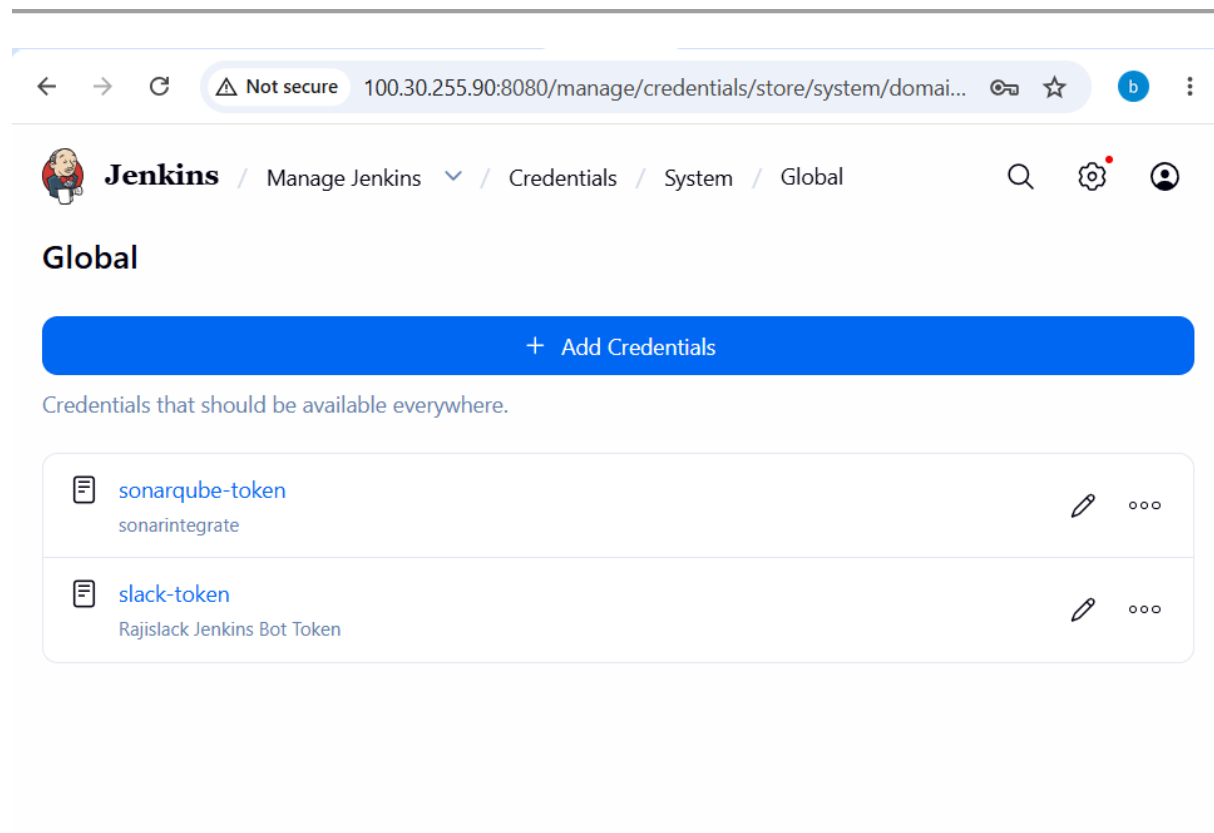
For example:

Slackraji Jenkins Bot Token

Click:

Create

Your token is now stored securely in Jenkins.



9. Configure Slack in Jenkins

Go to:

Manage Jenkins → System

Find:

Slack Notifications

Configure:

Workspace/Team Domain

Enter your workspace name.

For example, if your workspace URL is:

`https://rajkumari-devops.slack.com`

the domain is:

rajkumari-devops

Don't enter:

https://

and don't enter:

.slack.com

Credential

Select:

slack-token

Default channel

Enter:

jenkins-alerts

Custom Slack App Bot User

Since you're using your own Slackraji custom app, enable:

Custom slack app bot user

The Jenkins plugin documentation specifically calls out the custom Slack app bot-user setting and the Secret Text credential approach.

Then click:

Test Connection

← → ↻ ⚠ Not secure 100.30.255.90:8080/manage/configure ☆ b ⋮

 **Jenkins** / Manage Jenkins ▾ / System ▾

🔍 ⚙️ 👤

Advanced ▾  Edited

☐ Test configuration by sending test e-mail

Slack

Workspace ?

Credential ?

▾

+ Add

Default channel / member id ?

☒ Custom slack app bot user ?

Advanced ▾

Test Connection

Save

Apply

```
pipeline {  
  agent any  
  
  stages {  
  
    stage('Git Clone') {  
      steps {
```

```

        git branch: 'master',

        url: 'https://github.com/rajkumari-hub/VProfile-1.git'
    }
}

stage('Build') {
    steps {
        sh 'mvn clean verify'
    }
}

stage('SonarQube Analysis') {
    steps {
        withSonarQubeEnv('SonarQube') {
            sh '''
                mvn org.sonarsource.scanner.maven:sonar-maven-plugin:sonar \
                -Dsonar.projectKey=VProfile \
                -Dsonar.projectName=VProfile
            '''
        }
    }
}

post {
    success {

```

```

slackSend(

    channel: '#jenkins-alerts',

    color: 'good',

    message: "SUCCESS: ${env.JOB_NAME} #${env.BUILD_NUMBER}"

)

}

failure {

    slackSend(

        channel: '#jenkins-alerts',

        color: 'danger',

        message: "FAILURE: ${env.JOB_NAME} #${env.BUILD_NUMBER}"

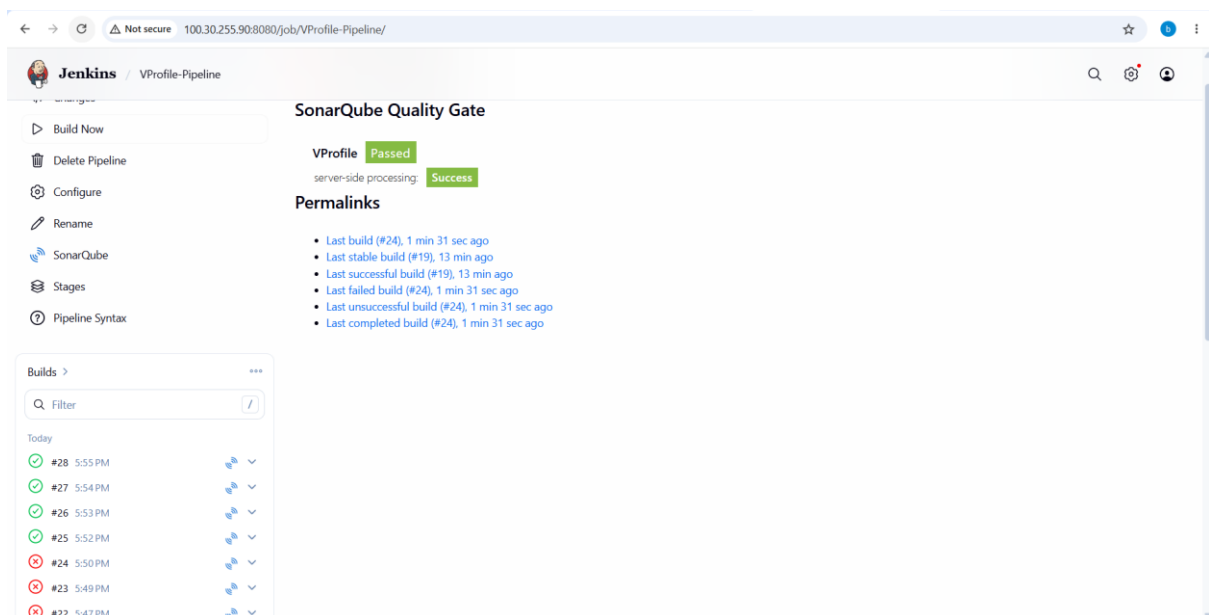
    )

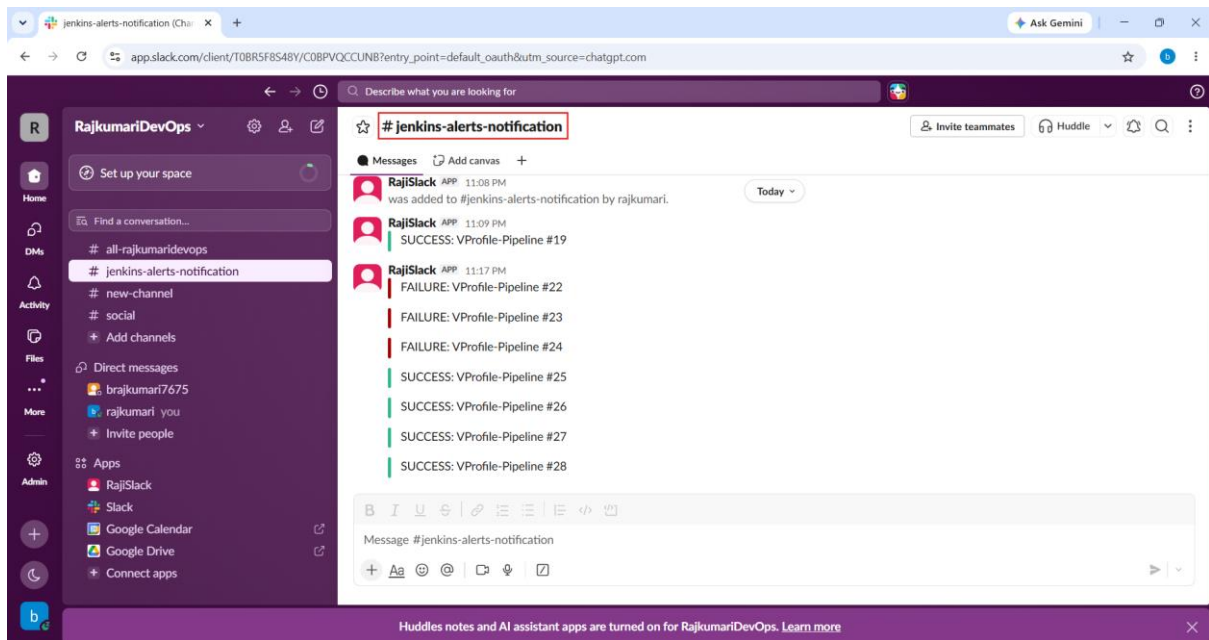
}

}

}

```





2. Create one Jenkins job using the below code and create three stages.

- stage1: Git clone to download the source code.
- stage2: Sonarqube Integration to check the quality of code.
- stage3: Slack Integration to send the alerts to slack.

Source Code: <https://github.com/betawins/hiring-app.git>

stage1: Git clone to download the source code.

Stage 1: Git Clone – Download Source Code

1. Objective

The objective of this stage is to **clone/download the application source code from GitHub into the Jenkins workspace** so that Jenkins can use the source code for the subsequent CI/CD stages such as build, testing, code analysis, packaging, deployment, and Slack notification.

2. Source Code

GitHub Repository:

<https://github.com/betawins/hiring-app.git>

The hiring-app repository contains the application source code that will be processed by the Jenkins Pipeline.

3. Tools Required

- Jenkins
- Git
- GitHub
- Jenkins Pipeline
- Git Plugin

Jenkins uses the Git plugin to perform Git operations such as clone, fetch, checkout, and branch operations.

4. Process Flow

GitHub Repository

|

| Git Clone

v

Jenkins

|

v

Jenkins Workspace

|

v

Application Source Code

5. Step 1 – Verify Git Installation

Login to the Jenkins server and verify that Git is installed:

```
git --version
```

Expected output:

```
git version 2.x.x
```

If Git is not installed on Amazon Linux, install it using:

```
sudo dnf install git -y
```

Then verify:

```
git --version
```

6. Step 2 – Create Jenkins Pipeline Job

1. Login to Jenkins.
2. Click **New Item**.
3. Enter the job name.

Example:

Hiring-App-CI

4. Select:

Pipeline

5. Click **OK**.

Jenkins Pipeline allows the CI/CD process to be defined as code using a Jenkinsfile.

7. Step 3 – Create the Git Clone Stage

Add the following stage to the Jenkinsfile:

```
stage('Git Clone') {  
    steps {  
        git branch: 'master',  
        url: 'https://github.com/betawins/hiring-app.git'
```

```
}  
}
```

The git Pipeline step accepts the repository URL and branch and checks the source code out into the Pipeline workspace.

Note: If your GitHub repository uses a different default branch, such as main, replace master with main.

8. Complete Jenkinsfile for Stage 1

For testing Stage 1 independently, you can use:

```
pipeline {  
    agent any  
  
    stages {  
  
        stage('Git Clone') {  
            steps {  
                git branch: 'master',  
                    url: 'https://github.com/betawins/hiring-app.git'  
            }  
        }  
    }  
}
```

9. Step 4 – Save the Pipeline

After entering the Jenkinsfile:

1. Click **Save**.

2. Open the Jenkins job.
3. Click **Build Now**.

Jenkins will allocate a workspace and execute the Pipeline stage. The agent directive provides Jenkins with an executor and workspace for the Pipeline.

10. Step 5 – Verify the Console Output

Open:

Jenkins Job



Build #1



Console Output

You should see output similar to:

[Pipeline] stage

[Pipeline] { (Git Clone)

[Pipeline] git

The recommended git tool is: NONE

Cloning the remote Git repository

Cloning repository <https://github.com/betawins/hiring-app.git>

Fetching upstream changes

Checking out Revision ...

[Pipeline] }

The exact commit ID and Git messages will vary depending on the current repository state.

11. Step 6 – Verify the Jenkins Workspace

On the Jenkins server, the source code will be available inside the job workspace.

For example:

```
cd /var/lib/jenkins/workspace/Hiring-App-CI
```

List the files:

```
ls -la
```

You should see the files and directories from the hiring-app repository.

For example:

```
.
```

```
..
```

```
.git
```

```
src
```

```
pom.xml
```

```
...
```

The exact files depend on the repository structure.

12. Step 7 – Verify Git Clone Manually

If required for troubleshooting, you can test the repository separately:

```
cd /tmp
```

```
git clone https://github.com/betawins/hiring-app.git
```

Then:

```
cd hiring-app
```

```
ls
```

This confirms that the repository can be accessed using Git.

13. Authentication

If the GitHub repository is **public**, no Jenkins credentials are normally required for cloning over HTTPS.

If the repository is **private**, configure GitHub credentials in Jenkins and reference the credential from the Pipeline.

Example:

```
git credentialsId: 'github-credentials',  
    branch: 'master',  
    url: 'https://github.com/betawins/hiring-app.git'
```

Jenkins recommends using a Jenkins credential for secured repositories instead of embedding credentials in the repository URL.

14. Expected Result

After successful execution:

GitHub

|

| Clone

v

Jenkins

|

| Checkout source code

v

Jenkins Workspace

|

v

Hiring App Source Code

Jenkins should display:

Finished: SUCCESS

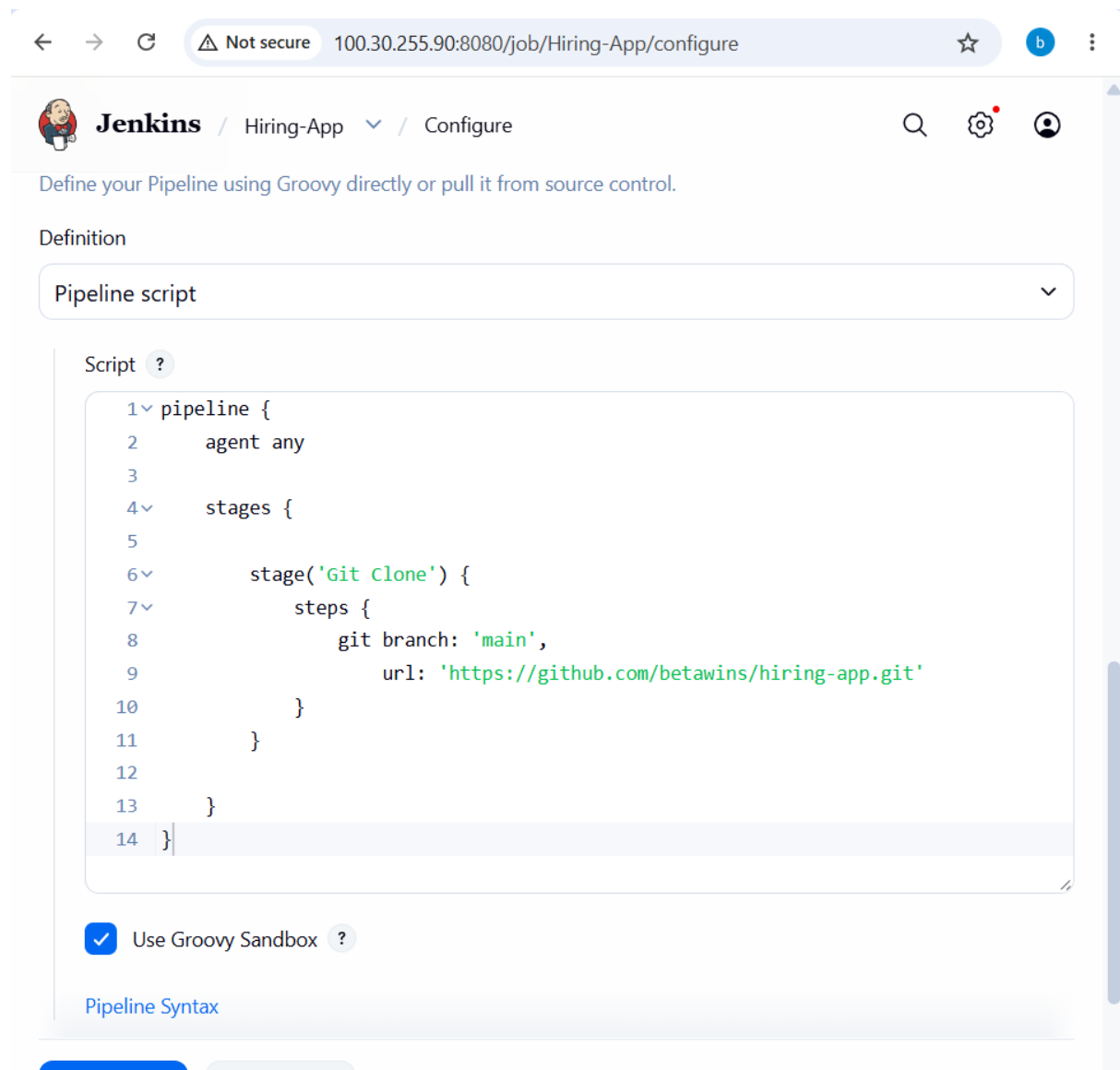
15. Observation

The Git Clone stage successfully downloads the hiring-app source code from the GitHub repository into the Jenkins workspace. This source code becomes the input for the next stages of the CI/CD pipeline.

16. Conclusion

Stage 1 successfully establishes the connection between **GitHub and Jenkins**. Jenkins retrieves the latest source code from the specified repository and places it in the Jenkins workspace.

This stage is the **first step of the CI/CD pipeline**, because all subsequent activities such as Maven build, testing, SonarQube analysis, deployment, and Slack notifications depend on the source code obtained during this stage.



The screenshot shows the Jenkins web interface for configuring a pipeline. The browser address bar indicates the URL `100.30.255.90:8080/job/Hiring-App/configure`. The Jenkins logo and breadcrumb navigation (`Hiring-App / Configure`) are visible. Below the navigation bar, there is a search icon, a settings icon, and a user profile icon. The main heading is "Define your Pipeline using Groovy directly or pull it from source control." Below this, the "Definition" section is expanded, showing a dropdown menu for "Pipeline script". The "Script" section is active, displaying a Groovy pipeline script in a text editor. The script defines a pipeline with an agent and a single stage named "Git Clone" that clones the repository from GitHub. The "Use Groovy Sandbox" checkbox is checked. A "Pipeline Syntax" link is located at the bottom of the script editor.

```
1 pipeline {
2   agent any
3
4   stages {
5
6     stage('Git Clone') {
7       steps {
8         git branch: 'main',
9           url: 'https://github.com/betawins/hiring-app.git'
10      }
11    }
12
13  }
14 }
```

☒ Use Groovy Sandbox ?

[Pipeline Syntax](#)

← → ↻ ⚠ Not secure 100.30.255.90:8080/job/Hiring-App/2/console ☆ b ⋮

 **Jenkins** / Hiring-App ▾ / #2 ▾ 🔍 ⚙️ 👤

```
No credentials specified
> git rev-parse --resolve-git-dir /var/lib/jenkins/workspace/Hiring-App/.git #
timeout=10
Fetching changes from the remote Git repository
> git config remote.origin.url https://github.com/betawins/hiring-app.git # timeout=10
Fetching upstream changes from https://github.com/betawins/hiring-app.git
> git --version # timeout=10
> git --version # 'git version 2.50.1'
> git fetch --tags --force --progress -- https://github.com/betawins/hiring-app.git
+refs/heads/*:refs/remotes/origin/* # timeout=10
> git rev-parse refs/remotes/origin/main^{commit} # timeout=10
Checking out Revision 9405b107fdffdf68e69fe953ea1f9669784f9d6a (refs/remotes/origin/main)
> git config core.sparsecheckout # timeout=10
> git checkout -f 9405b107fdffdf68e69fe953ea1f9669784f9d6a # timeout=10
> git branch -a -v --no-abbrev # timeout=10
> git checkout -b main 9405b107fdffdf68e69fe953ea1f9669784f9d6a # timeout=10
Commit message: "Update README.md"
First time build. Skipping changelog.
[Pipeline] }
[Pipeline] // stage
[Pipeline] }
[Pipeline] // node
[Pipeline] End of Pipeline
Finished: SUCCESS
```

← → ↻ ⚠ Not secure 100.30.255.90:8080/job/Hiring-App/ ☆ b ⋮

 **Jenkins** / Hiring-App 🔍 ⚙️ 👤

Status

</> Changes

▶ Build Now

🗑 Delete Pipeline

⚙ Configure

✎ Rename

📅 Stages

🔗 Pipeline Syntax

🟢 Hiring-App

Add description

Permalinks

- Last build (#2), 2 min 26 sec ago
- Last stable build (#2), 2 min 26 sec ago
- Last successful build (#2), 2 min 26 sec ago
- Last failed build (#1), 3 min 56 sec ago
- Last unsuccessful build (#1), 3 min 56 sec ago
- Last completed build (#2), 2 min 26 sec ago

Builds > ⋮

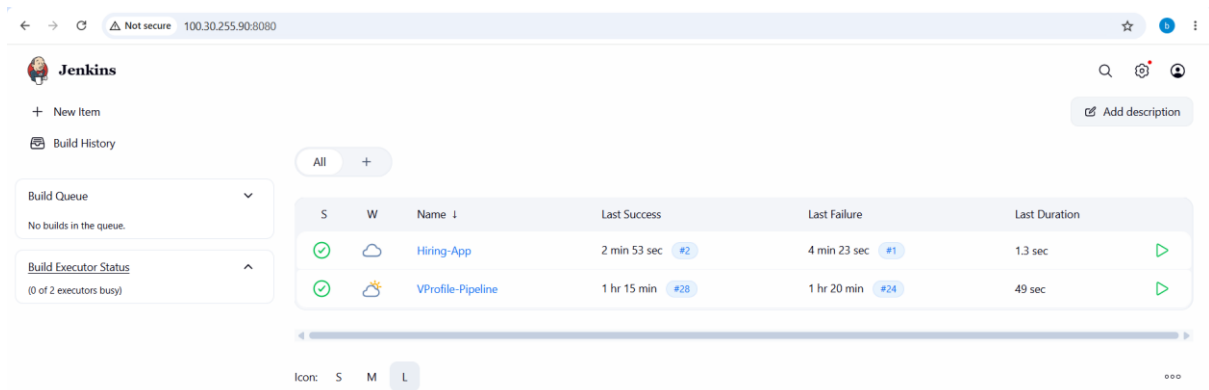
🔍 Filter /

Today

🟢 #2 7:07 PM ▾

🔴 #1 7:06 PM ▾

REST API Jenkins 2.568.2



stage2: Sonarqube Integration to check the quality of code.

Source Code: <https://github.com/betawins/hiring-app.git>

Stage 2: SonarQube Integration – Code Quality Analysis

1. Objective

The objective of Stage 2 is to integrate **Jenkins with SonarQube** and analyze the source code downloaded from GitHub.

SonarQube performs static code analysis to identify code-quality issues, bugs, vulnerabilities, code smells, and other maintainability problems.

The flow is:

GitHub



Stage 1: Git Clone



Jenkins Workspace



Stage 2: SonarQube Analysis



SonarQube Server



Code Quality Report

2. Source Code

GitHub Repository:

<https://github.com/betawins/hiring-app.git>

This repository is cloned during **Stage 1 – Git Clone**.

Stage 2 analyzes the source code that is already present in the Jenkins workspace.

3. Prerequisites

Before configuring Stage 2, make sure you have:

- Jenkins server
- Jenkins Pipeline job
- Git installed
- Maven installed
- Java installed
- SonarQube server running
- SonarQube project
- SonarQube authentication token
- Jenkins SonarQube Scanner plugin

The SonarQube Jenkins integration requires the SonarQube Scanner plugin, and SonarSource recommends configuring the SonarQube server and authentication credentials globally in Jenkins.

4. Step 1 – Verify SonarQube Server

Open your SonarQube server in a browser.

For example:

`http://<SONARQUBE-IP>:9000`

Example:

`http://3.239.40.138:9000`

Use your actual SonarQube server IP address.

Login to SonarQube.

5. Step 2 – Create SonarQube Project

In SonarQube:

1. Login.
2. Go to **Projects**.
3. Click **Create Project**.
4. Select **Manually** if prompted.
5. Enter a project key.

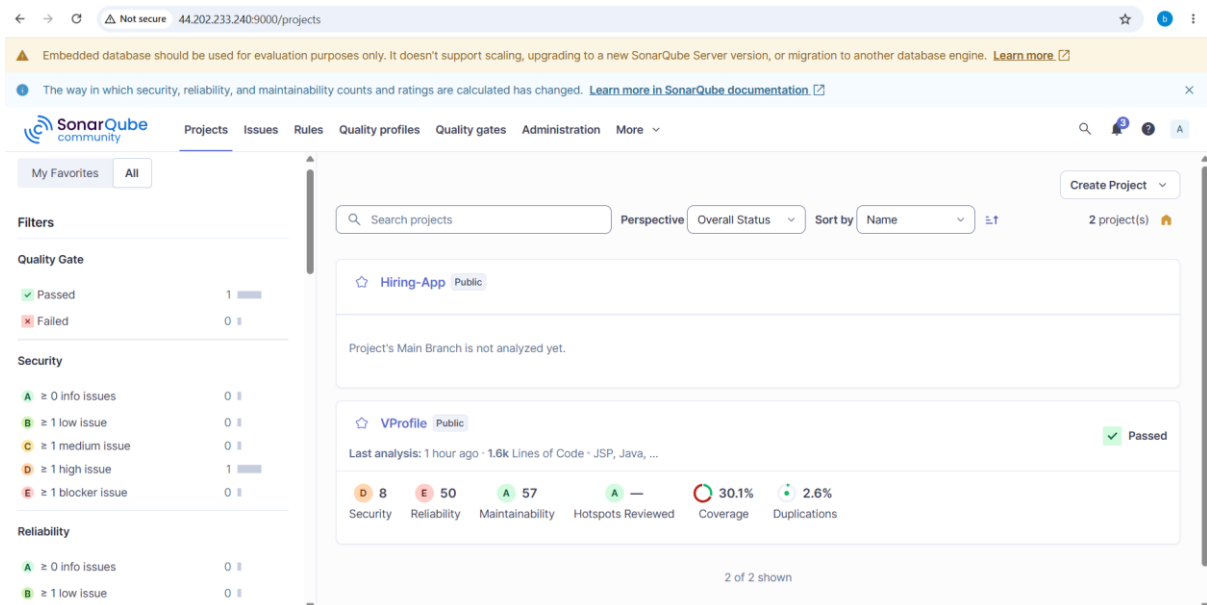
For example:

hiring-app

Project name:

Hiring-App

Click **Create Project**.



6. Step 3 – Generate SonarQube Token

The Jenkins integration needs authentication to communicate with SonarQube.

In SonarQube:

1. Click your profile.
2. Go to **My Account**.
3. Open **Security**.
4. Generate a token.
5. Give it a name.

Example:

jenkins-token

6. Click **Generate**.
7. Copy the token.

Important

Do not put the token directly into your Jenkinsfile.

Store it in Jenkins Credentials as a **Secret Text** credential. SonarSource's Jenkins setup documentation recommends this approach.

 Embedded database should be used for evaluation purposes only. It doesn't support scaling, upgrading t

 [Projects](#) [Issues](#) [Rules](#) [Quality profiles](#) [Quality gates](#) [Administrati](#)

A

My account / Security

Security



Name

Enter Token Name

Type

Select Token Type

Expires in

30 days

Generate

 New token "jenkins-token-key"

Click to copy to clipboard

 sure you copy it now, you won't b

sqa_3072c687fb14eabb87ae7aecb4173e65488108e3



Name	Type	Project	Last use
jenkins-vprofile	Global		1 hour a
jenkins-token-key	Global		Never

Enter a new password

7. Step 4 – Install SonarQube Scanner Plugin in Jenkins

In Jenkins:

Manage Jenkins

↓

Plugins

↓

Available Plugins

Search for:

SonarQube Scanner

Install the **SonarQube Scanner** plugin.

The official Jenkins plugin provides integration with SonarQube Server, SonarQube Cloud, and SonarQube Community Build.

Restart Jenkins if required.

8. Step 5 – Add SonarQube Token to Jenkins Credentials

Go to:

Jenkins



Manage Jenkins



Credentials



System



Global credentials

Click:

Add Credentials

Select:

Kind: Secret text

Enter the SonarQube token in:

Secret

Enter an ID such as:

sonarqube-token

Description:

SonarQube authentication token

Click:

Create

← → ↻ ⚠ Not secure 100.30.255.90:8080/manage/credentials/store/system/domai... 🔑 ☆ b

 **Jenkins** / Manage Jenkins ▾ / Credentials / System / Global 🔍 ⚙️ 👤

Global

[+ Add Credentials](#)

Credentials that should be available everywhere.

 sonarqube-token sonarintegrate	 ⋮
 slack-token Rajislack Jenkins Bot Token	 ⋮
 sonarhiringapp-token SonarQube authentication token	 ⋮

REST API

Jenkins 2.568

9. Step 6 – Configure SonarQube Server in Jenkins

Go to:

Manage Jenkins



System

Find:

SonarQube servers

Click:

Add SonarQube

Enter:

Name:

SonarQube

For the server URL, enter your actual SonarQube URL:

`http://<SONARQUBE-IP>:9000`

Example:

`http://3.239.40.138:9000`

Select the Jenkins credential:

`sonarqube-token`

Save the configuration.

SonarSource documents this global configuration as the way Jenkins provides the SonarQube connection details to Pipeline analysis through `withSonarQubeEnv`.

The screenshot shows the Jenkins web interface in a browser. The address bar indicates the URL is 100.30.255.90:8080/manage/configure. The Jenkins logo and navigation links are at the top. The 'System' tab is selected, showing 'Environment variables' and 'SonarQube installations'. Under 'SonarQube installations', there is a 'List of SonarQube installations' section. A configuration card for 'SonarQube' is visible, containing fields for 'Name' (SonarQube), 'Server URL' (http://172.31.8.227:9000), and 'Server authentication token' (SonarQube authentication token). There is also an 'Advanced' dropdown and a '+ Add' button. At the bottom, there are 'Save' and 'Apply' buttons.

← → ↻ ⚠ Not secure 100.30.255.90:8080/manage/configure ☆ b

Jenkins / Manage Jenkins ▾ / System ▾ 🔍 ⚙️ 👤

☐ Environment variables

SonarQube installations

List of SonarQube installations

Name

SonarQube

Server URL

Default is http://localhost:9000

http://172.31.8.227:9000

Server authentication token

SonarQube authentication token. Mandatory when anonymous access is disabled.

SonarQube authentication token ▾ + Add

Advanced ▾

Save Apply

10. Step 7 – Configure Maven

Because hiring-app is a Maven application, verify Maven on the Jenkins server.

Run:

```
mvn -version
```

You should see something similar to:

Apache Maven 3.x.x

Java version: ...

If Maven is not installed, configure/install Maven before running the analysis.

11. Step 8 – Add SonarQube Stage to Jenkins Pipeline

```
pipeline {
    agent any

    stages {

        stage('Git Clone') {
            steps {
                git branch: 'main',
                    url: 'https://github.com/betawins/hiring-app.git'
            }
        }

        stage('SonarQube Analysis') {
            steps {
                withSonarQubeEnv('SonarQube') {
                    sh '''
                        mvn clean verify org.sonarsource.scanner.maven:sonar-maven-
plugin:sonar \
                        -Dsonar.projectKey=Hiring-App \
                        -Dsonar.projectName=Hiring-App
                    '''
                }
            }
        }
    }
}
```

```
}
```

}SonarQube must match the **Name** you configured under Jenkins → System → SonarQube servers.

The withSonarQubeEnv step supplies the configured SonarQube connection information to the analysis command.

12. Complete Pipeline for Stage 1 + Stage 2

For your current project, you can use:

```
pipeline {
    agent any

    stages {

        stage('Git Clone') {
            steps {
                git branch: 'main',
                    url: 'https://github.com/betawins/hiring-app.git'
            }
        }

        stage('SonarQube Analysis') {
            steps {
                withSonarQubeEnv('SonarQube') {
                    sh '''
                        mvn clean verify org.sonarsource.scanner.maven:sonar-maven-
plugin:sonar \
```



```
-Dsonar.projectKey=Hiring-App \
-Dsonar.projectName=Hiring-App
'''

}

}

}
```

Important

If your repository branch is not main, use the branch that you verified in Stage 1.

For example, if your repository uses master:

```
git branch: 'master',
```

```
url: 'https://github.com/betawins/hiring-app.git'
```

13. Step 9 – Save the Pipeline

In Jenkins:

Pipeline Job

↓

Configure

↓

Pipeline

Paste the updated Pipeline script.

Click:

Apply

Save

The screenshot shows the Jenkins web interface in a browser. The address bar indicates the URL is `100.30.255.90:8080/job/Hiring-App/configure`. The page title is "Jenkins / Hiring-App / Configure". Under the "Pipeline" section, the instruction "Define your Pipeline using Groovy directly or pull it from source control." is shown. The "Definition" dropdown is set to "Pipeline script". A script editor displays the following Groovy code:

```
10     }
11 }
12
13 stage('SonarQube Analysis') {
14     steps {
15         withSonarQubeEnv('SonarQube') {
16             sh '''
17                 mvn clean verify org.sonarsource.scanner.maven:sonar-maven-plugin:
18                 -Dsonar.projectKey=Hiring-App \
19                 -Dsonar.projectName=Hiring-App
20             '''
21         }
22     }
23 }
24
```

Below the script editor, the checkbox "Use Groovy Sandbox" is checked. At the bottom, there are "Save" and "Apply" buttons.

14. Step 10 – Run the Pipeline

Click:

Build Now

Jenkins will execute:

Stage 1

Git Clone

↓

Stage 2

SonarQube Analysis

15. Step 11 – Check Console Output

Open:

Build #X



Console Output

You should see:

[Pipeline] stage

[Pipeline] { (Git Clone)

...

Cloning repository <https://github.com/betawins/hiring-app.git>

...

[Pipeline] }

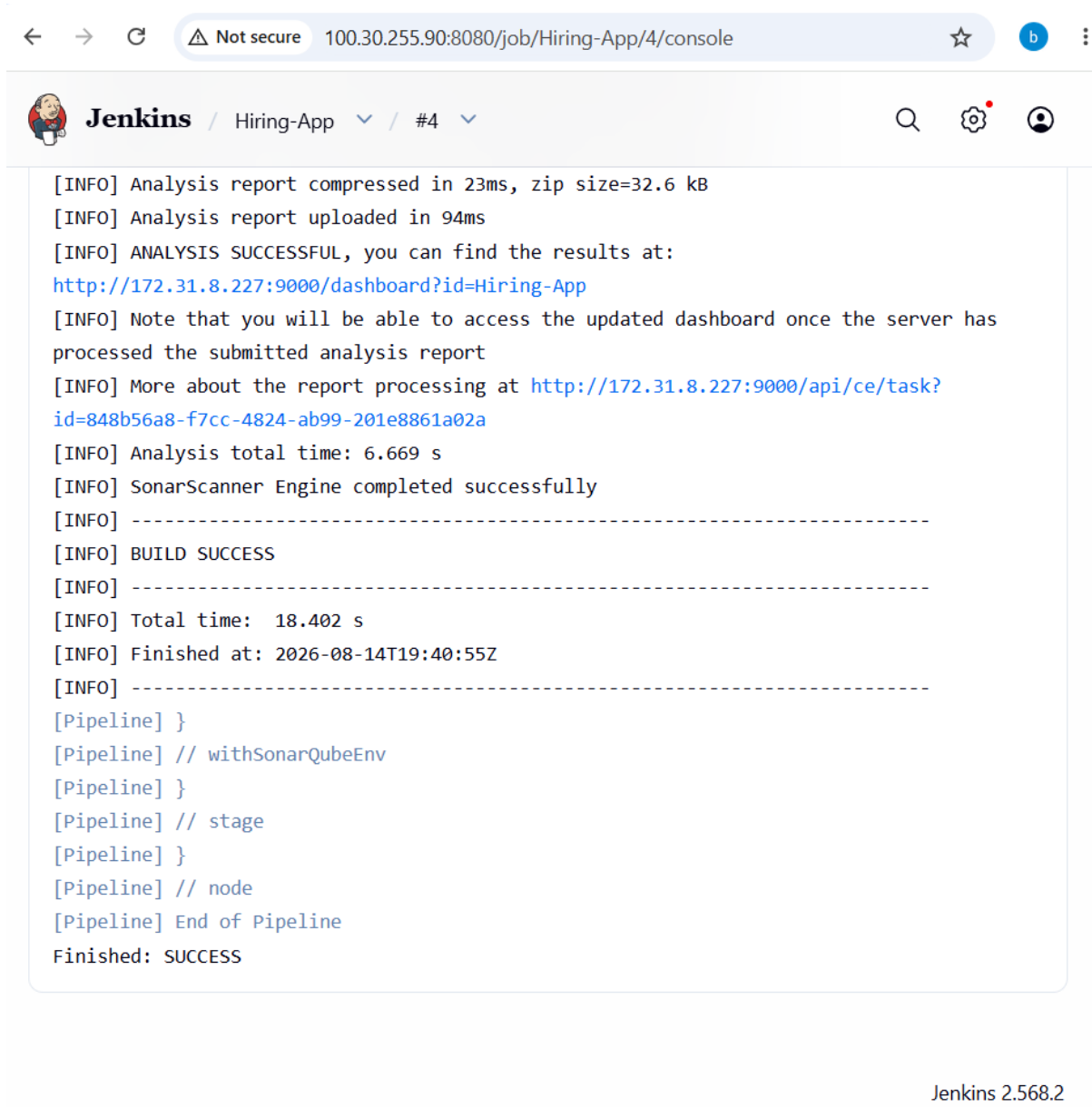
[Pipeline] stage

[Pipeline] { (SonarQube Analysis)

...

Then Maven will compile/verify the project and execute the SonarQube analysis.

You should eventually see a message indicating that the analysis was submitted successfully.



The screenshot shows a web browser window with the address bar displaying '100.30.255.90:8080/job/Hiring-App/4/console'. The Jenkins interface header shows the 'Jenkins' logo, the job name 'Hiring-App', and the build number '#4'. The console output contains the following log messages:

```
[INFO] Analysis report compressed in 23ms, zip size=32.6 kB
[INFO] Analysis report uploaded in 94ms
[INFO] ANALYSIS SUCCESSFUL, you can find the results at:
http://172.31.8.227:9000/dashboard?id=Hiring-App
[INFO] Note that you will be able to access the updated dashboard once the server has
processed the submitted analysis report
[INFO] More about the report processing at http://172.31.8.227:9000/api/ce/task?id=848b56a8-f7cc-4824-ab99-201e8861a02a
[INFO] Analysis total time: 6.669 s
[INFO] SonarScanner Engine completed successfully
[INFO] -----
[INFO] BUILD SUCCESS
[INFO] -----
[INFO] Total time: 18.402 s
[INFO] Finished at: 2026-08-14T19:40:55Z
[INFO] -----
[Pipeline] }
[Pipeline] // withSonarQubeEnv
[Pipeline] }
[Pipeline] // stage
[Pipeline] }
[Pipeline] // node
[Pipeline] End of Pipeline
Finished: SUCCESS
```

Jenkins 2.568.2

16. Step 12 – Verify the Analysis in SonarQube

Open SonarQube:

<http://<SONARQUBE-IP>:9000>

Go to:

Projects

Open:

Hiring-App

You should see the analysis results.

Typical information includes:

- Bugs
- Vulnerabilities
- Code Smells
- Security Hotspots
- Coverage
- Duplications
- Lines of Code
- Quality Gate

17. Step 13 – Configure Quality Gate

If your objective is only:

"SonarQube Integration to check the quality of code"

then Stage 2 can finish after the analysis.

However, for a stronger CI/CD implementation, you can make Jenkins **wait for the SonarQube Quality Gate**.

Add another stage:

```
stage('Quality Gate') {  
    steps {  
        timeout(time: 5, unit: 'MINUTES') {  
            waitForQualityGate abortPipeline: true  
        }  
    }  
}
```

The `waitForQualityGate` step waits for SonarQube to calculate the Quality Gate and can abort the Jenkins Pipeline when the gate fails.

18. Step 14 – Configure SonarQube Webhook

For waitForQualityGate to work correctly, configure a SonarQube webhook pointing to:

`http://<JENKINS-IP>:8080/sonarqube-webhook/`

The trailing / is required.

Example:

`http://3.239.40.138:8080/sonarqube-webhook/`

Use your actual Jenkins IP/domain.

In SonarQube:

Administration



Configuration



Webhooks



Create

Enter the Jenkins webhook URL.

The Jenkins documentation specifically requires this webhook for waitForQualityGate.

19. Complete Stage 1 + Stage 2 + Quality Gate Pipeline

For your documentation, the complete pipeline can be:

```
pipeline {
```

```
    agent any
```

```
    stages {
```

```
stage('Git Clone') {  
    steps {  
        git branch: 'main',  
            url: 'https://github.com/betawins/hiring-app.git'  
    }  
}
```

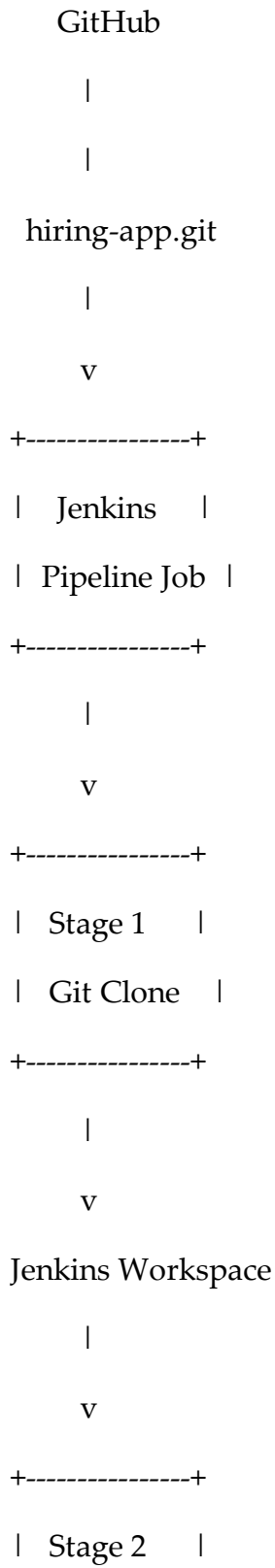
```
stage('SonarQube Analysis') {  
    steps {  
        withSonarQubeEnv('SonarQube') {  
            sh 'mvn clean verify sonar:sonar'  
        }  
    }  
}
```

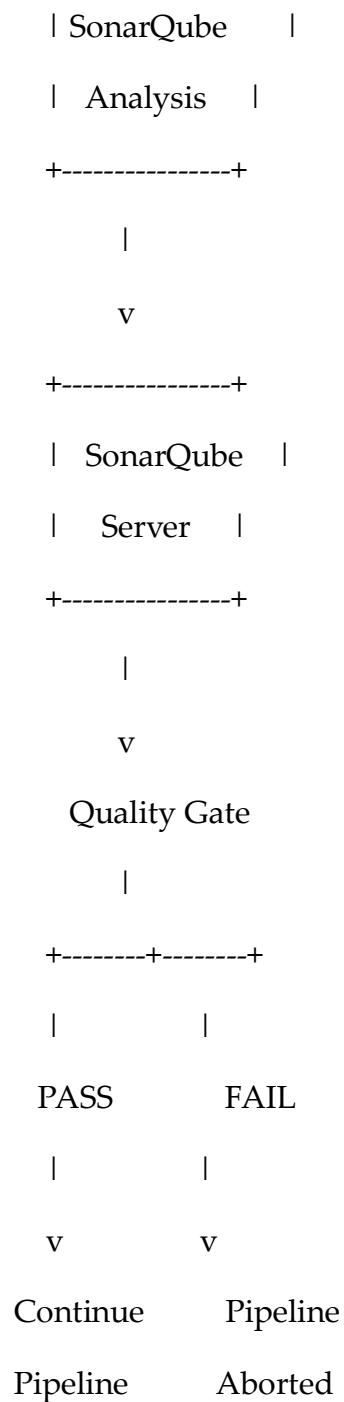
```
stage('Quality Gate') {  
    steps {  
        timeout(time: 5, unit: 'MINUTES') {  
            waitForQualityGate abortPipeline: true  
        }  
    }  
}
```

```
}  
}
```

20. Pipeline Flow

Your implementation now becomes:





21. Expected Result

If Quality Gate passes:

Git Clone SUCCESS

SonarQube Analysis SUCCESS

Quality Gate PASSED

Pipeline SUCCESS

If Quality Gate fails:

Git Clone SUCCESS

SonarQube Analysis SUCCESS

Quality Gate FAILED

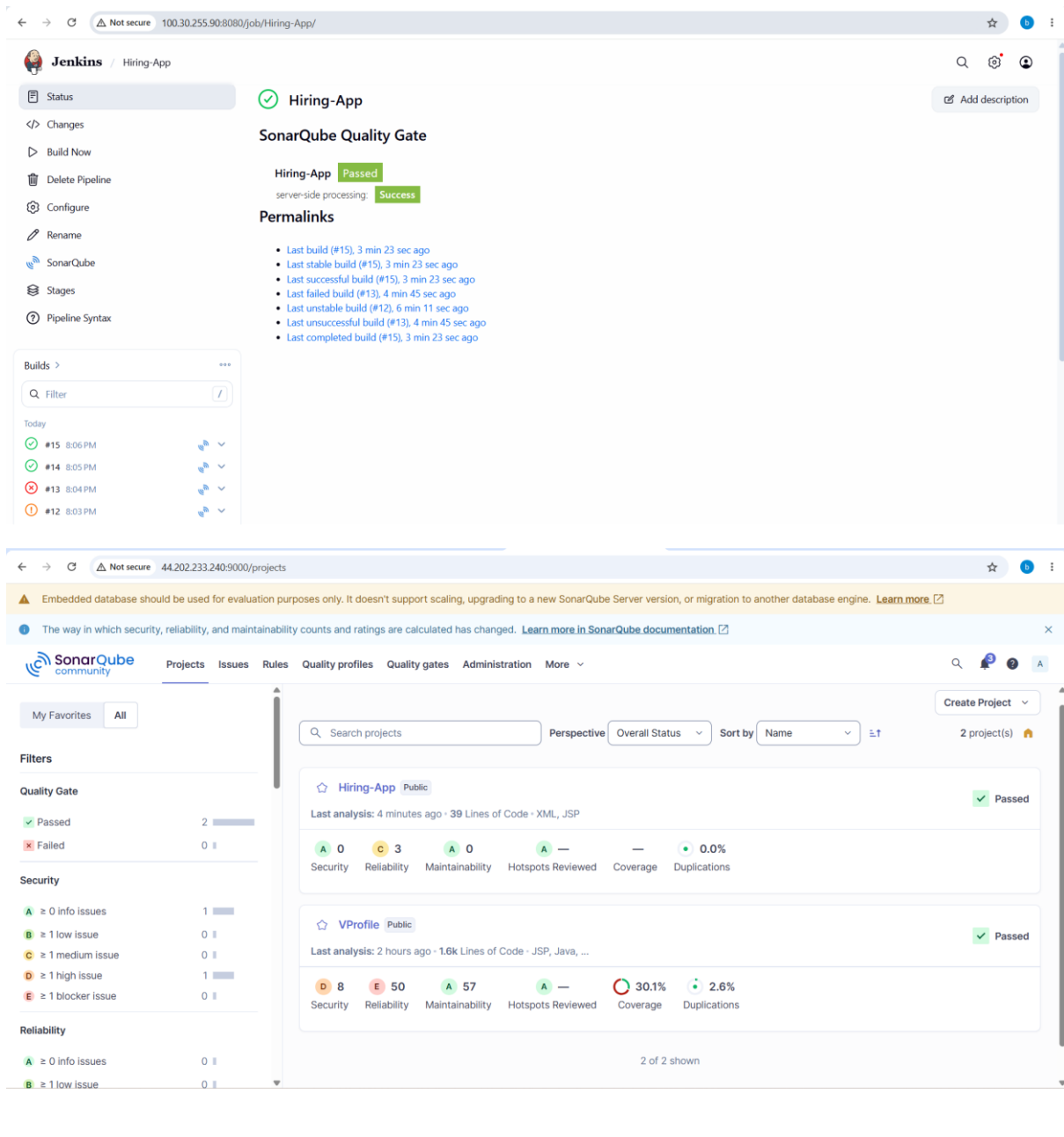
Pipeline FAILURE/ABORTED

With abortPipeline: true, Jenkins can stop the Pipeline when the Quality Gate fails.

22. Screenshots to Include in Your Documentation

For Stage 2, capture these screenshots:

1. SonarQube dashboard
2. SonarQube project creation
3. SonarQube project key/name
4. SonarQube token generation — **hide the token**
5. Jenkins SonarQube Scanner plugin installation
6. Jenkins Credentials showing sonarqube-token — **do not expose the secret**
7. Jenkins → Manage Jenkins → System → SonarQube Servers
8. Jenkins Pipeline containing SonarQube Analysis
9. Jenkins Console Output showing Maven/SonarQube analysis
10. SonarQube project analysis results
11. SonarQube Quality Gate
12. Jenkins Quality Gate result



23. Observation

The hiring-app source code downloaded from GitHub was successfully analyzed by SonarQube through the Jenkins Pipeline. SonarQube generated a code-quality report containing information about bugs, vulnerabilities, code smells, and other quality-related metrics.

24. Conclusion

Stage 2 successfully integrates **Jenkins** and **SonarQube** into the CI/CD pipeline.

Jenkins retrieves the application source code during Stage 1 and then invokes SonarQube analysis during Stage 2. The analysis results are available in SonarQube, allowing the development team to identify and resolve code-quality and security issues.

The optional **Quality Gate** stage further improves the pipeline by preventing the pipeline from continuing when the defined SonarQube quality criteria are not satisfied.

Stage 2 Summary

Item	Details
Stage	SonarQube Integration
Purpose	Analyze code quality
Source	hiring-app.git
CI Tool	Jenkins
Analysis Tool	SonarQube
Build Tool	Maven
Jenkins Step	withSonarQubeEnv
Optional Step	waitForQualityGate
Result	Code Quality Report
Quality Gate	Pass/Fail

stage3: Slack Integration to send the alerts to slack.

Source Code: <https://github.com/betawins/hiring-app.git>

Stage 3 — Jenkins → Slack Alerts

The Jenkins Slack Notification plugin provides the slackSend Pipeline step, and it supports sending messages to a specific channel.

Your final flow will be:

GitHub



Jenkins



Git Clone



SonarQube Analysis



Build Result



Slack #jenkins-alerts

1. Install Slack Notification Plugin

In Jenkins:

Manage Jenkins → Plugins → Available plugins

Search:

Slack Notification

Install the **Slack Notification** plugin.

Your Jenkins version 2.568.2 is newer than the plugin's current minimum requirement of 2.479.3, so the current plugin is compatible with your Jenkins version.

2. Configure your Slack App

You already created a Slack app from your previous work.

You need a Slack bot/app token configured for Jenkins.

In your Slack app, configure the appropriate bot permissions and install/reinstall the app into your workspace. Slack's official documentation describes creating an app and configuring incoming messaging integrations.

Make sure the Jenkins bot has access to your target channel, for example:

#jenkins-alerts

3. Add the Slack token to Jenkins Credentials

Go to:

Jenkins → Manage Jenkins → Credentials → System → Global credentials

Click:

Add Credentials

Choose:

Kind: Secret text

Secret: <your Slack token>

ID: slack-token

Description: Slack Bot Token

The Slack plugin recommends storing the integration token in Jenkins Credentials rather than exposing it directly in the Jenkinsfile.

Do not put your actual Slack token inside the Jenkinsfile.

4. Configure Slack in Jenkins

Go to:

Manage Jenkins → System

Find:

Global Slack Notifier / Slack Notifications

Configure the Slack integration using your credential.

For a custom Slack app/bot, enable the appropriate **Custom slack app bot user** option if your setup uses a custom bot. The plugin documentation specifically distinguishes custom Slack app bot mode from the default integration.

5. Use this Jenkinsfile

For your Hiring-App project, I recommend this structure:

```
pipeline {
    agent any

    stages {

        stage('Git Clone') {
            steps {
                git branch: 'main',
                    url: 'https://github.com/betawins/hiring-app.git'
            }
        }

        stage('SonarQube Analysis') {
            steps {
                withSonarQubeEnv('SonarQube') {
                    sh '''
                        mvn clean verify org.sonarsource.scanner.maven:sonar-maven-
plugin:sonar \
                        -Dsonar.projectKey=Hiring-App \
                        -Dsonar.projectName=Hiring-App
                    '''
                }
            }
        }
    }
}
```

```
    }  
  }  
}
```

```
stage('Slack Notification') {  
  steps {  
    slackSend(  
      channel: '#jenkins-alerts',  
      color: 'good',  
      message: "SUCCESS: ${env.JOB_NAME} #${env.BUILD_NUMBER}  
completed successfully. ${env.BUILD_URL}"  
    )  
  }  
}  
}
```

```
post {  
  failure {  
    slackSend(  
      channel: '#jenkins-alerts',  
      color: 'danger',  
      message: "FAILED: ${env.JOB_NAME} #${env.BUILD_NUMBER} failed.  
Check Jenkins: ${env.BUILD_URL}"  
    )  
  }  
  
  success {
```



```

    slackSend(
        channel: '#jenkins-alerts',
        color: 'good',
        message: "SUCCESS: ${env.JOB_NAME} #${env.BUILD_NUMBER}
completed successfully. ${env.BUILD_URL}"
    )
}
}
}

```

Important correction

I would actually **remove the separate Slack Notification stage** from the above and use only the post section.

That way:

- successful pipeline → Slack success alert
- failed pipeline → Slack failure alert
- you don't accidentally send two success messages

Use this cleaner version:

```

pipeline {
    agent any

    stages {

        stage('Git Clone') {
            steps {
                git branch: 'main',
                    url: 'https://github.com/betawins/hiring-app.git'
            }
        }
    }
}

```

```

    }

    stage('SonarQube Analysis') {
        steps {
            withSonarQubeEnv('SonarQube') {
                sh '''
                    mvn clean verify org.sonarsource.scanner.maven:sonar-maven-
plugin:sonar \
                    -Dsonar.projectKey=Hiring-App \
                    -Dsonar.projectName=Hiring-App
                '''
            }
        }
    }
}

post {
    success {
        slackSend(
            channel: '#jenkins-alerts',
            color: 'good',
            message: "SUCCESS: ${env.JOB_NAME} #${env.BUILD_NUMBER}
completed successfully. ${env.BUILD_URL}"
        )
    }

    failure {

```

```

    slackSend(
        channel: '#jenkins-alerts',
        color: 'danger',
        message: "FAILED: ${env.JOB_NAME} #${env.BUILD_NUMBER} failed.
Check Jenkins: ${env.BUILD_URL}"
    )
}
}
}

```

The slackSend Pipeline step officially supports channel, message, color, and credential-based authentication.

6. Expected Slack message

When Jenkins succeeds, your #jenkins-alerts channel should receive something like:

SUCCESS: Hiring-App #4 completed successfully.

<http://<jenkins-server>/job/Hiring-App/4/>

If the build fails:

FAILED: Hiring-App #5 failed.

Check Jenkins: <http://<jenkins-server>/job/Hiring-App/5/>

Your Stage 3 checklist

- Create/use #jenkins-alerts
- Configure Slack app/bot
- Give the bot access to #jenkins-alerts
- Install Jenkins **Slack Notification** plugin
- Store Slack token in Jenkins Credentials
- Configure Slack integration in Jenkins
- Update Jenkinsfile
- Commit and push to GitHub

- Run Jenkins build
- Verify message in #jenkins-alerts

Jenkins

Hiring-App

Status

Changes

Build Now

Delete Pipeline

Configure

Rename

SonarQube

Stages

Pipeline Syntax

Hiring-App

SonarQube Quality Gate

Hiring-App Passed

server-side processing: Success

Permalinks

- Last build (#15), 3 min 23 sec ago
- Last stable build (#15), 3 min 23 sec ago
- Last successful build (#15), 3 min 23 sec ago
- Last failed build (#13), 4 min 45 sec ago
- Last unstable build (#12), 6 min 11 sec ago
- Last unsuccessful build (#13), 4 min 45 sec ago
- Last completed build (#15), 3 min 23 sec ago

Builds >

Filter

Today

#15 8:06 PM

#14 8:05 PM

#13 8:04 PM

#12 8:03 PM

#11 8:02 PM

Jenkins

+ New Item

Build History

Build Queue

No builds in the queue.

Build Executor Status

(0 of 2 executors busy)

All

S	W	Name	Last Success	Last Failure	Last Duration
		Hiring-App	2 min 49 sec #15	4 min 11 sec #13	16 sec
		VProfile-Pipeline	2 hr 13 min #28	2 hr 18 min #24	49 sec

Icon: S M L

REST API Jenkins 2.568.2

Not secure

100.30.255.90:8080/job/Hiring-App/14/console

☆

b

 Jenkins

Hiring-App

#14

[INFO] SonarScanner Engine completed successfully

[INFO] -----

[INFO] BUILD SUCCESS

[INFO] -----

[INFO] Total time: 15.738 s

[INFO] Finished at: 2026-08-14T20:06:13Z

[INFO] -----

[Pipeline] }

[Pipeline] // withSonarQubeEnv

[Pipeline] }

[Pipeline] // stage

[Pipeline] stage

[Pipeline] { (Declarative: Post Actions)

[Pipeline] slackSend

Slack Send Pipeline step running, values are - baseUrl: <empty>, teamDomain: rajkumaridevops, channel: #jenkins-alerts-notification, color: good, botUser: true, tokenCredentialId: slack-token, notifyCommitters: false, iconEmoji: <empty>, username: <empty>, timestamp: <empty>

[Pipeline] }

[Pipeline] // stage

[Pipeline] }

[Pipeline] // node

[Pipeline] End of Pipeline

Finished: SUCCESS

Jenkins 2.568.2

← → ↻ ⚠ Not secure 100.30.255.90:8080/job/Hiring-App/ ☆ b ⋮

 **Jenkins** / Hiring-App 🔍 ⚙️ 👤

Builds > 🔍 Filter /

Today

✓ #15	8:06 PM		✖	⌵
✓ #14	8:05 PM		🔊	⌵
✖ #13	8:04 PM		🔊	⌵
⚠ #12	8:03 PM		🔊	⌵
✖ #11	8:02 PM		🔊	⌵
✖ #10	8:01 PM		🔊	⌵
✓ #9	7:57 PM		🔊	⌵
✓ #8	7:57 PM		🔊	⌵
✓ #7	7:53 PM		🔊	⌵
✓ #6	7:46 PM		🔊	⌵
✓ #5	7:43 PM		🔊	⌵
✓ #4	7:40 PM		🔊	⌵
✖ #3	7:27 PM		🔊	⌵
✓ #2	7:07 PM			⌵
✖ #1	7:06 PM			⌵

✖ **Hiring-App**

RajkumarDevOps ⚙️ 👤

Set up your space 🔍 Find a conversation... 📁 Huddles 📁 Directories 📁 Activity 📁 Files ⋮ More ⚙️ Admin

📁 Channels

- all-rajkumardevops
- jenkins-alerts-notification
- new-channel
- social

✚ Add channels

📁 Direct messages

- brajkumar17675
- rajkumar1-you

✚ Invite people

📁 Apps

- RajStack
- Slack
- Google Calendar
- Google Drive

✚ Connect apps

☆ #jenkins-alerts-notification 🔍 Invite teammates 📄 Huddle 🔍 ⋮

📁 Messages 📄 Add canvas +

Yesterday

- SUCCESS: VProfile-Pipeline #27
- SUCCESS: VProfile-Pipeline #28

Today

 **RajStack** 1:27 AM

- SUCCESS: Hiring-App #8 completed successfully. Jenkins URL: <http://100.30.255.90:8080/job/Hiring-App/8/>
- SUCCESS: Hiring-App #9 completed successfully. Jenkins URL: <http://100.30.255.90:8080/job/Hiring-App/9/>
- FAILED: Hiring-App #10. Please check Jenkins Console Output: <http://100.30.255.90:8080/job/Hiring-App/10/>
- FAILED: Hiring-App #11. Please check Jenkins Console Output: <http://100.30.255.90:8080/job/Hiring-App/11/>
- UNSTABLE: Hiring-App #12. Jenkins URL: <http://100.30.255.90:8080/job/Hiring-App/12/>
- FAILED: Hiring-App #13. Please check Jenkins Console Output: <http://100.30.255.90:8080/job/Hiring-App/13/>
- SUCCESS: Hiring-App #14 completed successfully. Jenkins URL: <http://100.30.255.90:8080/job/Hiring-App/14/>
- SUCCESS: Hiring-App #15 completed successfully. Jenkins URL: <http://100.30.255.90:8080/job/Hiring-App/15/>

🔍 ⚙️ 📄 📄 📄 📄 📄 📄

Message #jenkins-alerts-notification

✚ 📄 📄 📄 📄 📄 📄

Huddles notes and AI assistant apps are turned on for RajkumarDevOps. [Learn more](#)

<https://rajkumardevops.slack.com/archives/C0BP0VCCU1NR/p1786737442467609>

jenkins-alerts-notification (Ch... X +

Ask Gemini

app.slack.com/client/T0BR5F8S48Y/C0BPVQCCUNB?entry_point=default_oauth&utm_source=chatgpt.com

RajkumariDevOps

Set up your space

Find a conversation...

Huddles

Directories

Starred

Drag and drop important stuff here

Channels

all-rajkumari-devops

jenkins-alerts-notification

new channel

social

Add channels

Direct messages

rajkumari7675

rajkumari - you

invite people

Apps

RajStack

Slack

Google Calendar

Google Drive

Connect apps

jenkins-alerts-notification

Messages Add canvas +

Yesterday

SUCCESS: VProfile-Pipeline #27

SUCCESS: VProfile-Pipeline #28

Today

RajStack app 1:27 AM

SUCCESS: Hiring-App #8 completed successfully. Jenkins URL: <http://100.30.255.90:8080/job/Hiring-App/8/>

SUCCESS: Hiring-App #9 completed successfully. Jenkins URL: <http://100.30.255.90:8080/job/Hiring-App/9/>

FAILED: Hiring-App #10. Please check Jenkins Console Output: <http://100.30.255.90:8080/job/Hiring-App/10/>

FAILED: Hiring-App #11. Please check Jenkins Console Output: <http://100.30.255.90:8080/job/Hiring-App/11/>

UNSTABLE: Hiring-App #12. Jenkins URL: <http://100.30.255.90:8080/job/Hiring-App/12/>

FAILED: Hiring-App #13. Please check Jenkins Console Output: <http://100.30.255.90:8080/job/Hiring-App/13/>

SUCCESS: Hiring-App #14 completed successfully. Jenkins URL: <http://100.30.255.90:8080/job/Hiring-App/14/>

SUCCESS: Hiring-App #15 completed successfully. Jenkins URL: <http://100.30.255.90:8080/job/Hiring-App/15/>

Message #jenkins-alerts-notification

Huddles notes and AI assistant apps are turned on for RajkumariDevOps. [Learn more](#)